

pySurvAnalysis — User Guide

This guide is the in-app manual (Help in the Analysis Hub, F1 or any ? button), exported as one document. Generated by `python -m pysurvanalysis.help.export` — edit the pages in `pysurvanalysis/help/topics/`, not this file.

Contents

Getting started

- [What pySurvAnalysis does](#)
- [Installing and launching](#)
- [Your first analysis](#)
- [Batches, Projects, Experiments and Focuses](#)
- [The Analysis Hub](#)
- [Where results are written](#)

Data and configuration

- [Input data formats](#)
- [survival_config.yaml reference](#)
- [The input: section](#)
- [The global: section](#)
- [Censoring policy](#)
- [The omit: section — leaving analyses and figures out](#)
- [project.yaml and Project Defaults](#)
- [Experiment Types](#)
- [Validating configuration](#)

Projects and batches

- [Creating and opening a Project](#)
- [The Experiments card — members](#)
- [Project actions — reports, figures, Plot Editor](#)
- [Project Scripts](#)
- [Batch runs](#)
- [The Batch preflight](#)
- [The Experiment panel](#)

Focuses

- [What a Focus is](#)
- [The Focus window](#)
- [Reference Levels](#)
- [Focus Shape — what a Focus is offered](#)
- [Not Applicable and Left Out](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)

- [Defined Plots](#)

Quality control

- [The QC panel](#)
- [Exclusion Groups](#)
- [The Chamber QC viewer](#)

Statistical analyses

- [The Analyze panel and Run analysis](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [Summary statistics, median and mean survival](#)
- [Log-rank pairwise](#)
- [Log-rank omnibus](#)
- [Gehan-Wilcoxon pairwise](#)
- [Pairwise hazard ratios](#)
- [Parametric AFT models](#)
- [Interaction analyses \(Factorial Battery\)](#)
- [The Cox factorial model](#)
- [The RMST factorial model](#)
- [Checking proportional hazards](#)
- [Reading p-values and multiple comparisons](#)

Figures

- [The Plots panel and Generate plots](#)
- [Kaplan-Meier curves](#)
- [KM curves with at-risk table](#)
- [Nelson-Aalen cumulative hazard](#)
- [Log-log diagnostic](#)
- [Cumulative deaths](#)
- [Hazard rate](#)
- [Smoothed hazard](#)
- [Mortality \(qx\)](#)
- [Number at risk](#)
- [Lifespan distribution](#)
- [Hazard-ratio forest](#)
- [Faceted Kaplan-Meier](#)
- [Lifespan interaction plot](#)
- [The Plot Editor](#)
- [Plot Editor — Figure](#)
- [Plot Editor — Canvas & type](#)
- [Plot Editor — Curves & points](#)

- [Plot Editor — Panels & legend](#)
- [Plot Editor — Colours](#)
- [Publication Figures, Specs and Styles](#)

Scripts

- [Experiment and Project Scripts](#)
- [Running Experiment Scripts](#)
- [The Script Editor](#)
- [Script action reference](#)

Reports

- [The experiment report](#)
- [The Run Summary](#)
- [The Project Report](#)
- [The AI narrative](#)

Reference

- [Glossary](#)
- [Command-line use](#)
- [Troubleshooting](#)

Getting started

What pySurvAnalysis does

pySurvAnalysis analyses lifespan experiments: cohorts of individuals (flies in vials or chambers, for example) scored at each census until death. It reads a DLife census workbook (.xlsx) or a CSV/TSV cohort, builds life tables and Kaplan-Meier survivorship, compares treatments with the usual tests (log-rank, Gehan-Wilcoxon, Cox hazard ratios, parametric AFT fits and, for designs that cross two or more factors, Cox and RMST interaction models), draws the standard figures, and writes a PDF report for every analysis it runs.

Everything the app does can also be run headless from the command line, and the same settings drive both, so a report made in the Hub and one made by an unattended Batch Run are the same report.

The ideas you need

- **Experiment Directory** — one data file's folder: `survival_config.yaml` at its root, the workbook or CSV at the root or in `data/`, results in `analysis/<focus>/`, quality-control state in `qc/`.
- **Focus** — a named slice of the factors and levels found in the data file (for example *Females only, 20x vs 40x density*). Each Focus is analysed on its own, and every output file carries its name. A config with no Focuses has one anyway, called `Unfiltered`.
- **Project** — a folder with a `project.yaml` whose subfolders are **Member Experiments**: separate experiments asking one question in slightly different ways. A Project binds their results into one Project Report but **never pools them**.
- **Batch** — any folder with Projects somewhere beneath it, used to run a script in every Project unattended.

- **Exclusion Group** — a named set of chambers removed from analysis after quality control. The active group is written into the config and stamped on every result.

[Batches, Projects, Experiments and Focuses](#) explains how these fit together.

What it will not do

- It never pools members of a Project, or Focuses within one file, into a combined statistic. The Project Report puts independent sections side by side.
- It never analyses silently on your behalf. A report built from saved results says "not analysed" or "out of date" rather than quietly re-running anything.
- It never drops an analysis without saying so. An analysis the data cannot support is recorded as **Not Applicable**, and one you unticked is recorded as **Left Out**. Both appear in the log, the Run Summary and the report.

The main window

The **Analysis Hub** is the app's main window. Along the top runs a strip of three tiles, **Batch · Project · Experiment**, with a status readout beside them. Below is the Output log, and figures open beside it as tabs. Clicking a tile opens its panel of controls. See [The Analysis Hub](#).

The Hub opens with nothing selected. Use **Project ► Open project...** (or **Recent** in the top bar) to choose a folder.

Getting help in the app

- **Help** in the Hub's top bar, or **F1**, opens this manual at the page for whichever panel is open.
- A **?** button beside a card, control or checkbox opens the page that explains it.
- Most buttons have a tooltip. Hover to read what the button does, and why it is disabled when it is.
- In the manual, use the contents tree or the search box (Ctrl+F) on the left, and **◀ Back** / **Forward ►** to retrace your steps.

Where to start

New users should follow [Your first analysis](#), which walks through the real UI from an empty Project to a finished report.

See also

- [Your first analysis](#)
- [Batches, Projects, Experiments and Focuses](#)
- [The Analysis Hub](#)
- [Where results are written](#)
- [Glossary](#)

Installing and launching

pySurvAnalysis is a Python application with a Qt interface. It runs on Windows, macOS and Linux, needs **Python 3.13 or later**, and is installed into its own environment with [uv](#). No separate Qt installation is needed: PyQt6 brings its own.

Installing

1. Install uv once, if you do not have it:

```
## Windows (PowerShell)
```

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex

## macOS / Linux
curl -Lsf https://astral.sh/uv/install.sh | sh
```

2. Get the code and create the environment. From the folder you want it in:

```
git clone https://github.com/PletcherLab/pySurvAnalysis
cd pySurvAnalysis
uv sync
```

`uv sync` creates a `.venv` folder inside the repository and installs everything the app needs: lifelines (Kaplan-Meier, log-rank, Cox models), statsmodels (the RMST regressions), scipy, numpy and pandas, matplotlib and plotnine (figures), openpyxl (DLife workbooks), reportlab (PDF reports), PyQt6 with its theme and icon packages, PyYAML, and the anthropic and openai client libraries used only by the optional [AI narrative](#).

Updating

After pulling a newer version (`git pull`), run `uv sync` again so the environment matches. `uv sync --upgrade` moves every package to its newest compatible release.

Launching

From the repository folder:

Command	Opens
<code>uv run pysurv</code>	The Analysis Hub (also <code>uv run pysurv-hub</code>)
<code>uv run pysurv-hub path/to/Project</code>	The Hub with that Project, Batch folder or Experiment Directory already selected
<code>uv run pysurv-qc path/to/experiment</code>	The Chamber QC viewer on one experiment
<code>uv run pysurv-plots path/to/experiment</code>	The Plot Editor on one experiment
<code>uv run python main.py</code>	The Hub (the same program, through the script in the repository)

The Hub opens with nothing selected unless a path is given: choose **Open project...** on the Project panel, or **Recent** in the top bar. Everything else — analysis, figures, reports, scripts, batches — is done from the Hub. The same work can also be run without the interface; see [Command-line use](#).

The manual

This manual is part of the app. Open it with **Help** in the Hub's top bar or with **F1** in any window (it opens at the page for whatever you are looking at), or with any small **?** button beside a control. A copy of the whole manual is kept in the repository as `doc/user_guide.md`.

See also

- [Your first analysis](#)
- [Command-line use](#)
- [The Analysis Hub](#)
- [Troubleshooting](#)

Your first analysis

This walk-through takes you from an empty folder to a finished report, using the Analysis Hub. It assumes you have one DLife census workbook (.xlsx). Each step names the panel, the card and the button you click. Panels open when you click a tile in the strip across the top, and close again when you click the tile a second time, press Esc, or click elsewhere in the window.

1. Open or create a Project

Click the **Project** tile. The first card, **Create/Load**, has the ways in:

- **Open project...** if you already have a folder with a `project.yaml`.
- **Create project...** to start a new one. In the dialog, set **Directory** to the folder the Project should be (it is created if it does not exist), give it a **Project name** (it defaults to the folder name) and a one-line **Question**, which appears on the Project Report's cover. Leave **Experiment type** at *Standard Lifespan* and check the **Global** settings (`time_unit`, `time_label`, `assume_censored` and `min_n_per_chamber`). These are Project Defaults that every member inherits unless it states its own. Click **Save**.
- **Initialize existing directory...** if you already have a folder of experiment subfolders and want it to become a Project in place.

The Project is now open. The Create/Load card shows its name, type and member count, and the **Experiments** card below it becomes usable. You do not need a Project to analyse one workbook (opening a folder that holds a `survival_config.yaml` loads it directly as a standalone experiment), but a Project is the usual way to work. See [Creating and opening a Project](#).

2. Add a member

On the **Experiments** card, under *Bring one in from outside*, click **Add experiment...** and choose your workbook. The app checks that it is a valid DLife workbook (Design and RawData sheets, with at least one factor column after `StartTime`), then:

- creates a member folder named after the file (`cohort_a.xlsx` becomes `cohort_a/`),
- puts the workbook in that folder's `data/` (copied from outside the Project, or moved if it was already loose inside the Project), and
- writes a minimal `survival_config.yaml` that inherits everything else from the Project Defaults.

The member appears in the members table with **Config** = `yes` and **Analysed** = `no`. The other ways to add a member are described in [The Experiments card — members](#).

3. Load the member

Double-click its row in the members table. The member loads, the status readout shows it on its *Experiment* row, and the **Experiment** panel opens on its own. Everything you do from here applies to this member.

4. Choose or create a Focus

The **Focus** card at the top of the Experiment panel says which slice of the data the next steps act on, the **Active Focus**. A new member has one Focus, *Unfiltered*: every factor found in the Design sheet, at every level. The line under the selector describes the Focus (its varying factors and levels, the Reference Level of each, any filters and anything pooled over), its shape (for example 2×2), and which comparisons and factorial analyses it offers.

- To analyse everything together, keep *Unfiltered*.

- To analyse a subset, click **New....** The Focus window opens with a new Focus. Name it, untick the factors you want to pool over, tick the levels to keep (one level makes the factor a filter), choose each factor's Reference Level, then click **Save**. The new Focus becomes the Active Focus. See [The Focus window](#).

The selector lists each Focus with its state: not analysed, analysed, out of date or blocked.

5. Quality control

Click the **QC** sub-tile in the Experiment panel.

1. Click **Chamber QC viewer....** It draws every chamber's survival curve, panel by panel per treatment. Click a curve to flag that chamber for exclusion, then use **Save Exclusions...** to save the flagged chambers under a group name (for example default).
2. Back in the Hub's QC panel, pick that group in **Active group** and click **Set active group**. This writes `exclusions: {group: ...}` into `survival_config.yaml`, so every later run drops those chambers and records which group did it. If the new group is not in the list yet, type its name into the box (it accepts typing).

If nothing needs excluding, skip this step. See [The QC panel](#).

6. Tick the analyses and run them

Click the **Analyze** sub-tile. There is one checkbox for each analysis the Active Focus is offered (log-rank pairwise, log-rank omnibus, Gehan-Wilcoxon pairwise, pairwise hazard ratios, parametric AFT models and, when the Focus varies two or more factors, *Interaction analyses*). Untick anything you do not want. The ticks are saved in the config's `omit: block`, so scripts and Batch Runs leave out the same things. A greyed box is **Not Applicable**: the Focus asks the question but the data cannot answer it. Hover over the box to see why.

Click **Run analysis**. Progress streams into the **Output** tab, starting with ► and ending with ✓ (done) or ✗ (failed). The results, the figures and the report go to `analysis/<focus>/` inside the member's folder. See [The Analyze panel and Run analysis](#).

7. Generate plots

Click the **Plots** sub-tile. There is one checkbox per figure in the Focus's Plot Set. Tick the ones you want, then click **Generate plots**. Each figure opens as a tab beside **Output** and is saved to `analysis/<focus>/plots/`. Run analysis already draws the ticked figures into the report. Generate plots is for looking at them in the Hub. See [The Plots panel and Generate plots](#).

8. Read the report

Each run writes `<member>_report_<focus>.pdf` (plus a Markdown `report_<focus>.md`) in `analysis/<focus>/`. To open it, go back to the **Project** tile and click **View reports** on the **Actions** card. It opens the Project Report, if there is one, and every member report in your PDF viewer. To bind all members' Focuses into one document, click **Project report** first. It writes `<project>_report.pdf` in the Project folder from the saved results, without re-analysing anything.

To read a report, see [The experiment report](#) and [The Project Report](#).

Next steps

- Add more members, and run every one in a single step with the Project's batch script: [Project Scripts](#).
- Curate journal-ready figures in the [Plot Editor](#).

See also

- [Batches, Projects, Experiments and Focuses](#)
- [The Analysis Hub](#)
- [The Experiment panel](#)
- [What a Focus is](#)
- [Where results are written](#)
- [Troubleshooting](#)

Batches, Projects, Experiments and Focuses

pySurvAnalysis organises work in four nested levels. Each level is a folder, or a named slice inside a folder's data, and each has one job. Knowing which level you are on tells you what a button acts on and where its results go.

```
Batch/                                any folder with Projects beneath it (batch.yaml opti
└─ Sept2026/                          grouping folders are transparent
    └─ ProjA/                          a Project: project.yaml
        └─ plot_specs.yaml             the Project's Plot Specs and Styles
            └─ cohort_a/               a Member Experiment (an Experiment Directory)
                └─ survival_config.yaml
                    └─ data/cohort_a.xlsx
                        └─ qc/remove_chambers.csv
                            └─ analysis/
                                └─ Unfiltered/    one folder per Focus
                                    └─ FemalesOnly/
```

Experiment Directory

One data file's folder. It holds `survival_config.yaml` at its root, the input workbook or CSV either at the root or in `data/`, results in `analysis/<focus>/`, QC state in `qc/` and Publication Figures in `figures/`. The data file may contain several experiments, and the directory holds them all. Its Focuses tell them apart.

An Experiment Directory can stand alone with no Project above it. Opening it in the Hub loads it directly.

Focus

A **Focus** is a named selection of the factors and levels **discovered** in the data file. You never declare factors: they are read from the Design sheet's columns after `StartTime` (or a CSV's factor columns), with levels in the order they first appear. Each discovered factor plays one of three roles in a Focus:

- named with **two or more levels**, it is **varying**, and its levels label the treatments;
- named with **one level**, it is a **filter** (keep only that level);
- **not named**, it is **pooled over**.

The Focus is the unit of analysis. Each Focus is analysed independently and writes to its own `analysis/<focus>/` folder, and every file in that folder carries the Focus's name as a suffix (`run_summary_<focus>.json`, for example). A config with no `focuses:` block has one Focus anyway, `Unfiltered` (every factor at every level), which is written into the file the first time it is analysed or edited.

What a Focus is offered depends on its **Focus Shape**. Comparisons need two or more treatments, and the interaction analyses need two or more varying factors. See [What a Focus is](#)

and [Focus Shape](#).

Project and Member Experiments

A **Project** is a folder with a `project.yaml` at its root. Its immediate subfolders that hold a `survival_config.yaml` are its **Member Experiments**: separate experiments addressing one question, often with different designs.

- **A Project never pools.** There is no combined analysis or pooled statistic. The Project Report binds one independent section per Focus of every member.
- **Project Defaults** (the `defaults:` section of `project.yaml`) are a seed, not a rule. A member that says nothing about a setting (time unit, censoring policy, input settings, Exclusion Group name) inherits it. A member that states its own value keeps it. Focuses are never inherited.
- **The Experiment Type is the one rule.** Every member must share the Project's Experiment Type. Today there is one, Standard Lifespan.
- When members differ in data-source settings (censoring, active Exclusion Group, time unit), the difference is allowed but declared. The Project Report carries a **Divergence Note**.

A Project also holds the Project Scripts (including the `batch` script every new Project is created with), central Experiment Scripts that serve every member, and `plot_specs.yaml` for Publication Figures.

Batch

A **Batch** is any folder with at least one Project somewhere beneath it, at any depth. It exists only to run a Project Script in many Projects unattended. It holds no analysis and never combines results. Discovery stops at each Project and never looks inside it. Each Project in a Batch is a **Batch Project**, named by its path relative to the Batch folder (`Sept2026/ProjA`). An optional `batch.yaml` can hold central Project Scripts and a designated script. See [Batch runs](#).

Blocked, Not Applicable and Left Out

The app separates *what cannot run* from *what you chose not to run*:

- A **Blocked Member** is a member folder a run cannot use as it stands: data but no config, a config but no data, or several candidate data files and no `data_file:` naming one. The other members still run.
- A **Blocked Focus** names a factor or level the data no longer has, or keeps a cell that the active Exclusion Group empties. The member's other Focuses still run.
- **Not Applicable** means an analysis is relevant to the Focus but the data cannot compute it. It is recorded in the log, the Run Summary and the report.
- **Left Out** means you unticked the analysis or figure on the Analyze or Plots panel. It is recorded too, separately from Not Applicable.

See [Not Applicable and Left Out](#).

How this maps onto the Hub

The Hub's three tiles are the three folder levels: **Batch**, **Project** and **Experiment**. The Focus is chosen at the top of the Experiment panel and applies to everything below it (QC, Analyze, Plots, Scripts, AI). Selecting a folder sets the working level. Double-clicking a member in the Project panel loads that member.

See also

- [The Analysis Hub](#)
- [What a Focus is](#)
- [project.yaml and Project Defaults](#)
- [Batch runs](#)
- [Where results are written](#)
- [Glossary](#)

The Analysis Hub

The Analysis Hub is pySurvAnalysis's main window. It has a top bar, a strip of three tiles with a status readout, and a full-width output area below. Each tile shows only live status. Its controls live in a panel that drops down under it when you click. Only one panel is open at a time.

You can launch the Hub with a folder already selected by passing its path on the command line (`pysurvanalysis hub <path>`). Otherwise it opens with nothing selected.

Top bar

- **Recent**: a menu of folders you have opened before. Picking one selects it again.
- **Help**: opens this manual at the page for the panel that is open (or this page when none is). **F1** does the same.
- The **theme** button toggles light and dark.

The tile strip

Tile	Opens	Its summary shows
Batch	Batch runs	the number of Projects found and how many members are blocked, or a hint when the selection is not a Batch
Project	the Project panel (Create/Load , Experiments , Actions , Scripts)	member count, how many of the members' Focuses have current results (for example <code>3 member(s) · 4/5 Focuses analysed</code> — Out of Date results are not counted), and which member is loaded
Experiment	The Experiment panel	the loaded experiment and its Active Focus

The Batch and Project tiles are never dimmed. Their panels hold the pickers that create everything else, so they are always available. The **Experiment** tile is dimmed until an experiment is loaded, and clicking it then does not open a panel. Instead the Output log says *Load an experiment first — double-click a member in the Project panel, or open a standalone Experiment Directory*.

The five experiment-level surfaces (**QC · Analyze · Plots · Scripts · AI**) are sub-tiles inside the Experiment panel, under the Focus selector. Each opens its own panel, anchored under the Experiment tile.

The status readout

To the right of the tiles, a readout answers "what is open right now" without opening a panel. Its rows are **Selection** (the folder you picked), **Project** (name and Experiment Type), **Experiment** (the loaded member, or *none loaded*), **Focus** (the Active Focus and a one-line description of it,

once an experiment is loaded) and **Question** (when the Project has one). At most four rows are shown, so with an experiment loaded the Question moves to the tooltip and the Focus row always stays in view. Hover over the readout to see every row.

What the selection can be

Whatever you open (with **Open project...**, **Choose batch folder...** or **Recent**) is classified:

- a folder with a `project.yaml` is a **Project**: the members table fills, and you load a member by double-clicking it;
- a folder with a `survival_config.yaml` is a **standalone Experiment Directory** and loads at once. If its parent folder is a Project, that Project is opened alongside it;
- any other folder with Projects somewhere beneath it is a **Batch**;
- anything else is just a directory, and the Project tile then suggests *Create / Initialize*.

The Output log records what the selection was taken to be (Project: ..., Batch: ..., and so on).

Panels

A panel opens under its tile and closes when you click the same tile again, open another tile, press **Esc**, or click anywhere outside the panel in the Hub window. Clicking in another window (a dialog, the QC viewer, the Script Editor) does not close it. Some actions move you to a panel: double-clicking a Batch row opens the Project panel, and double-clicking a member opens the Experiment panel.

Cards whose controls have nothing to act on yet are **dimmed**, but stay usable. In the Project panel, the Experiments, Actions and Scripts cards dim until a Project is open. The QC, Analyze, Plots and Scripts panels dim until an experiment is loaded, and the AI panel dims until a Project is open and an AI provider key is configured.

The output area

Below the strip is a tabbed dock:

- **Output**, the first tab, which cannot be closed: the log of everything the Hub does. A task starts with a ► <name> line, streams its progress, and ends with ✓ (completed) or ✗ (failed, with the error above it). Lines beginning ! are warnings, and most are also shown in a message box.
- **Figure tabs**: each figure a task produces opens as its own tab (a zoomable image). Figure tabs can be closed individually and dragged to reorder.
- At the top right, **Clear plot tabs** closes every figure tab and returns to Output, and **Clear output** erases the log.

The Hub runs **one task at a time**. Starting a second while one is running gives *A task is already running*. When a task finishes, every table and panel refreshes from disk.

If something goes wrong that no action anticipated, the Hub does not close: the Output log shows ✗ *Unexpected error — the action was abandoned, the Hub carries on*: with the error's details (also printed to the console), and you can keep working. A configuration file that is not valid YAML is reported the same way — in a message naming the file and line — wherever the Hub reads it.

During a Batch Run the **Suppress plot tabs during batch runs** box on the Batch panel (on by default) stops figure tabs from opening. Figures are still written to disk. It never suppresses figures you ask for on one experiment.

See also

- [Your first analysis](#)
- [Batches, Projects, Experiments and Focuses](#)
- [The Experiment panel](#)
- [Creating and opening a Project](#)
- [Batch runs](#)

Where results are written

Every analysis is run under a **Focus**, and everything it produces goes into that Focus's own folder, `analysis/<focus>/`, with the Focus's name at the end of every file name. Two Focuses of the same experiment never overwrite each other. A figure copied out of its folder still says which slice of the data it shows. Nothing is written outside the experiment directory except the Project-level files described at the end.

`<focus>` below is the Focus name made safe for file names: any run of characters other than letters, digits, `.`, `_` and `-` becomes `_`. A Focus called `Crowding 20 v 40` writes to `analysis/Crowding_20_v_40/`. The default Focus writes to `analysis/Unfiltered/`.

An Experiment Directory

```
cohort_a/
├── survival_config.yaml           the experiment's configuration
├── data/
│   └── cohort_a.xlsx             the data file (or at the directory root)
├── qc/
│   └── remove_chambers.csv       Exclusion Groups
├── analysis/
│   └── <focus>/                 one folder per Focus – see below
├── figures/
│   └── <focus>/                 rendered Publication Figures
└── plot_specs.yaml              Publication Figure specs (standalone experiment)
```

Inside the analysis/*focus* folder

```
analysis/Crowding20v40/
├── run_summary_Crowding20v40.json
├── cohort_a_report_Crowding20v40.pdf
├── report_Crowding20v40.md
├── report_Crowding20v40_figures/  images used by the Markdown report
├── plots/
├── data_output/
└── statistics/
```

File	What it is
<code>run_summary_<focus>.json</code>	The Run Summary : counts, the Focus's definition, the Exclusion Group and chambers removed, figures written, Not Applicable and Left Out items, the omnibus test, factorial model headlines. The Hub and the Project Report read this instead of re-analysing.
<code><experiment>_report_<focus>.pdf</code>	The experiment report for this Focus.
<code>report_<focus>.md</code>	The same report as Markdown. Its images are in <code>report_<focus>_figures/</code> .

plots/ (PNG, 150 dpi)

File	Figure
km_with_risk_table_<focus>.png	KM curves with at-risk table
kaplan_meier_<focus>.png	Kaplan-Meier curves
cumulative_events_<focus>.png	Cumulative deaths
survival_distribution_<focus>.png	Lifespan distribution
mortality_qx_<focus>.png	Mortality (qx)
hazard_rate_<focus>.png	Hazard rate
smoothed_hazard_<focus>.png	Smoothed hazard
nelson_aalen_<focus>.png	Nelson-Aalen cumulative hazard
number_at_risk_<focus>.png	Number at risk
hazard_ratio_forest_<focus>.png	Hazard-ratio forest (2+ treatments)
log_log_diagnostic_<focus>.png	Log-log diagnostic (2+ treatments)
km_faceted_<focus>.png	Faceted Kaplan-Meier (Focus varies 2+ factors)
interaction_lifespan_<focus>.png	Lifespan interaction plot (Focus varies 2+ factors)
defined_<plot name>_<focus>.png	A Defined Plot drawn under this Focus

Only figures that were drawn are present. A figure that is Left Out, Not Applicable, or not offered to this Focus has no file, and a copy left by an earlier run is deleted. The same goes for a Defined Plot's figure that this run did not draw.

data_output/ (CSV)

File	Contents
individual_data_<focus>.csv	One row per individual in the Focus: time, event, chamber, treatment, factors.
lifetables_<focus>.csv	The life table per treatment (see Life tables).
summary_<focus>.csv	Summary statistics per treatment.
median_survival_<focus>.csv	Median survival per treatment, with its 95% confidence interval.
mean_survival_<focus>.csv	Mean survival (RMST to the common τ) per treatment.

statistics/ (CSV)

File	Written when
survival_quantiles_<focus>.csv	always
lifespan_treatment_stats_<focus>.csv	always (when there is data)
lifespan_factor_stats_<focus>.csv	the Focus varies two or more factors
logrank_pairwise_<focus>.csv	log-rank pairwise ran
gehan_wilcoxon_pairwise_<focus>.c	

sv	Gehan-Wilcoxon ran
hazard_ratios_<focus>.csv	pairwise hazard ratios ran
factorial_01_coefficients_<focus>.csv, factorial_01_ph_test_<focus>.csv	the Cox factorial model ran
factorial_02_coefficients_<focus>.csv	the RMST factorial model ran
parametric_aic_<focus>.csv	parametric AFT models ran (one row per treatment × model, including those not fitted and why)

The omnibus log-rank result is in the Run Summary and the report, not in a CSV. The parametric AFT comparison is in its CSV, the Run Summary and the report.

Which button writes what

- **Run analysis** (Analyze panel), the `run_analysis` script step, and a Batch Run write everything above for the Focus. Each run also deletes files an earlier run wrote for items now left out or not computable.
- **Generate plots** (Plots panel) redraws only the ticked figures, and the Defined Plots about the Focus, into `plots/`. The statistics, the report and the Run Summary stay as the last full run wrote them.
- **Render publication figures** (Project panel) writes curated Publication Figures to `<experiment>/figures/<focus>/<plot_id>_<focus>.svg` (or `.pdf`, `.png`). It writes them only for Focuses whose saved results are current.

Project level

```
MyProject/
├── project.yaml
├── plot_specs.yaml           Publication Figure specs and styles
├── MyProject_report.pdf      the Project Report
├── MyProject_report.md
├── MyProject_report_figures/
├── MyProject_narrative.json  the saved AI narrative, when one was writte
├── cohort_a/                 a Member Experiment, laid out as above
└── cohort_b/
```

The Project Report and the AI narrative are named after the Project's directory. The narrative's paragraphs are deleted from that file once the Focus they describe is re-analysed (see [The AI narrative](#)). A Batch folder holds an optional `batch.yaml` — written when you choose a script in the Batch panel's picker — and no results of its own.

Renaming and deleting Focuses

- Renaming a Focus **in the Focus window** moves its folder and renames the `_<focus>` suffix of every file inside it, so nothing is lost. The Markdown report's image folder (`report_<focus>_figures/`) is renamed with it, and the report's image links are rewritten to match.
- Renaming one **by hand in the YAML**, or deleting one, leaves its folder behind as **Orphaned Results**. The Hub lists these and offers *adopt as...* or *delete*. See [Analysed, Out of Date, Blocked and Orphaned results](#).
- `plots`, `statistics` and `data_output` directly under `analysis/`, `analysis/`

`run_summary.json`, `analysis/report.md` and `analysis/*_report.pdf` are results from before Focuses existed. They are shown as legacy results and offered for deletion, never adopted. For this reason, a Focus cannot be named `plots`, `statistics` or `data_output`.

Pitfalls

- **Re-running replaces.** A new run of a Focus overwrites that Focus's previous files. Copy a folder elsewhere if you want to keep an earlier version.
- **Open files.** If a figure is open in another program when a run tries to delete or overwrite it, the old file can be left in place.
- **Moving the data file** does not move results. Results live under `analysis/`, not next to the data file.

See also

- [The Run Summary](#)
- [The experiment report](#)
- [What a Focus is](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Publication Figures, Specs and Styles](#)
- [The Project Report](#)

Data and configuration

Input data formats

pySurvAnalysis reads three kinds of file: a **DLife Excel workbook** (`.xlsx`), which records chambers of individuals scored at each census, and **CSV/TSV files** (`.csv`, `.tsv`) holding one record per individual, in either *long* or *wide* layout. Whatever the format, the loader turns the file into one row per individual with an age (`time`), an outcome (`event`: 1 = death observed, 0 = right-censored), a chamber id, and one column per treatment factor. Every analysis starts from that table.

The file type is decided by its extension. For a workbook, `input.format` in `survival_config.yaml` makes no difference. For a CSV/TSV, `input.format` chooses between long and wide (see [The input: section](#)).

Where the data file lives

Each Experiment Directory has exactly one data file. The app looks for it in this order:

1. The file named by `data_file`: in `survival_config.yaml`, if that key is set (a path relative to the experiment directory, or an absolute path).
2. Otherwise, any `.xlsx`, `.csv` or `.tsv` in the `data/` subdirectory.
3. Otherwise, any such file at the directory root.

The search stops at the first place that holds anything. If that place holds **more than one** candidate, the app refuses to guess and the member is *Blocked* until you name one with `data_file`: (the **Set data file...** button in the Project panel's *Experiment configs...* dialog writes this key for you). Excel lock files (`~$...`) and `remove_chambers.csv` are never treated as data.

The DLife workbook (.xlsx)

A DLife workbook holds a census: for each chamber, how many individuals died (or were censored) between successive census times. The app reads these sheets:

Sheet	Required	What the app reads
Design	yes	One row per chamber: <code>Chamber</code> , <code>SampleSize</code> , <code>StartTime</code> , then the treatment factors
RawData	yes	One row per chamber per census: <code>AgeH</code> , <code>Chamber</code> , <code>IntDeaths</code> , <code>Censored</code>
ChamberFlags	no	<code>Chamber</code> and <code>Excluded</code> ; chambers with <code>Excluded</code> = 1 are removed from every run
DefinedPlots	no	Named treatment groupings drawn as extra Kaplan-Meier figures
PrivateData	no	<code>AssumeCensored</code> (1 or 0), read only when a config is first written for the workbook

Other sheets and extra columns are ignored.

Design sheet

- `Chamber` — the chamber id, matching `Chamber` in `RawData`.
- `SampleSize` — how many individuals were placed in the chamber at the start. Used only when assumed censoring is on.
- `StartTime` — marks where the factor columns begin. Its value is not used in any calculation.
- **Every column after `StartTime` is a treatment factor** (for example `Sex`, `Genotype`, `Diet`). You never declare factors anywhere else; they are *discovered* from these columns. There must be at least one.

The levels of each factor are discovered in **order of first appearance** down the Design sheet, not alphabetically. That order becomes the default display order and, through it, the default Reference Level of each factor, so put the control level in the first rows if you want it to be the baseline. A blank factor cell means "no level recorded": those individuals belong to no treatment of any Focus that names that factor, and each run says how many there were.

RawData sheet

- `AgeH` — the age at that census. It is used as-is as the survival time. No unit conversion is done; `global.time_unit` and `global.time_label` only label the axes.
- `Chamber` — which chamber the row belongs to.
- `IntDeaths` — deaths observed since the previous census; each becomes one individual with `event` = 1 at this age.
- `Censored` — individuals removed alive (escaped, lost, sacrificed) since the previous census; each becomes one individual with `event` = 0 at this age.

`RawData` rows whose chamber does not appear on the Design sheet are skipped.

Assumed censoring

With `global.assume_censored: true` (the default), each chamber's starting number is its `SampleSize`. Individuals not accounted for by `IntDeaths` and `Censored` are added as right-

censored at that chamber's **last census age**. With `false`, a chamber's cohort is just the deaths plus explicit censorings in `RawData`, and nothing is added. See [Censoring policy](#).

The `PrivateData` sheet's `AssumeCensored` value is read only when the app writes a config around a workbook (adding a workbook or directory to a Project). It then seeds `global.assume_censored`. After that, the config decides, and editing the sheet has no effect.

ChamberFlags sheet

Any chamber with `Excluded = 1` is dropped before analysis, in every run, whatever Exclusion Group is active. Reports show these as removed "by the workbook" when no group is active. The Chamber QC viewer still shows flagged chambers, because it loads every chamber. To exclude chambers from inside the app, use an [Exclusion Group](#).

DefinedPlots sheet

There is one column per plot. **Row 1** holds the plot's name. From **row 6** down, each cell names one treatment as a full label, with the levels of every Design factor joined by `/` in Design-column order (e.g. `Female/20x`). Only the text before the first comma in a cell is used, and the first empty cell ends that column's list. See [Defined Plots](#).

CSV/TSV, long format

One row per individual. `.csv` is comma-separated and `.tsv` is tab-separated.

```
Age,Event,Sex,Genotype
52.5,1,F,wCS
61.0,1,F,wCS
70.0,0,M,mDilp235bx
```

- **Time column** — default name `Age`. Change it with `input.time_col`.
- **Event column** — default name `Event`. Change it with `input.event_col`. It must contain only 0 (censored) and 1 (death).
- **Factor columns** — by default every other column. To use only some columns as factors (for example, to ignore an ID column), list them in `input.factor_cols`.

Rows whose time or event is blank or not numeric are dropped, and counted: the run log and the report's Data quality section say how many (N row(s) dropped: their time or event is blank or not a number.). Levels are discovered in the order they first appear in the file.

CSV/TSV, wide format

One column per group, where a group is a combination of two factor levels plus an outcome. Each column lists the ages of the individuals in that group. Columns can have different lengths.

```
F_20x_dead,F_20x_cens,F_40x_dead,F_40x_cens,M_20x_dead,M_20x_cens,M_40x_dead,M_40
50,70,48,70,44,70,41,70
55,,52,,47,,45,
```

The wide format supports **exactly two factors**. You must give `input.factor_names`, the two factor names, and one way to tell which levels and outcome each column holds (see [The input: section](#) for both examples):

- `input.col_mapping` — an explicit entry per column. Columns not named in the mapping are ignored. The factor levels are ordered as they first appear in `col_mapping`.
- `input.factor_levels` — each factor's levels. The app then reads each column's name: it must mention exactly one level of each factor and an outcome word (dead,

death, died or event for deaths; cens, censor, censored or alive for censored). Whole words separated by `_`, `-`, spaces or dots are matched first, so `Female_20x_dead` is Female and not Male. Every level combination must have both a death and a censored column, or loading fails asking for `col_mapping`. The levels are ordered as `factor_levels` lists them.

Each non-empty cell becomes one individual.

Long or wide? (format: auto)

With `input.format: auto` (the default), a CSV/TSV is read as **long** if its header contains both the time column and the event column, and as **wide** otherwise. If you renamed your columns, set `time_col/event_col` or set `format: long` explicitly. Otherwise the file will be treated as wide and fail for lack of `factor_names`.

CSV inputs have no chambers

Individual-level CSV data has no chamber identities (the chamber column reads N/A). So an Exclusion Group cannot remove anything from a CSV experiment, and the Chamber QC viewer does not apply. Reports record the group in force with zero chambers removed.

Error messages from the loader

Message	Cause and fix
No treatment factor columns found after StartTime in Design sheet	Add factor columns to the right of StartTime.
RawData sheet missing columns: {...}	RawData needs AgeH, Chamber, IntDeaths, Censored.
Long-format CSV missing columns: [...]	The time/event column names don't match. Set <code>input.time_col / input.event_col</code> .
No factor columns found in CSV. Specify <code>factor_cols</code> explicitly.	The file has only time and event columns.
Factor columns not found in CSV: [...]	A name in <code>input.factor_cols</code> is not a column header.
Event column must contain only 0 and 1 values.	Recode the event column to 0/1.
<code>factor_names</code> is required for wide-format CSV loading.	Give <code>input.factor_names</code> (two names), or fix the column names if the file is really long format.
Wide format requires exactly 2 factor names.	<code>input.factor_names</code> must list exactly two names.
Wide-format column not found: ...	A column: in <code>col_mapping</code> is not a header in the file.
Wide column '...' must set event to 0 or 1.	Each <code>col_mapping</code> entry needs <code>event: 0</code> or <code>event: 1</code> .
A wide-format CSV needs <code>input.col_mapping</code> , or <code>input.factor_levels</code> (each factor's levels, matched against the column names).	Give one of the two.

Could not infer complete wide mapping from column names. Mapped N columns, expected M. Provide explicit col_mapping.	With factor_levels, some column names do not name one level of each factor plus an outcome, or a combination lacks its death or censored column. Rename the columns or give col_mapping.
Unsupported file type: ...	Only .xlsx, .csv and .tsv are read.

Pitfalls

- **Level order is set by the data.** It is set by the first rows of the Design sheet (or CSV), not by the alphabet. Once a Focus is saved, its level order is frozen in the config, so re-sorting the sheet later does not move a Reference Level.
- **More deaths than SampleSize.** If RawData records more deaths and censorings for a chamber than its SampleSize, nothing extra is added, N uses the recorded count, and the run warns: Chamber <id>: RawData records <n> deaths and censorings but its SampleSize is <s> – N uses the <n> recorded; check the Design sheet or RawData. The warning appears in the run log and the report's Data quality section, whatever the censoring policy.
- **Ages are not adjusted for StartTime.** Survival time is AgeH exactly as recorded.

See also

- [The input: section](#)
- [Censoring policy](#)
- [Defined Plots](#)
- [Exclusion Groups](#)
- [survival_config.yaml reference](#)

survival_config.yaml reference

Every Experiment Directory has a `survival_config.yaml` at its root. The file is what makes a folder an experiment. It also holds every setting that decides a result: how to read the data file, the censoring policy, the active Exclusion Group, the declared Focuses, and which analyses and figures a run leaves out. The same file and the same data always give the same result, whether the run comes from the Hub, a script, or a Batch Run.

Factors are **never declared** here. They are discovered from the data file (see [Input data formats](#)).

In the app

The Hub writes most of this file for you:

- **Create experiment..., Initialize existing directory...** and the *Experiment configs...* dialog scaffold it.
- The QC panel's **Set active group** writes `exclusions:.`
- The Focus window writes `focuses:.`
- The Analyze and Plots checkboxes write `omit:.`
- The Script Editor writes `scripts:.`

To edit the file by hand, use **Edit config...** in the Project panel's *Experiment configs...* dialog. It opens the file in your desktop's YAML editor. Keys the app does not recognise are kept

untouched when the app rewrites the file.

A complete example

```
experiment_type: standard_lifespan
data_file: data/cohort_2026.xlsx # only needed when data/ holds several fil
global:
  time_unit: days
  time_label: Age (days)
  assume_censored: true
  min_n_per_chamber: 5
input:
  format: auto # auto | excel | long | wide
  time_col: Age
  event_col: Event
  factor_cols: null
  factor_names: null
  col_mapping: null
exclusions:
  group: default
focuses:
  Crowding20v40:
    factors:
      Sex: [Female, Male]
      Density: ["20x", "40x"]
    reference:
      Density: "40x"
    display_names:
      Female/20x: "F, 20x"
    colours:
      Female/20x: "#b2182b"
omit:
  analyses: [parametric_aft]
  plots: [number_at_risk]
scripts:
- name: Standard analysis
  notes: Runs the standard analysis battery once per Focus.
  steps:
  - action: run_in_focuses
  - action: run_analysis
```

Top-level keys

Key	Type	Default	Effect
experiment_type	string	standard_lifespan	The Experiment Type . Omitted, empty, or a retired key (interaction, custom) all mean Standard Lifespan. Any other unknown value is an error.
data_file	path	none	Names the data file, relative to the experiment directory or absolute. Overrides the search of data/ and the root. Required when that search finds several files.
global	mapping	type defaults	Time unit, axis label, censoring policy, quality threshold. See The global: section .

input	mapping	see below	How to read a CSV/TSV. See The input: section .
exclusions	mapping	none	group: <name> names the active Exclusion Group . Blank (group: ' ') means no group; absent means no group unless the Project Defaults name one, which a member then inherits.
focuses	mapping	Unfiltered	Named Focuses (below).
omit	mapping	nothing left out	Analyses and figures every run leaves out. See The omit: section .
scripts	list	Standard analysis	Saved Experiment Scripts. See Experiment and Project Scripts .
factors	mapping	none	Legacy. A pre-Focus factor block. If there is no focuses: block, it becomes one Focus called Interaction, and it is removed the next time Focuses are saved.

exclusions

exclusions:

group: default # a group name from qc/remove_chambers.csv

Key	Type	Default	Effect
group	string	none	Chambers listed under this group in qc/remove_chambers.csv are removed from every run. The name is stamped on every report and Run Summary. A name with no rows in the file removes nothing.

focuses

Each entry under focuses: is one Focus, keyed by its name. The name is used, made filesystem-safe, as the output directory analysis/<name>/ and as the _<name> suffix on every output file.

Key	Type	Default	Effect
factors	mapping factor → list of levels	required	Which factors and levels the Focus selects, in display order. A factor with two or more levels varies (it labels treatments). A factor with one level is a filter . A discovered factor not named is pooled over .
reference	mapping factor → level	first level listed	The Reference Level for Cox and RMST coding.
display_names	mapping treatment → text	none	Display names for treatments (labels such as Female/20x).

colours	mapping treatment → colour	none	Curve colours per treatment.
---------	----------------------------	------	------------------------------

If the file has no `focuses:` block, the experiment still has one Focus, `Unfiltered`: every discovered factor at every level. It is written into the file the first time the experiment is analysed or its Focuses are edited. See [What a Focus is](#).

Display names and colours don't change any number, so editing them never makes results [Out of Date](#). Changing `factors` or `reference`, or the Exclusion Group, does.

scripts

A list of saved step lists, each with `name`, optional `notes` and `steps` (each step has `action:` plus that action's parameters). If a file has **no** `scripts:` **key at all**, the default `Standard analysis` script is added the next time the app writes the file. An empty list (`scripts: []`) counts as a deliberate deletion and is left alone. For a Member Experiment, a script of the same name in the Project's `experiment_scripts:` takes precedence. See [Running Experiment Scripts](#).

Members and Project Defaults

For a Member Experiment, this file is merged with the Project's `defaults:` before use. Anything the member states wins. `global`, `input`, `exclusions` and `omit` merge key by key, so a member can override one setting without restating the whole section. `focuses` is never inherited. A member scaffolded by the app is deliberately minimal (often just `experiment_type` and `scripts`), so later edits to the Project Defaults keep reaching it. See [project.yaml](#) and [Project Defaults](#).

Checking the file

Validate YAMLS on the Project panel checks this file, and every member's in an open Project, and lists any problems in the output log. Every message and its fix is listed in [Validating configuration](#).

See also

- [The input: section](#)
- [The global: section](#)
- [The omit: section — leaving analyses and figures out](#)
- [project.yaml](#) and [Project Defaults](#)
- [What a Focus is](#)
- [Validating configuration](#)

The input: section

The `input:` section of `survival_config.yaml` tells the loader how to read a **CSV or TSV** data file: long or wide layout, which columns hold the time and the event, and which columns are factors. A DLife workbook (`.xlsx`) is self-describing: its Design sheet names the factors, and RawData has fixed column names. So for a workbook this section has no effect beyond the `format` value the app writes when adopting it.

All keys are optional. A missing key takes the default below.

Keys

Key	Type	Default	Effect
-----	------	---------	--------

format	auto / excel / long / wide	auto	For CSV/TSV: long or wide forces the layout. auto (and excel) detects it: long if the header holds both time_col and event_col, wide otherwise. Ignored for .xlsx, which is always read as a DLife workbook.
time_col	string	Age	Long format: the column holding each individual's age at death or censoring. Also used by auto detection.
event_col	string	Event	Long format: the column holding 1 (death observed) or 0 (censored). Also used by auto detection.
factor_cols	list of strings	null (every other column)	Long format: the columns that are treatment factors. Leave null to use every column except time and event. List them to ignore extra columns such as an individual ID.
factor_names	list of two strings	null	Wide format: the names of the two factors. Required for wide.
col_mapping	list of mappings	null	Wide format: what each column holds, column by column (see below). Wide needs this or factor_levels.
factor_levels	mapping of factor → list of levels	null	Wide format without col_mapping: each factor's levels, which the app looks for in the column names (see below). Its factors must

		be exactly those in factor_names.
--	--	--------------------------------------

Long format example

A file with columns Days, Dead, Line, Diet, FlyID:

```
input:
  format: long
  time_col: Days
  event_col: Dead
  factor_cols: [Line, Diet]      # FlyID is not a factor
```

Without factor_cols, FlyID would be treated as a factor, and every individual would be its own treatment.

Wide format example

A file with one column of ages per Sex × Density group and outcome:

```
input:
  format: wide
  factor_names: [Sex, Density]
  col_mapping:
    - {column: F_20x_dead, factor1_level: Female, factor2_level: "20x", event: 1}
    - {column: F_20x_cens, factor1_level: Female, factor2_level: "20x", event: 0}
    - {column: F_40x_dead, factor1_level: Female, factor2_level: "40x", event: 1}
    - {column: F_40x_cens, factor1_level: Female, factor2_level: "40x", event: 0}
    - {column: M_20x_dead, factor1_level: Male, factor2_level: "20x", event: 1}
    - {column: M_20x_cens, factor1_level: Male, factor2_level: "20x", event: 0}
    - {column: M_40x_dead, factor1_level: Male, factor2_level: "40x", event: 1}
    - {column: M_40x_cens, factor1_level: Male, factor2_level: "40x", event: 0}
```

Each col_mapping entry has four keys:

Key	Meaning
column	A header in the file.
factor1_level	The level of the first factor in factor_names for every age in this column.
factor2_level	The level of the second factor.
event	1 if the ages in this column are deaths, 0 if they are censorings.

Every non-empty cell in a mapped column becomes one individual. Columns left out of the mapping are ignored. The order of the entries sets the order of each factor's levels (here Female before Male and 20x before 40x), so put the control level first.

Without a mapping: factor_levels

When the column names already say what each column holds, list each factor's levels instead of mapping every column:

```
input:
  format: wide
  factor_names: [Sex, Density]
  factor_levels:
    Sex: [Female, Male]
    Density: ["20x", "40x"]
```

Each column name must then mention exactly one level of each factor and an outcome word: dead, death, died or event for deaths, cens, censor, censored or alive for censorings

(Female_20x_dead, Male-40x-censored). Case and punctuation are ignored, and whole words are matched first, so Female_20x_dead is not mistaken for Male. Every combination of levels needs both a death column and a censored column; if the names cannot be read that way, loading fails asking for `col_mapping`. The levels are ordered as `factor_levels` lists them, so put the control first.

Where the format is first written

When the app builds a config around a data file (adopting a workbook or directory into a Project), it writes `input: {format: excel}` for a workbook. For a CSV it writes `format` (long or wide, as detected) together with `time_col: Age` and `event_col: Event`. In a Project, the `input:` block can also live in `defaults:` and is inherited key by key. A member that sets only `time_col` keeps the Project's other input settings.

Checking it

Validation checks `format` and the wide-format keys:

- ``input.format`` must be `auto`, `excel`, `long` or `wide` (got '...'). — fix the spelling.
- ``input.format: wide`` needs ``input.factor_names`` (the two factor names). and ``input.factor_names`` must name exactly two factors for the wide format.
- ``input.format: wide`` needs ``input.col_mapping`` or ``input.factor_levels`` to say which cell each column holds.
- ``input.factor_levels`` must map each factor to its levels, ..., ``input.factor_levels.<factor>`` must be a non-empty list of levels, ..., lists a level twice. and ``input.factor_levels`` must give levels for exactly the factors in ``input.factor_names`` (...).
- ``input.col_mapping`` must be a list of ... entries.

Wrong column names show up when the file is loaded (see the error table in [Input data formats](#)), not during validation.

Pitfalls

- **Renamed columns and `auto`.** If your time/event columns are not called `Age/Event` and you leave `time_col/event_col` unset, `auto` detection will not find them. The file is then treated as wide, and loading fails asking for `factor_names`.
- **ID columns.** In long format every non-time, non-event column is a factor unless you list `factor_cols`.
- **No chambers.** CSV data has no chambers, so Exclusion Groups and the Chamber QC viewer do not apply to it.

See also

- [Input data formats](#)
- [survival_config.yaml](#) reference
- [project.yaml](#) and Project Defaults
- [Validating configuration](#)

The global: section

The `global:` section of `survival_config.yaml` holds the data-source settings an Experiment Type supplies defaults for: the time unit and axis label, the censoring policy, and a

quality threshold. In a Project these usually live once in `project.yaml` under `defaults:`
`global:` and are inherited key by key by every member that does not state its own value.

Example

```
global:
  time_unit: days
  time_label: Age (days)
  assume_censored: true
  min_n_per_chamber: 5
```

This is exactly what a Standard Lifespan experiment is scaffolded with.

Keys

Key	Type	Default (Standard Lifespan)	Effect
time_unit	string	days	The unit your ages are recorded in, as a word. It is used to build the axis label when <code>time_label</code> is empty (<code>Age (<unit>)</code>), and the Project Report's Divergence Note compares it across members. It does not convert anything: ages are analysed exactly as they appear in the data file.
time_label	string	Age (days)	The time-axis label on every analysis figure (and the QC viewer), and the default x-axis label of Publication Figures in the Plot Editor.
assume_censored	true / false	true	DLife workbooks only. When true, each chamber starts with its Design-sheet <code>SampleSize</code> , and individuals not accounted for by deaths or explicit censorings are right-censored at the chamber's last census. When false, only what <code>RawData</code> records is counted. No effect on CSV data. See

			Censoring policy.
<code>min_n_per_chamber</code>	non-negative integer	5	A per-chamber minimum sample size, where 0 means "off". A chamber with fewer individuals is flagged, never excluded : the Chamber QC viewer draws it in amber and lists it, and the report's Data quality section names it as a warning. Whether to exclude it is your QC decision.

Keys the type does not define are kept as they are. The Project editor shows them too, so they are not lost when the project is saved.

Where to set it

- **For a whole Project:** open the Project panel and choose **Edit config....** The *Project Defaults* box has a **Global** form, built from the Experiment Type's own keys. A blank text field means "use the type's default", and nothing is written for it. A checkbox is always written as true or false. See [project.yaml](#) and [Project Defaults](#).
- **For one experiment:** add the key under `global:` in its `survival_config.yaml`. Only state the keys you want to differ. Restating a Project default freezes it in this member, so later Project edits stop reaching it.

Checking it

Validate YAMLS reports:

Message	Fix
<code>`global:`</code> must be a mapping.	<code>global:</code> must be followed by indented <code>key: value</code> lines, not a list or a single value.
<code>`global.time_unit`</code> must be a string (e.g. <code>days</code>).	Write a word, e.g. <code>time_unit: days</code> .
<code>`global.assume_censored`</code> must be true or false.	Write true or false, without quotes. "yes" or 1 are not accepted.
<code>`global.min_n_per_chamber`</code> must be a non-negative integer (0 turns the check off).	Write a whole number ≥ 0 , without quotes.

Pitfalls

- **Changing the unit does not rescale the data.** If your workbook's `AgeH` column is in hours and you set `time_unit: days`, every number in every table is still in hours. Only the labels change.
- **`time_label` labels every figure's time axis:** the Hub's and the report's figures, the Defined Plots, and a new Publication Figure's x label. A Publication Figure saved earlier keeps the x label stored in its Spec until you change it in the Plot Editor.
- **Censoring differences between members.** If members of a Project use different

`assume_censored` values, the Project Report declares it in its Divergence Note. This difference changes how every unscored individual is counted.

See also

- [Censoring policy](#)
- [survival_config.yaml](#) reference
- [project.yaml](#) and Project Defaults
- [Experiment Types](#)
- [Validating configuration](#)

Censoring policy

An individual is **right-censored** when its death was not observed: we know it was alive up to a certain age, but not when it died. Censored individuals still carry information. They count as "at risk" up to their censoring time and then leave the analysis without counting as a death. Every survival method in the app (Kaplan-Meier, log-rank, Cox, RMST, the parametric fits) uses censored individuals this way. This page explains where censored individuals come from in your data, and the one policy setting that controls them: **Assumed Censoring**.

Where censored individuals come from

Every input is turned into one row per individual, each with a `time` and an `event` flag (1 = observed death, 0 = censored). You can inspect the result in `analysis/<focus>/data_output/individual_data_<focus>.csv`.

DLife Excel workbooks (census data)

For each chamber, each census row in the **RawData** sheet contributes:

- `IntDeaths` deaths at that census age (`AgeH`), each an individual with `event` = 1;
- `Censored` explicitly censored individuals at that age (for example flies that escaped or were lost in handling), each with `event` = 0.

Then the censoring policy decides what happens to individuals that were never accounted for:

- **Assumed censoring on** (the default): the chamber's starting cohort is its `SampleSize` from the **Design** sheet. Any individuals not accounted for by deaths and explicit censorings (`SampleSize` – deaths – censored) are added as **censored at that chamber's last census age**. This is the DLife convention: anyone still unaccounted for at the end was alive when scoring stopped.
- **Assumed censoring off**: the cohort is only the individuals that appear in the census (deaths plus explicit censorings). `SampleSize` is ignored and nothing is added.

If a chamber's recorded deaths and censorings already add up to `SampleSize` or more, assumed censoring adds nobody.

CSV and TSV files

These are already one row per individual, so censoring comes from the data: in long format, from the event column (default name `Event`, 1 = death, 0 = censored); in wide format, from the columns mapped to event 0. **The assumed-censoring setting has no effect on CSV or TSV input.** See [Input data formats](#).

Setting the policy

The policy is the `assume_censored` key of the `global:` section of the experiment's `survival_config.yaml`:

```
global:
  assume_censored: true      # unaccounted individuals are censored at the last ce
```

- If the key is absent, the Experiment Type's default applies. For Standard Lifespan, that default is `true`.
- In a Project, `assume_censored` can be set once in the **Project Defaults** (Project panel, **Edit config...**, the Global fields). A member that does not state its own value inherits it. See [project.yaml](#) and [Project Defaults](#).
- When the app first writes a config for a DLife workbook, it copies the workbook's own setting (the `AssumeCensored` flag on the **PrivateData** sheet, 1 = on) into `assume_censored`. From then on, the config is the authority; editing the workbook flag later changes nothing.

The value must be `true` or `false`; [Validate YAMLS](#) reports anything else.

How it shows in the results

- **Report cover:** the "Censoring policy" line reads *unaccounted individuals censored* (on) or *unaccounted individuals ignored* (off). The cover also gives the total number and percentage censored.
- **Per-treatment summary:** `n_censored` and `pct_censored` for each treatment ([Summary statistics](#)).
- **Run Summary:** `assume_censored` is recorded in `run_summary_<focus>.json`.
- **Project Report:** when members of a Project use different policies, a Divergence Note says so, because the same raw counts give different survival estimates under the two policies.
- **Top-percentile lifespans:** the `top_10pct_mean` and `top_5pct_mean` columns of the lifespan statistics are produced only when assumed censoring is **off**.

What the choice does to the statistics

With assumed censoring **on**, unscored individuals count as alive until the last census. The KM curve then usually ends above zero, the RMST is restricted to the observation window, and the median can be undefined if too many are censored. Survival estimates are higher than if those individuals were ignored.

With it **off**, the unscored individuals simply do not exist in the analysis. This is appropriate when `SampleSize` is not reliable, or when every individual was scored to death and the difference is only bookkeeping. If individuals really did disappear alive, turning it off biases survival downward.

Pitfalls

- **Changing the policy does not mark existing results Out of Date.** Out of Date tracks the Focus definition and the Exclusion Group, not the censoring policy. After changing `assume_censored`, re-run every Focus you rely on.
- **A wrong SampleSize becomes censoring.** With the policy on, a typo that makes `SampleSize` too large adds phantom censored individuals at the last census. Check `pct_censored` for any treatment that looks implausible.
- **Censoring must be uninformative.** Every method here assumes that being censored says nothing about when an individual would have died. Escapes due to sickness, or removing the frailest individuals, break this assumption.
- **The last census.** Assumed-censored individuals are placed at each chamber's own last

census age, so chambers scored for different lengths of time contribute censorings at different ages.

See also

- [Life tables and the Kaplan-Meier estimator](#)
- [Input data formats](#)
- [The global: section](#)
- [Summary statistics, median and mean survival](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)

The omit: section — leaving analyses and figures out

The `omit:` section of `survival_config.yaml` lists the optional analyses and figures that **every run of this experiment leaves out**. It is what the unticked boxes on the Hub's Analyze and Plots panels are saved as. Because it is configuration rather than a setting of the Hub window, **Run analysis** in the Hub, an Experiment Script's `run_analysis` step, and a Batch Run all leave out the same things and produce the same report.

The block records what is left *out*, not what is included. A config nobody has touched runs everything, and an analysis added by a later version of the app is included until someone unticks it.

Example

```
omit:
  analyses: [parametric_aft, gehan_wilcoxon]
  plots: [number_at_risk, survival_distribution]
```

A single id may be written without brackets (`analyses: parametric_aft`). An empty or missing list leaves nothing out.

Keys

Key	Type	Default	Effect
analyses	list of analysis ids	[]	These optional analyses are not computed, and each is recorded as Left Out.
plots	list of plot ids	[]	These figures are not drawn, and each is recorded as Left Out.

Any other key under `omit:` is a validation error.

Analysis ids

Id	Hub label	Offered when
logrank_pairwise	Log-rank pairwise	the Focus implies 2+ treatments
logrank_omnibus	Log-rank omnibus	the Focus implies 2+ treatments
gehan_wilcoxon	Gehan-Wilcoxon pairwise	the Focus implies 2+ treatments
hazard_ratios	Pairwise hazard ratios	the Focus implies 2+ treatments
parametric_aft	Parametric AFT models	always
interaction	Interaction analyses (Cox + RMST factorial models)	the Focus varies 2+ factors

Lifetables, summary statistics, median and mean survival are the survivorship core. They always run and cannot be omitted.

Plot ids

Id	Figure	In the Standard Lifespan Plot Set
km_risk_table	KM curves with at-risk table	yes (the Headline Figure unless the Focus crosses factors)
km_curves	Kaplan-Meier curves	yes
cumulative_events	Cumulative deaths	yes
survival_distribution	Lifespan distribution	yes
mortality	Mortality (qx)	yes
hazard	Hazard rate	yes
smoothed_hazard	Smoothed hazard	yes
nelson_aalen	Nelson-Aalen cumulative hazard	yes
number_at_risk	Number at risk	yes
hazard_ratio_forest	Hazard-ratio forest	yes, with 2+ treatments
log_log	Log-log diagnostic	yes, with 2+ treatments
km_faceted	Faceted Kaplan-Meier	added when the Focus varies 2+ factors
interaction_lifespan	Lifespan interaction plot	added when the Focus varies 2+ factors

In the app

On the Hub, load an experiment and open **Analyze** or **Plots** under the Experiment tile. Each item the Active Focus is offered has a checkbox:

- **Unticking** a box adds its id to `omit`: immediately. **Ticking** it removes the id. The output log confirms the change (... left out of <experiment>'s runs – survival_config.yaml omit:).
- A box that is **greyed** is either Not Applicable to this Focus's data (the tooltip gives the reason), or a figure that draws from an unticked analysis.
- Items the Focus is not offered at all are not shown as boxes. One quiet line ("Not offered for Focus ...") names them.

The `omit`: block belongs to the **experiment**, not to a Focus. Unticking an analysis leaves it out of every Focus that would have been offered it.

What a run does with it

- Each left-out item is written in the run log (Left out – <label>: unticked in the Hub (omit: in survival_config.yaml)), in the Run Summary's `left_out` list, and in a **Left out of this run** table in the Focus's report section.
- **The forest follows the hazard ratios.** If `hazard_ratios` is omitted, the Hazard-ratio

forest is left out too, with the reason "draws from Pairwise hazard ratios, which is unticked". Its Plots box is greyed until you tick the analysis again.

- Files an earlier run wrote for an item that is now left out (its PNG, or its statistics CSV) are **deleted**, so the folder never shows an old figure as if it were current.
- Only items the Focus is offered can be left out of it. Omitting `interaction` means nothing to a one-factor Focus and is not reported there.
- Left Out is not the same as **Not Applicable**. Left Out is your choice. Not Applicable means the data could not support the item. See [Not Applicable and Left Out](#).

In a Project

`omit:` can be set under `defaults:` in `project.yaml`, and members inherit it. Inheritance is per list: a member that states `omit: {plots: [...]}` keeps the Project's `analyses:` list but replaces its `plots:` list entirely (the lists are not combined). When you tick a box in a member that inherits an omission, the app writes the member's own list, an empty `[]` if necessary, to override the Project's.

Checking it

Message	Fix
<code>`omit:` must be a mapping, e.g. `{analyses: [parametric_aft]}`.</code>	Put <code>analyses:</code> and/or <code>plots:</code> under <code>omit:</code> .
<code>`omit.<key>` is not something a run can leave out (use <code>analyses</code> or <code>plots</code>).</code>	Only <code>analyses</code> and <code>plots</code> are allowed.
<code>`omit.<kind>` must be a list of ids.</code>	Write a list, e.g. <code>[parametric_aft]</code> .
<code>`omit.<kind>` names '<code><id></code>', which no run produces. Known: ...</code>	A misspelt id omits nothing. Copy the id from the list in the message.

See also

- [The Analyze panel and Run analysis](#)
- [The Plots panel and Generate plots](#)
- [Not Applicable and Left Out](#)
- [Focus Shape — what a Focus is offered](#)
- [survival_config.yaml reference](#)

project.yaml and Project Defaults

A **Project** is a directory with a `project.yaml` at its root. Each immediate subdirectory that holds a `survival_config.yaml` is one of its **Member Experiments**. `project.yaml` names the Project, states the question its members address, holds the **Project Defaults** every member inherits, and stores the Project's scripts. A Project never pools its members: each is analysed on its own, and the Project Report binds their results side by side.

In the app

- **Create project...** and **Initialize existing directory...** (Project panel, Create/Load card) write a new `project.yaml`.
- **Edit config...** opens the same editor on the open Project. It has fields for the name, the question, the Experiment Type and the `global:` defaults. Everything else in the file

(scripts, other default sections, keys from a newer version) is carried through unchanged when you save.

- **Validate YAMLS** checks `project.yaml` and every member's config.

A complete example

```
name: Crowding study
question: Does larval density change adult lifespan, and does it depend on sex?
defaults:
  experiment_type: standard_lifespan
  global:
    time_unit: days
    time_label: Age (days)
    assume_censored: true
    min_n_per_chamber: 5
  input:
    format: excel
  exclusions:
    group: default
  omit:
    analyses: [parametric_aft]
scripts:
- name: batch
  notes: What a Batch Run runs here unless another script is designated.
  steps:
  - action: run_in_experiments
    script: Standard analysis
  - action: render_publication_figures
  - action: project_report
experiment_scripts: []
```

Top-level keys

Key	Type	Default	Effect
name	string	the directory name	The Project's display name.
question	string	empty	The one-line question the members address. It is shown on the Project Report cover and read by the AI narrative.
defaults	mapping	type defaults	Project Defaults: settings every member inherits unless it states its own.
scripts	list	the batch script	Project Scripts. A Batch Run executes <code>batch</code> unless another is designated. If the key is missing entirely, <code>batch</code> is added on the next save. An empty list is left alone, and such a Project fails a Batch Run that designates no script (the Project card still offers the built-ins). See Project Scripts .
experiment_scripts	list	[]	Central Experiment Scripts. A script here serves every member and takes precedence over a member's own script of the same name — in a Batch

		Run, in <code>run_in_experiments</code> and on the Hub's Experiment scripts card alike.
--	--	---

The Project Report is written to `<project>/<directory name>_report.pdf` (and `.md`).
Publication Figure specs and styles live in `<project>/plot_specs.yaml`.

Keys inside defaults:

Key	Type	Default	Effect on members
<code>experiment_type</code>	string	<code>standard_lifespan</code>	The Project's Experiment Type . This is the one value checked across members: every member must resolve to the same type.
<code>global</code>	mapping	the type's defaults	Merged key by key. See The global: section .
<code>input</code>	mapping	none	Merged key by key. See The input: section .
<code>exclusions</code>	mapping	none	Merged key by key. <code>group</code> : sets the default active Exclusion Group name. Each member has its own <code>qc/remove_chambers.csv</code> , so the name only removes chambers in members that define that group.
<code>omit</code>	mapping	none	Merged per list (analyses, plots). See The omit: section .
any other key	any	none	Inherited as a whole value when the member does not state the key. (Experiment Scripts are the exception: share them through <code>experiment_scripts</code> : above, not <code>defaults</code> .)
<code>focuses</code>	—	—	Never inherited. Validation warns and the key is ignored.
<code>factors</code>	—	—	Legacy. Validation warns. Members that inherit it become an Interaction Focus.

How inheritance works

When a member is loaded, its `survival_config.yaml` is merged over `defaults::`

1. Start from every `defaults:` key except `focuses`.
2. For each key the member states:
 - `global`, `input`, `exclusions` and `omit` (when both sides are mappings) are merged key by key. The member's value wins for each key it states.
 - Any other key replaces the default wholesale.

Defaults are a *seed and template*, not an authority. A member that states a value keeps it. A

member that says nothing follows the next edit to `project.yaml`. That is why members created by the app are deliberately minimal: restating a default in a member freezes today's value there. When you copy a config in from another member, the log names any sections that are now stated in the member and so no longer follow the Project.

Why Focuses are never inherited

Members of one Project rarely share factors. An inherited Focus would be a Blocked Focus in every member it does not fit, and editing it would put every member's results Out of Date at once. To share a design, use **Copy Focuses from...** in the Focus window, which checks each Focus against the receiving member's data before writing it.

What is enforced, and what is only declared

- **Enforced:** every member's Experiment Type must match the Project's. A mismatch is a validation problem, and a config of the wrong type is refused when copied into a member. There is only one type today, so this check cannot currently fail.
- **Declared, not enforced:** differences in `time_unit`, `assume_censored` and the active Exclusion Group are legal. The Project Report lists them in its **Divergence Note**, and **Validate YAMLS** prints each as `divergence - ...`. Differences in factors and Focuses are expected and appear in the Focus Inventory.

Validation messages from project.yaml

Message	Fix
project.yaml: `defaults: focuses:` is ignored — a member never inherits Focuses. Copy them into the members that share the design (the Focus window's Copy Focuses from...).	Remove <code>focuses:</code> from <code>defaults:</code> and copy Focuses into the members that need them.
project.yaml: `defaults: factors:` predates Focuses. Members that inherit it become an `Interaction` Focus on their next analysis; remove it once each has been analysed.	Analyse each member once (so the Focus is written into its config), then delete <code>factors:</code> from <code>defaults:</code> .
project.yaml: Unknown experiment_type '...'. Known types: standard_lifespan, or omit the key.	Fix or remove <code>defaults: experiment_type</code> .
<member> is a <type> but the Project's type is <type>. Every Member Experiment must share the Project's Experiment Type ...	Make the member's <code>experiment_type</code> match, or remove it so it inherits.

See also

- [survival_config.yaml reference](#)
- [Creating and opening a Project](#)
- [The Experiments card — members](#)
- [Experiment Types](#)
- [Validating configuration](#)
- [Project Scripts](#)

Experiment Types

An **Experiment Type** describes the *data source*: what kind of file an experiment reads, the time unit and axis label, the default censoring policy, the default quality settings, the report's sections, and the base set of figures and analyses a run can produce. It deliberately says nothing about the experimental *design*. One data file can hold several designs, so which analyses apply is decided per Focus by its [Focus Shape](#).

Every Member Experiment of a Project must share the Project's Experiment Type. This is the only value checked across members.

There is one type: Standard Lifespan

Key (in YAML)	standard_lifespan
Label (in the app)	Standard Lifespan
Describes	A census-scored lifespan cohort (DLife workbook): chambers of individuals scored to death, with unaccounted individuals treated as censored.

Standard Lifespan is the general case. Every analysis the app can do is reachable from it, gated by Focus Shape rather than by type. CSV/TSV cohorts are read under it too (see [Input data formats](#)).

Default global: settings

```
global:
  time_unit: days
  time_label: Age (days)
  assume_censored: true
  min_n_per_chamber: 5
```

See [The global: section](#) for what each does.

Base Plot Set, in report order

1. KM curves with at-risk table (`km_risk_table`) — the Headline Figure unless the Focus crosses factors
2. Kaplan-Meier curves (`km_curves`)
3. Cumulative deaths (`cumulative_events`)
4. Lifespan distribution (`survival_distribution`)
5. Mortality (qx) (`mortality`)
6. Hazard rate (`hazard`)
7. Smoothed hazard (`smoothed_hazard`)
8. Nelson-Aalen cumulative hazard (`nelson_aalen`)
9. Number at risk (`number_at_risk`)
10. Hazard-ratio forest (`hazard_ratio_forest`) — only when the Focus implies 2+ treatments
11. Log-log diagnostic (`log_log`) — only when the Focus implies 2+ treatments

A Focus that varies two or more factors also gets the **Faceted Kaplan-Meier** (`km_faceted`, which becomes its Headline Figure) and the **Lifespan interaction plot** (`interaction_lifespan`).

Analysis Set

Log-rank pairwise, log-rank omnibus, Gehan-Wilcoxon pairwise, pairwise hazard ratios (each offered when the Focus implies 2+ treatments), parametric AFT models (always offered), and the interaction analyses (offered when the Focus varies 2+ factors). Lifetables, summary statistics, median and mean survival always run. See [The Analyze panel and Run analysis](#).

Report sections

Each run writes an experiment report for its Focus, with these sections in order: Focus · Experiment summary · Survivorship figures · Factorial analysis · Lifespan statistics · Survival comparisons · Data quality. A section with nothing to show is left out of that report. See [The experiment report](#).

Where the type is set

- **In a Project:** choose it in the Project editor (**Create project...**, **Initialize existing directory...** or **Edit config...**). It is stored as `defaults: experiment_type:` in `project.yaml`. Members scaffolded by the app also write `experiment_type:` into their own config.
- **For a standalone experiment:** the `experiment_type:` key in its `survival_config.yaml`.

A config with no `experiment_type` key is a Standard Lifespan.

Retired types

Older configs may name types that no longer exist:

Old key	What it means now
interaction	The former "Interaction Experiment" (a 2×2 design). Read as Standard Lifespan. Its 2×2 analyses are now the Factorial Battery, offered to any Focus that varies two or more factors.
custom	The former "no particular type". Read as Standard Lifespan.

Both keep working. The key is rewritten to `standard_lifespan` the next time the experiment's Focuses are saved. Any other unknown key is an error rather than a silent fallback. An experiment must not be analysed as though it were something else.

Validation messages

Message	Fix
Unknown <code>experiment_type</code> '...'. Known types: <code>standard_lifespan</code> , or omit the key.	Correct the spelling, or delete the key.
<member> is a <type> but the Project's type is <type>. ...	Make the member match the Project (or remove its <code>experiment_type</code> so it inherits).
<file> is a <type> but the Project's type is <type>. Every Member Experiment must share it ...	Shown when copying a config into a member. Nothing was copied; choose a config of the Project's type.

See also

- [The global: section](#)
- [Focus Shape — what a Focus is offered](#)
- [project.yaml and Project Defaults](#)

- [The Plots panel and Generate plots](#)
- [Validating configuration](#)

Validating configuration

Validation checks your YAML files for mistakes before a run depends on them: a misspelt key value, a Focus that names a level twice, an `omit: id` that no run produces, or a member whose Experiment Type differs from its Project's. It reads **only the configuration and the directory layout**. It does not load your data. Problems that need the data, such as a Focus naming a level the workbook lacks, are reported as Blocked Focuses (below).

In the app

- **Validate YAMLs** (Project panel, Create/Load card) checks `project.yaml` and every member's `survival_config.yaml` in the open Project, re-reading them from disk, so an edit made since the Project was opened is checked too. The output log shows:
 - Project validation problems: followed by one line per problem, or Project validation passed.
 - divergence — ... for each data-source difference between members (legal, but declared).
 - blocked — `<member>: <status> — <detail>` for each subdirectory a run cannot use.
 - A closing count: `[validate] N file(s) checked, N problem(s), N blocked member(s).`
- With a **standalone experiment** loaded (no Project), the same button checks that one config: `<name> config problems:` and a list, or `<name>: config is valid.`
- The Project card's summary shows `N problem(s) — run Validate YAMLs.` whenever the open Project has problems.
- A Project Script step `validate_project` runs the same check and **stops the script** if anything is wrong.
- The Project Report lists a member's problems under "Why it cannot be analysed as configured" for any Focus that has not been analysed.

Member problems are prefixed with the member's name (`cohort_a: ...`).

survival_config.yaml problems

Message	What to do
survival_config.yaml, line N: not valid YAML — <code><problem></code> . (<code><path></code>)	The file is not YAML at all, so none of the checks below can run (see <i>YAML syntax errors</i> below).
Config must be a YAML mapping. / ... must contain a YAML mapping, got list.	The file must be <code>key: value</code> lines at the top level, not a list or a single value.
Unknown experiment_type '...'. Known types: <code>standard_lifespan</code> , or omit the key.	Correct or delete <code>experiment_type</code> . When this appears, the other checks are skipped until it is fixed.
<code>`input.format`</code> must be <code>auto</code> , <code>excel</code> , <code>long</code> or <code>wide</code> (got '...').	Fix the value. See The input: section .

<code>`input.format: wide`</code> needs .../ <code>`input.factor_names`</code> .../ <code>`input.factor_levels`</code> .../ <code>`input.col_mapping`</code> ...	A wide-format CSV's keys are missing or malformed. See The input: section .
<code>`exclusions:`</code> must be a mapping, e.g. <code>{group: default}`</code> .	Write <code>exclusions:</code> then an indented group: <code><name></code> .
<code>`global:`</code> must be a mapping.	See The global: section .
<code>`global.time_unit`</code> must be a string (e.g. <code>days</code>).	Write a word.
<code>`global.assume_censored`</code> must be true or false.	Use unquoted <code>true</code> or <code>false</code> .
<code>`global.min_n_per_chamber`</code> must be a non-negative integer (0 turns the check off).	A whole number ≥ 0 .
No input data file found in <code><dir></code> (looked in <code>data/</code> and the directory root).	Put the <code>.xlsx/.csv/.tsv</code> in <code>data/</code> or the directory root, or fix <code>data_file:</code> .

focuses: problems

Message	What to do
<code>`focuses:`</code> must be a mapping of Focus name $\rightarrow$ <code>{factors: ...}</code> .	Each Focus is a name followed by an indented body.
A Focus has an empty name.	Give it a name.
Focus <code>'...'</code> would write to <code>analysis/<slug>/</code> , which is reserved for pre-Focus results; rename it.	<code>plots, statistics</code> and <code>data_output</code> cannot be Focus names.
Focus <code>'...'</code> and Focus <code>'...'</code> would write to the same output directory (<code><slug></code>); rename one.	Two names that differ only in characters replaced in folder names (spaces, slashes, etc.) clash. Rename one.
Focus <code>'...'</code> must be a mapping with a <code>`factors:`</code> key.	Add a <code>factors:</code> block.
Focus <code>'...'</code> names no factors – give <code>`factors:`</code> at least one factor with its levels.	List at least one factor.
Focus <code>'...':</code> factor <code>'...'</code> lists no levels.	Give the factor at least one level.
Focus <code>'...':</code> factor <code>'...'</code> lists a level twice.	Remove the duplicate.
Focus <code>'...':</code> <code>`reference:`</code> must map factor $\rightarrow$ level.	e.g. <code>reference: {Density: "40x"}</code> .
Focus <code>'...':</code> reference names factor <code>'...'</code> , which the Focus does not name.	Remove it, or add the factor to <code>factors:</code> .
Focus <code>'...':</code> reference level <code>'...'</code> is not one of <code><factor></code> 's levels (...).	Choose one of the listed levels. Watch quoting: <code>40x</code> and <code>"40x"</code> are the same, but

	20 and "20" may not be.
Focus '...': `display_names:` must map treatment → value. (or colours:)	Use treatment label: value pairs.

omit: problems

Message	What to do
`omit:` must be a mapping, e.g. `{analyses: [parametric_aft]}`.	See The omit: section .
`omit.<key>` is not something a run can leave out (use analyses or plots).	Only analyses and plots are allowed.
`omit.<kind>` must be a list of ids.	Write a list.
`omit.<kind>` names '...', which no run produces. Known: ...	A misspelt id omits nothing. Use one of the listed ids.

project.yaml problems

Message	What to do
project.yaml: `defaults: focuses:` is ignored – a member never inherits Focuses. ...	Remove it. Copy Focuses into members with Copy Focuses from....
project.yaml: `defaults: factors:` predates Focuses. ...	Analyse each member once, then remove it.
project.yaml: Unknown experiment_type '...'. ...	Fix defaults: experiment_type. Members are not checked until it is fixed.
<member> is a <type> but the Project's type is <type>. ...	Make the member's type match the Project's.
<path> is not a Project – no project.yaml. ...	Shown when opening a folder without one. Use Create project... or Initialize existing directory...

Blocked members (layout)

A subdirectory of a Project that a run cannot use is a **Blocked Member**. It is listed by Validate YAMLs and the Batch preflight. The rest of the Project still runs.

Status and detail	Fix
no config – holds <file> but no survival_config.yaml	Scaffold one: double-click its row in the members table, or Create config in <i>Experiment configs....</i>
no config – holds N data files but no survival_config.yaml	Scaffold the config first, then name the data file.
no data – survival_config.yaml but no .xlsx/.csv/.tsv in data/ or the directory root	Add the data file.
no data – `data_file: ...` does not exist (looked at ...)	Correct the path in data_file:.
ambiguous – N data files in data/	

(...) – which is the experiment?	Set data file... in <i>Experiment configs...</i> (or the preflight) writes <code>data_file:</code> for you.
unreadable – cannot be listed (permissions?)	Fix the folder's permissions.
bad config – <code>survival_config.yaml</code> , line N: not valid YAML – ...	Fix the YAML at that line (see <i>YAML syntax errors</i> below).

When loading, the same conditions appear as `<name>: N data files in data (...)`. Name one with `data_file:` in `survival_config.yaml`. **and** `<name>: no .xlsx/.csv/.tsv found in ...` or its `data/` subdirectory.

Blocked Focuses (need the data)

These are not part of Validate YAMLs. They appear in the Hub's members table (red row, reasons in the tooltip), in the Batch preflight, and as Focus '...' is blocked: when a run starts. See [Analysed, Out of Date, Blocked and Orphaned results](#).

Message	Fix
Focus '...' names factor '...', which the data file does not have (factors: ...) – rename it to '...'?	The Design sheet's column was renamed. Accept the suggested rename (preflight) or edit the Focus.
Focus '...' names level '...' of <factor>, which the data file does not contain (levels: ...) – rename it to '...'?	As above, for a level.
Focus '...': treatment '...' is in the data file but has no individuals once exclusions are applied – change the Exclusion Group or drop that level from the Focus	QC removed every chamber of a cell.
Focus '...' selects no individuals – edit the Focus	The Focus's filters match nobody.

When adding data to a Project

Adding a workbook or directory checks the workbook's headers first. These messages mean it is not a usable DLife workbook:

- `<file>`: not a file. / `<file>`: not an .xlsx workbook. / `<file>`: cannot be opened as an Excel workbook (...).
- `<file>`: no 'Design' sheet. / `<file>`: no 'RawData' sheet.
- `<file>`: Design sheet has no 'Chamber' column. (or 'SampleSize', 'StartTime')
- `<file>`: Design sheet has no treatment factor columns after 'StartTime'.
- `<file>`: RawData sheet is missing AgeH, Censored, ...
- `<dir>` has no `data/` subdirectory. / No .xlsx file in `<dir>/data`. (Add directory)

See [Input data formats](#) for the sheet layout.

Pitfalls

- **Valid is not the same as runnable.** Validation does not open the data. A config can pass and still fail to load, for example with a wrong `time_col`.
- **YAML syntax errors.** A file that is not valid YAML at all (a tab used for indentation, an unclosed quote, a missing colon) is reported as `<file>, line N: not valid YAML - <what the reader found>. (<path>)`. The line is where the broken construct starts: the line with the unclosed quote, or the key missing its colon. A tab gets the hint "(YAML indents with spaces, never tabs)".
 - A member's `survival_config.yaml`: the member is still listed, with that message as its problem (and as a `bad config Blocked Member`), and the rest of the Project validates and runs normally. Nothing runs or writes under the broken file until it is fixed, because a write would replace your file with defaults.
 - `project.yaml`: the Project cannot be opened, since its defaults are unknown. Selecting it shows the message, and the folder stays selected, so **Validate YAMLS** reports it too (`<folder>: validation problems: and the line`). Fix the file and select it again. A standalone Experiment Directory whose `survival_config.yaml` does not parse is treated the same way.
- **Divergence is not a problem.** `divergence - ...` lines are informational. They also appear in the Project Report's Divergence Note.

See also

- [survival_config.yaml reference](#)
- [project.yaml and Project Defaults](#)
- [The Batch preflight](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Troubleshooting](#)

Projects and batches

Creating and opening a Project

The **Create/Load** card is the first card in the Project panel (click the **Project** tile). It has one button for each state a folder can be in before it is a Project, an editor for the Project that is open, and a validator. Below the buttons, a summary describes what is loaded.

The buttons

Button	Use it when	What it does	Disabled when
Open project...	the folder already has a <code>project.yaml</code>	Opens a folder picker. The folder you choose becomes the selection: a Project, a Batch, or a standalone Experiment Directory (a Project is never required). Choosing the Project that is already open re-reads it from disk, so	never

		changes made outside the Hub show up.	
Create project...	the Project does not exist yet	Opens the Project editor (below). The directory you name is created if needed, and <code>project.yaml</code> is written into it.	never
Initialize existing directory...	you have a folder, usually with experiment subfolders in it, but no <code>project.yaml</code>	Opens the Project editor in <i>initialize</i> mode: the folder keeps its own name, its subfolders become the members, and <code>project.yaml</code> is written there. Nothing is moved.	never
Edit config...	a Project is open	Opens the Project editor on the open Project's <code>project.yaml</code> .	no Project is open. It only edits: to give a folder its first <code>project.yaml</code> , use <i>Initialize existing directory...</i>
Validate YAMLS	any time	Checks the Project's <code>project.yaml</code> and every member's <code>survival_config.yaml</code> , then writes the results to the Output log. See below.	never (with no Project and no experiment loaded it warns <i>No Project selected.</i>)

The Project editor

Create project..., *Initialize existing directory...* and *Edit config...* share one dialog.

- **Directory:** the Project folder. Type a path or use **Browse....** In create mode a path that does not exist yet is created. Choosing a folder that is already a Project loads its values, and saving edits it.
- **Project name:** written as `name:`. It defaults to the folder name. In initialize mode it is read-only and is always the folder's own name. The Project Report's file name always comes from the folder name, not from this field.
- **Question:** the one question the members address. The Project Report's cover and the AI narrative both read it.
- **Project Defaults — seeded into every new member:**
 - **Experiment type:** the one value every member must share. In initialize mode it is preset to the type of the first subfolder that already has a config.
 - **Global:** one field for each of the type's `global:` settings. For Standard Lifespan these are `time_unit` (default `days`), `time_label` (default `Age (days)`), the

`assume_censored` checkbox (default on) and `min_n_per_chamber` (default 5). A text field left empty is not written, so the type's default applies.

There are no factor fields. Factors are discovered from each member's data file, and each member names its own Focuses. Keys the dialog does not show (`scripts:`, `experiment_scripts:`, `other_defaults:` entries, anything from a newer version) are carried through a save unchanged.

Save writes `project.yaml` and opens the Project. A new `project.yaml` is created with a Project Script named `batch` (what a Batch Run runs here) and an empty `experiment_scripts:` list.

When Save refuses or asks

- No directory entered: *Choose a directory first*.
- Initialize mode, folder does not exist: use *Create project...* instead.
- Initialize mode, folder is already a Project: open it and use *Edit config...*
- The folder holds a `survival_config.yaml` (it is an Experiment Directory): the dialog offers to create the Project on its parent folder instead, so this folder becomes a member.

The summary line

Under the buttons, the card describes the open Project: **name · Experiment Type · N member(s)**, then, when they apply:

- *N directory(ies) with no config:* ...: subfolders that are not members yet (see [The Experiments card — members](#));
- *Question:* ...;
- *Divergence:* ...: the data-source settings (time unit, censoring, exclusion group) on which members differ;
- *N problem(s) — run Validate YAMLs.*

With nothing open it reads *No Project loaded*.

Validate YAMLs in detail

With a Project open, the Output log lists:

- *Project validation passed.* or *Project validation problems:* followed by each problem. Problems include a member whose Experiment Type differs from the Project's, each member's own config problems, and a `defaults:` that contains `focuses:` (ignored, because Focuses are never inherited) or a pre-Focus `factors:`;
- each divergence;
- each **blocked** member folder: data but no config, a config but no data, or several candidate data files (ambiguous);
- a final line: `[validate] N file(s) checked, N problem(s), N blocked member(s).`

With no Project open but a standalone experiment loaded, it validates that experiment's config alone. See [Validating configuration](#).

See also

- [project.yaml and Project Defaults](#)
- [The Experiments card — members](#)

- Validating configuration
- Batches, Projects, Experiments and Focuses
- Experiment Types

The Experiments card — members

The **Experiments** card is the second card in the Project panel. It lists the open Project's Member Experiments, along with any subfolders that are not members yet, and holds the five ways to make a new member. Double-click a member to load it. The card is dimmed, and all five buttons are disabled, until a Project is open.

The members table

There is one row per member (a subfolder with a `survival_config.yaml`), in name order, followed by one row per subfolder that has no config yet. Every value is read from the config and the saved Run Summaries. Nothing is re-analysed to fill the table, and it re-reads from disk after every task and every change made in the Hub.

Column	Shows	Possible values
Member	the folder name	the member's directory name
Config	whether the folder has a <code>survival_config.yaml</code>	yes, or missing (in red) for a folder that is not a member yet
Focuses	how many Focuses the member has, and the first three names	2: Unfiltered, FemalesOnly, 5: A, B, C ... when there are more than three; – when none can be resolved (no declared Focus and the data file cannot be read, for example a member with no data yet)
N	individuals in one analysed slice	for the loaded member, the Active Focus's total when it has current results; otherwise the largest Focus with current results. Hover over the cell to see which Focus it is. – when no Focus has current results (an Out of Date result is not counted). It is never a sum over Focuses, which overlap
Analysed	how far the member's Focuses are analysed (see below)	no, 1/3, a timestamp, yes, re-run needed, 1/3 · 1 blocked, blocked: no data, blocked: ambiguous
Exclusions	the active Exclusion Group in the member's config	the group name, or none

The Analysed column

Hover over the **N** and **Analysed** column headers for a reminder of what they count. The first rule that applies wins:

- blocked: no data / blocked: ambiguous: the member itself cannot run — its config names no data file the loader can find, or the folder holds several candidate data files and no `data_file`: says which is the experiment (see *Red rows* below).

- $k/n \cdot b$ blocked: at least one Focus is **Blocked** (it names a level the data lacks, or the active Exclusion Group empties one of its cells). Here k is the number of Focuses with current results, n the number of Focuses, and b the number blocked.
- re-run needed: at least one analysed Focus is **Out of Date**. Its saved results were produced under a different Focus definition (factors, levels, Reference Levels) or a different Exclusion Group from the one the config now declares.
- k/n : some Focuses have no current results yet.
- the date and time of the most recent run (for example 2026-10-02T14:03:11): every Focus is analysed and current. It shows `yes` if no run recorded a time.
- no: the member has no Focuses that can be resolved.

Red rows

A member row turns red when the member is blocked, or any Focus is blocked or analysed but out of date. Hover over the row for the reasons: for a blocked member, what is wrong and how to fix it (for example *Blocked — 2 data files in data/ (a.xlsx, b.xlsx) — which is the experiment? Name the experiment's file with Experiment configs... → Set data file..., or for no data file, Put its .xlsx/.csv/.tsv in data/ (or the directory root), or point data_file: at it.*), then one line per affected Focus, such as *FemalesOnly: blocked — ... or Unfiltered: out of date — ...*. A missing row is red too, and its tooltip says why (for example *holds cohort_c.xlsx but no survival_config.yaml*).

Double-click

- On a member row: loads that member and opens the [Experiment panel](#), where you choose the Active Focus.
- On a missing row: asks *Scaffold one from the Project Defaults and make it a member?* **Yes** writes a minimal config (as *Initialize existing directory...* does) and the row becomes a member.

Selecting a row (single click) also tells **Plot editor...** on the Actions card which member to open.

Making a member

The first three buttons cover folders inside the Project.

Button	When	What it does
Create experiment...	the member folder does not exist yet	Asks for a folder name, creates it with an empty <code>data/</code> , and writes a minimal <code>survival_config.yaml</code> that inherits everything from the Project Defaults. It then offers Add data file... (copies a <code>.xlsx</code> , <code>.csv</code> or <code>.tsv</code> into <code>data/</code>), Copy config from... (replaces the scaffold with another member's config, refused if its Experiment Type differs), or Later . It refuses a name that is not a single folder name, and a folder that already exists (use <i>Initialize</i>

		for that).
Initialize existing directory...	the folder is in the Project but has no config	Lists the Project's subfolders that have no config, each with its state (for example <code>empty</code> , <code>no config</code>), and scaffolds a minimal config in the one you pick. Nothing is moved, because the loader already finds a data file at the folder root or in <code>data/</code> . If the folder holds several candidate data files, it then asks which one is the experiment and writes <code>data_file:</code> into the config. When every folder already has a config it says so and points you to <i>Create experiment....</i> A folder that cannot be listed (<code>unreadable</code>) is refused, and has to be fixed outside the app.
Experiment configs...	to see or fix many at once	Opens the bulk view (below).

The two buttons under **Bring one in from outside** reach outside the Project.

Button	What it needs	What it does
Add directory...	a folder outside the Project with a <code>data/</code> subfolder holding a <code>.xlsx</code> that validates as a DLife workbook	Copies the whole folder into the Project. If it has no <code>survival_config.yaml</code> , a minimal one is written (with <code>data_file:</code> when <code>data/</code> holds more than one data file). A folder that is already a direct child of the Project is refused with a pointer to <i>Initialize existing directory....</i> A destination that already exists is refused rather than merged.
Add experiment...	one DLife workbook (<code>.xlsx</code>)	Creates a member named after the file (<code>cohort_a.xlsx</code> becomes <code>cohort_a/</code>), puts the workbook in its <code>data/</code> (moved if it was already inside the Project tree, copied otherwise), and writes a minimal config.

A workbook validates as DLife when it has Design and RawData sheets with the columns each needs and at least one factor column after `StartTime`.

A config written by any of these routes is **minimal**: it states the Experiment Type, plus, for an

adopted workbook, the `input: format` and `assume_censored` if the workbook's PrivateData sheet disagrees with the Project's default. Everything else is inherited, so later edits to the Project Defaults keep reaching the member. A config copied in with *Copy config from...* restates whatever it contains. The log then names the keys that will no longer follow the Project.

The Experiment configs dialog

This dialog lists every subfolder of the Project that is a member or could become one.

Column	Values
Directory	folder name
Config	yes or missing
Data files	the candidate data files found (in <code>data/</code> , else the folder root), or none
Status	ok, no config, no data, ambiguous (several files and no <code>data_file:</code>), unreadable, or empty

Hover over a row to see the detail behind its status.

Button	Does	Enabled when
Create config	scaffolds the selected folder's config from the Project Defaults	the selected row has no config
Create all missing	scaffolds every missing config, after confirming the list	any row has no config
Set data file...	asks which candidate file is the experiment and writes <code>data_file:</code>	the selected row is ambiguous
Edit config...	opens the member's <code>survival_config.yaml</code> in your system's YAML editor (there is no built-in config editor)	the selected row has a config

Double-clicking a row edits it if it has a config, or creates one if it does not.

See also

- [Creating and opening a Project](#)
- [The Experiment panel](#)
- [survival_config.yaml reference](#)
- [project.yaml and Project Defaults](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Input data formats](#)

Project actions — reports, figures, Plot Editor

The **Actions** card is the third card in the Project panel. Its buttons act on the open Project as a whole: they build and open reports, author Publication Figures, and render them for every member. The card is dimmed until a Project is open. Every action reads **saved** results. None of them analyses anything.

Project report

Builds the **Project Report** from each member's saved results and writes two files to the Project

folder: `<project-folder>_report.pdf` and `<project-folder>_report.md`. If the PDF cannot be rendered, the Markdown is still written and the log says why.

The report has a cover with the Project's question, a **Focus Inventory** listing every Focus of every member with its state, a **Divergence Note** when members' data-source settings (censoring, active Exclusion Group, time unit) differ, and then one independent section per Focus. A Focus that has never been analysed gets a "not analysed" section. An Out of Date Focus is named as such and its old numbers are not shown. Nothing is pooled across members or Focuses, and nothing is re-run. See [The Project Report](#).

The button runs in the background, like every task. Watch the Output tab for `Wrote ... lines`. With no Project open it warns *No Project selected*. The AI panel's **Project report with narrative** builds the same report with an [AI narrative](#).

View reports

Opens the Project Report PDF and **every member report PDF** (one per analysed Focus: `analysis/<focus>/<member>_report_<focus>.pdf`) in your system's PDF viewer, all at once. The Output log says how many were opened.

State	Button
no Project open	disabled, with the tooltip <i>Select a Project first</i> .
no Project Report and no member report on disk	disabled, with the tooltip <i>No reports yet — run Project report first</i> .
either kind exists	enabled. The tooltip says how many of each will open

Member reports are written by **Run analysis** (on the experiment's Analyze panel) or by any script that runs the analysis. You do not need a Project Report to open them.

Plot editor...

Opens the [Plot Editor](#), where you author Publication Figures: each figure's Spec (labels, axis limits, a narrowing of treatments) and its Style. A Spec belongs to one experiment, so the editor always opens on one member, chosen in this order:

1. the member **selected** (single-clicked) in the members table on the Experiments card;
2. otherwise the **loaded** member;
3. otherwise the only member, if there is just one;
4. otherwise it asks *Author which member's figures?*

With no members it warns that there is nothing to author. The editor is the only way into Publication Figure authoring.

Render publication figures

Renders the Project's **curated** Publication Figures (those saved into the Project's `plot_specs.yaml` from the Plot Editor) for every member, in the format chosen in the box beside the button:

- `svg` (default) or `pdf`: vector output with editable text, the point of a publication figure;
- `png`: for a slide.

Files go to each member's `figures/<focus>/` folder, one set per Focus, each file named for its Focus. A member with nothing curated is skipped (`nothing curated — skipped`). Within a member, a Focus that is not analysed, or whose results are Out of Date, is named in

the log and skipped, because a figure of it would describe a slice the config no longer declares. If no member has curated figures, the log says *nothing rendered*. *Curate figures in the Plot Editor first*.

This button runs the same `render_publication_figures` action a Project Script step does, so the button and a Batch Run render the same figures. See [Publication Figures, Specs and Styles](#).

Pitfalls

- **Reports never refresh themselves.** After re-running members, click **Project report** again. Otherwise the PDF still shows the earlier results.
- **The Project Report file is named after the folder**, not the Project's `name` :.
- **Render needs current results.** Re-run Out of Date Focuses first, or they are skipped.

See also

- [The Project Report](#)
- [The experiment report](#)
- [The Plot Editor](#)
- [Publication Figures, Specs and Styles](#)
- [Project Scripts](#)
- [Where results are written](#)

Project Scripts

A **Project Script** is a saved, re-runnable list of project-level steps, stored in `project.yaml` under `scripts:`. It is how you analyse every member, render figures and rebuild the Project Report in one action. A Batch Run is the same thing done in many Projects at once. The **Scripts** card, the last card in the Project panel, runs and edits them. It is dimmed until a Project is open.

The Scripts card

- **The script picker** lists the Project's own scripts first (from `project.yaml`), then the built-ins **Standard pipeline** and **Report pipeline**, unless the Project already has scripts with those names.
- **Run script** runs the picked script in the background. Progress streams to the Output tab as `[1/3] Run in experiments, [2/3] ...`, with each member's lines prefixed `[member]`. A script with a step naming an unknown action refuses to start. With no Project open it warns *No Project selected*.
- **Edit scripts...** opens the [Script Editor](#) on the Project's `project.yaml`, at the *Project scripts* level, whether or not an experiment is loaded. Switch the level to *Central experiment scripts* to edit `experiment_scripts:`. When you save, the Hub re-reads the Project, so the picker lists the saved scripts at once.

The batch script

Every new `project.yaml` is created with one Project Script named `batch`. It is the script a Batch Run executes in this Project unless another is designated:

```
scripts:
- name: batch
  notes: Created with the project, and what a Batch Run runs here ...
  steps:
    - action: run_in_experiments
```

```

    script: Standard analysis
  - action: render_publication_figures
  - action: project_report
experiment_scripts: []

```

It is an ordinary script: you can edit, rename or replace it. A Project whose `scripts:` list is empty cannot be run by a Batch Run that has no designated script. The run fails that Project, says to author a script, and moves on. A `project.yaml` with no `scripts:` key at all gets the batch script added on its next save.

Project actions

Action (YAML key)	Parameters	What it does
Validate project (<code>validate_project</code>)	none	Checks every member shares the Project's Experiment Type and that each config is valid. Any problem fails the script . Divergences are logged without failing.
Run in experiments (<code>run_in_experiments</code>)	<code>script</code> : an Experiment Script name; <code>only</code> : member names (blank = every member)	Runs the named Experiment Script in each member, <code>continue-on-error</code> : a failing member is logged and the others still run.
Render publication figures (<code>render_publication_figures</code>)	<code>format</code> : svg (default), pdf or png	Same as the Actions card button: curated figures for every member and current Focus.
Project report (<code>project_report</code>)	<code>formats</code> : default pdf, md; <code>with_narrative</code> : true/false	Writes the Project Report from saved results. Fails the script if nothing was written.
Generate AI narrative (<code>generate_ai_narrative</code>)	<code>provider</code> : blank, anthropic or openai	Writes the narrative. A provider failure is logged and the script continues.

How `run_in_experiments` finds the Experiment Script

For each member, the named Experiment Script is looked up in this order:

1. the Project's central `experiment_scripts:` in `project.yaml` (one recipe serving every member);
2. the member's own `scripts:` in its `survival_config.yaml`;
3. the built-in of that name. The only built-in is `Standard analysis`:
`run_in_focuses`, then `run_analysis`.

Every new `survival_config.yaml` is created with its own `Standard analysis` script (the same two steps), so you can see and edit it per member. Because `run_in_focuses` runs the rest of the script once per Focus, the batch script analyses **every Focus of every member**, not just the Active Focus.

A member with no script of that name is logged as skipped and counted as a failure. A member whose script contains a step its Experiment Type does not provide **stops the whole Project's run**: this is a hard error, not a skip. At the end, the log reports

`run_in_experiments(<name>):` k/n member(s) completed, and the script reports Completed with N member failure(s): ... when any member failed.

The built-in Project Scripts

Name	Steps
Report pipeline	<code>run_in_experiments (Standard analysis) → render_publication_figures → project_report</code> . When no member has curated figure specs, the render step is dropped up front and the log says so.
Standard pipeline	<code>validate_project → run_in_experiments (Standard analysis) → render_publication_figures → project_report</code> . Validation failures stop it before anything runs, which is why Report pipeline suits unattended runs better.

The seeded `batch` script has the same steps as Report pipeline, but as a copy in your file it always keeps its render step. With nothing curated, that step just logs *nothing rendered*.

When a picked name is resolved

Run script looks the picked name up in the Project's own `scripts`: first, then among the built-ins. A Project script you name `Report pipeline` therefore overrides the built-in.

See also

- [Experiment and Project Scripts](#)
- [The Script Editor](#)
- [Script action reference](#)
- [Running Experiment Scripts](#)
- [Batch runs](#)
- [project.yaml and Project Defaults](#)

Batch runs

A **Batch Run** executes one Project Script in every checked Project inside a folder, unattended and continue-on-error. Use it to re-analyse and re-report many Projects overnight. A Batch has no analysis of its own and never combines results across Projects. Open the **Batch** tile to use it.

Choosing a Batch

Choose batch folder... picks the folder that holds your Projects. Discovery is **recursive**: the app walks down through any grouping folders (`Sept2026/`, `Archive/2025/`) until it finds a Project (a folder with `project.yaml` and at least one member), then stops there and never looks inside it. Projects can therefore sit at any depth, and a Project's own subfolders are never mistaken for Projects. The walk skips hidden (dot) folders, does not follow symbolic links, and logs any folder it could not read.

The walk is done once when you choose the folder. **Rescan** walks it again, after you add or repair Projects outside the app, re-reads `batch.yaml`, and logs `[batch] rescanned ...: N project(s), N blocked member(s)`.

If the folder is itself a Project, it is opened as a Project rather than a Batch, and the Batch tile says *selection is a project — open its parent to batch*. A very large folder (tens of thousands of subfolders) stops the scan early. The log then says so and suggests choosing a folder closer to the Projects.

Until a Batch is selected, the table, the script picker, **Run batch** and **Rescan** are disabled. **Choose batch folder...** and the suppress-tabs box are always available.

The Projects table

One row per Project found, in key order.

Column	Shows
Project	a checkbox, and the Project's key : its path relative to the Batch folder (Sept2026/ProjA, or just ProjA at the top level). Hover to see the full path
Type	the Project's Experiment Type, or <code>error: ...</code> if its <code>project.yaml</code> cannot be read
Members	usable members / all member folders, for example 4/5. A folder that holds data but no config counts in the total
Report	<code>yes</code> if the Project Report PDF exists, <code>no</code> if not (– on error)
Status	<code>ok</code> , or <code>N blocked</code> when some member folders cannot be used

A row with blocked members is shown in red. Hover over it for each blocked member and the reason. The checkbox decides whether the Project joins the next run. A row you have not touched is checked exactly when the Project has a usable member, so a Project that cannot run starts unchecked and becomes checked once it is repaired. Your own checks and unchecks are kept whenever the table refreshes, including after a rescan or a repair, and after opening a Project from the table and coming back. Choices you make in the preflight are carried back to this table when you confirm the run.

- **Double-click** a row to open that Project in the Project panel (the selection moves to the Project, and you reach the Batch again by choosing its folder).
- **Right-click** a row for **Fix blocked members in** `<key>...`, which is enabled only when the Project has blocked members and opens the preflight with that Project selected, and **Open** `<key>`.

The script picker

Script chooses what runs in each Project:

- **Each project's own 'batch' script (default)**: every Project runs its own script named `batch`, or its first script if none has that name. A Project with no scripts fails, and the run continues.
- A named script: the Batch folder's central `project_scripts:` from `batch.yaml` (when there is one), then the built-ins **Standard pipeline** and **Report pipeline**. A named script is looked up in each Project in this order: `batch.yaml`'s `project_scripts:`, then the Project's own `scripts:`, then the built-ins. A name found nowhere fails that Project only.

If `batch.yaml` designates a script (`script:`), the picker starts on it, even when it names a script only some Projects carry. Choosing in the picker **saves** the choice as `batch.yaml`'s designation (creating the file if needed, and keeping its `project_scripts:`), so a command-line `batch` run and the next session use the same script. Choosing **Each project's own 'batch' script (default)** removes the designation; it never creates a `batch.yaml` that does not exist. A `batch.yaml` in a nested folder is ignored, and the run log names it.

Run batch

Run batch always opens the **Batch preflight** first. It lists exactly which Projects will run, with

their blocked members and Blocked Focuses, and lets you repair them, uncheck Projects, or cancel. Nothing runs until you confirm there. Projects you leave unchecked are not touched at all.

During the run, every log line is prefixed with the Project's key ([Sept2026/ProjA] ...). Before each Project's script starts, its blocked members and Blocked Focuses are named (*will not be analysed*). A failing Project is logged (FAILED: ...) and the Batch moves on to the next.

The run ends with one summary line, for example:

```
Batch run: 3 Project(s) completed; incomplete: ProjB (3/4 members, 1 Focus(es) bl
Batch run: 2/3 Projects completed; failed: ProjC.
```

Incomplete lists Projects that succeeded while leaving members or Focuses out, so "completed" is never read as "analysed everything".

Suppress plot tabs during batch runs

On by default. A Batch Run would otherwise open a figure tab for every figure of every member. With the box ticked, figures are still written to disk, but no tabs open, and the log notes each figure that was not shown. It affects Batch Runs only. Figures you generate on a single experiment always open.

See also

- [The Batch preflight](#)
- [Project Scripts](#)
- [Batches, Projects, Experiments and Focuses](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [The Analysis Hub](#)

The Batch preflight

The **Batch Run — review** window always opens when you click **Run batch** on the Batch panel, even when nothing is wrong. Because Projects are discovered recursively, the folder you picked does not tell you what will run. This window is the one place that states the target list, and the last point at which you can change it. Nothing in it blocks a run: a Project with blocked members still runs its healthy ones.

The heading

The heading reads, for example, D:\Lifespan — 5 project(s) found, 4 checked to run, 2 blocked member(s), 1 blocked focus(es). A note below the buttons adds:

- *N directory(ies) skipped — see the Output log after the run.* (unreadable folders, symbolic links, folders marked as Projects with no members);
- *The scan stopped early ...* when the folder is too large to walk fully;
- *Nothing to run ...* when no Project was found.

The tree

One top-level row per Project, with child rows for what is wrong inside it.

Column	Project row	Child row
Project	checkbox and key (path relative to the Batch folder)	a blocked member's folder name, or member · Focus for a Blocked Focus
Members	usable / total member folders	blank

Status	ok, or N blocked, N focus(es) blocked, or both	the member's status (no config, no data, ambiguous, unreadable, bad config), or focus blocked
---------------	--	---

Projects with anything blocked are shown in red and expanded. Hover over a child row for the full reason.

What the statuses mean

- **no config:** the folder holds a data file but no `survival_config.yaml`. *Fixable here.*
- **ambiguous:** the folder has a config, but there are several candidate data files and no `data_file`: naming one. The loader refuses to guess. *Fixable here, one member at a time.*
- **no data:** a config with no `.xlsx/.csv/.tsv` in `data/` or the folder root, or a `data_file`: naming a file that does not exist. Supply the data outside the app.
- **unreadable:** the folder cannot be listed (permissions). Fix it outside the app.
- **bad config:** the member's `survival_config.yaml` is not valid YAML; the detail names the file and line (`survival_config.yaml, line N: not valid YAML - ...`). Fix the file in an editor; the rest of the Project runs.
- **focus blocked:** a Focus names a factor or level the data file no longer contains (**stale**, for example after a level was renamed in the Design sheet), or a cell it keeps has no individuals left after the active Exclusion Group (**empty**). Finding these means reading each usable member's Design sheet. The member's other Focuses still run.

Checking and unchecking

A checked Project runs, and an unchecked one is **not touched at all**. A Project nobody has decided about is checked exactly when it has at least one usable member; one with no usable member starts unchecked, because it could only fail. You can still check it.

What you decided is kept, wherever you decided it. On opening, the Batch panel's table is read as your choices: a Project you unchecked there starts unchecked here, and one with nothing usable that you checked there starts checked. Your clicks in this window are remembered too. After a repair or a **Rescan** the tree is rebuilt, and every Project keeps the box you gave it; the ones you never touched follow what they can do *now* — so a Project you have just repaired joins the run, unless you had unchecked it. When you click **Run batch**, your choices here are carried back to the Batch panel's table.

Buttons

Button	What it does	Enabled when
Fix selected...	On a no config row, writes a minimal config from that Project's Defaults. On an ambiguous row, asks which file is the experiment and writes <code>data_file:</code> . On a stale Blocked Focus row, shows the Focus rewritten with each missing level replaced by its close match in the data file, and asks you to	a row with a one-click fix is selected. For an empty Focus or a stale one with no close match, it stays disabled, and the tooltip suggests editing the Focus or changing the Exclusion Group

	confirm. Its saved results are then Out of Date until re-run.	
Scaffold every missing config	Writes a minimal config into every no config member across the whole Batch. Ambiguous members are left alone, because which file is the experiment is your decision.	always (it says so when there is nothing to scaffold)
Rescan	Walks the Batch folder again, for changes made outside the app.	always
Run batch	Closes the window and starts the run on the checked Projects.	at least one Project is checked
Cancel	Closes the window without running. Repairs you already made stay on disk.	always

Each repair is logged in the Hub's Output tab with a `[preflight]` prefix. Whether you run or cancel, the Hub rescans the Batch afterwards, so its table reflects your repairs.

See also

- [Batch runs](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [The Experiments card — members](#)
- [The Focus window](#)
- [Exclusion Groups](#)

The Experiment panel

The **Experiment** panel is where you work on one loaded experiment. At the top, the **Focus** card chooses which slice of the data everything else acts on. Below it, five sub-tiles (**QC · Analyze · Plots · Scripts · AI**) open the panels that do the work.

The panel opens by itself when you double-click a member in the Project panel's members table. You can also open it by clicking the **Experiment** tile once an experiment is loaded, either a member or a standalone Experiment Directory. With nothing loaded, the tile is dimmed and clicking it only logs *Load an experiment first*.

The Experiment tile's summary shows loaded: `<experiment>` and focus: `<Active Focus>`.

The Focus card

The Active Focus selector

The drop-down lists every Focus of the experiment as `name — state`, where the state is one of:

- `not analysed`: no saved results under this Focus;
- `analysed`: saved results that match the current definition;

- **out of date:** saved results produced from something that has since changed: the definition (factors, levels, Reference Levels), the Exclusion Group or its chambers, the data file, the censoring policy, or the analyses and plots left out (the reasons are listed);
- **blocked:** the Focus cannot run as declared (it names a level the data lacks, or the active Exclusion Group empties one of its cells).

A config with no `focuses`: block shows one Focus, `Unfiltered`. The Focus you pick is the **Active Focus**: the one the Analyze, Plots and QC viewer act on, and the one a script with no `run_in_focuses` step runs under. Switching it is navigation only. Nothing is written to disk, and every Focus's results stay where they are. The log records `Active Focus: <name>`. When you first load an experiment, the Active Focus is its first Focus.

New... and Edit...

- **New...** opens the [Focus window](#) with a new Focus (named `New focus`) over every discovered factor, ready to narrow down.
- **Edit...** opens the same window on the Active Focus, where you can edit, rename, duplicate or delete Focuses, import them from the workbook's `DefinedPlots` sheet, or copy them from another member.

Nothing is written until you click **Save** in that window. After a save, the Focus that was selected there becomes the Active Focus. If the data file cannot be read, the window does not open and a message says why.

The shape line

Under the selector, one line describes the Active Focus before anything runs. Its parts are separated by `:`:

- the definition: each varying factor with its levels in display order and its Reference Level (`Density: 20x, 40x (reference 40x)`), joined by `x`, then `; only Sex = Female for filters and ; pooled over Genotype for factors left out;`
- shape `2x2`: the number of levels of each varying factor. When some implied cells hold no individuals, it adds `(3 of 4 cells);`
- what the Focus is **offered**, for example `offers a comparison between treatments; a figure crossing factors; the Factorial Battery`. Each item is marked `(not computable: ...)` when the data cannot support it. A single-treatment Focus reads *offers the survivorship battery only*;
- when they apply: `not applicable last run: ..., BLOCKED - <reasons>, or out of date - <reasons>`.

If no Focus can be resolved, typically because the member has no data file yet or the file cannot be read, the line says *No Focus could be resolved — is the data file readable?*. See [Focus Shape](#).

Orphaned and pre-Focus results

A row appears under the shape line only when `analysis/` holds results no declared Focus names:

- *Orphaned results (no Focus names them):* `...:` left behind when a Focus was renamed or removed by hand in the YAML (renaming in the Focus window moves its results with it);
- *Pre-Focus results in analysis/ — never adopted and never replaced by a re-run; Delete...* removes them.: output from before Focuses existed. A re-run writes under `analysis/ <focus>/` and leaves these files where they are.

Button	What it does
--------	--------------

Adopt...	Pick an orphaned result folder, then a declared Focus that has no results yet. The folder becomes that Focus's results, and is judged against it like any other result (it may show as out of date). It refuses if there are no orphans (pre-Focus results cannot be adopted) or if every Focus already has results.
Delete...	Pick an orphaned folder or <i>pre-Focus results</i> , confirm, and it is deleted from <i>analysis/</i> . This cannot be undone.

Orphaned results are never bound into a report. See [Analysed, Out of Date, Blocked and Orphaned results](#).

The sub-tiles

Each sub-tile opens its own panel under the Experiment tile. The tiles show only their titles. Hover over one to read its live summary.

Sub-tile	Opens	Hover summary
QC	The QC panel : the active Exclusion Group and the Chamber QC viewer	<code>group: <name or none>, N chamber(s) excluded — the chambers of this data file that the active group and the workbook's ChamberFlags remove from every Focus's next run (chambers a group lists that the file does not have are not counted; a CSV has none to remove)</code>
Analyze	The Analyze panel : a checkbox per analysis, and Run analysis	<code>focus: <name>, then N individuals · <state> once analysed</code>
Plots	The Plots panel : a checkbox per figure, and Generate plots	<code>focus shape: <shape>, headline: <figure id> — the Active Focus's Headline Figure: km_faceted when it crosses factors, km_risk_table otherwise</code>
Scripts	Running Experiment Scripts	numbers of experiment scripts and Project Scripts
AI	The AI narrative	provider count, or no API key

QC, Analyze, Plots and Scripts are dimmed with nothing loaded. The AI sub-tile is dimmed until a Project is open *and* an API key for a provider is configured, because the narrative is written per Project. A dimmed sub-tile still opens its panel, but the cards inside are dimmed too.

QC comes first on purpose: decide which chambers to exclude before you analyse.

Pitfalls

- **The Active Focus is not saved.** It resets to the first Focus when the experiment is loaded again. Scripts run unattended analyse every Focus regardless.
- **A fresh member with no data has no Focus.** Add its data file (or use Experiment configs... to fix an ambiguous one) before anything here works.
- **Out of date is not an error.** Re-run the Focus with **Run analysis** on the Analyze panel.

See also

- [What a Focus is](#)
- [The Focus window](#)
- [Focus Shape — what a Focus is offered](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [The Analyze panel and Run analysis](#)
- [The Experiments card — members](#)

Focuses

What a Focus is

A **Focus** is a *named* slice of the factors and levels found in one data file. It is the unit the app analyses: every analysis, figure and statistic in a run groups individuals by the Focus's `treatment` label, every output file carries the Focus's name, and every report says which Focus it describes. One workbook often holds several experiments (a crowding experiment and a genotype experiment sharing the same chambers sheet, say). Each gets its own Focus, each is analysed independently, and their results sit side by side without overwriting each other.

Factors are discovered, not declared

You never type the factors into the config. The app reads them from the data file:

- **DLife workbook** – the factor columns are the Design sheet's columns after `StartTime`, one row per chamber.
- **CSV/TSV** – the factor columns of the file (see [Input data formats](#)).

Each factor's levels are listed in **first-appearance order**, i.e. the order you typed them in the Design sheet. They are never sorted alphabetically, because controls are usually entered first and alphabetical sorting could silently make a mutant the reference.

The three roles a factor can play

A Focus names some of the discovered factors and, for each one, the levels it keeps. Each discovered factor then plays exactly one role:

Role	How you get it	What it does
Varying	named with two or more levels	its levels make up the treatment labels (e.g. <code>Female/20x</code>)
Filter	named with exactly one level	keeps only individuals at that level; does not appear in the label
Pooled over	not named at all	individuals at every level are combined

Pooling changes every number (N, every curve, every p-value), so the Hub, the Focus window and every report state the role of each factor. The report's Focus section has a "Focus definition" table with Factor, Levels (display order), Role and Reference level columns.

If a Focus varies nothing (every named factor has one level) it is a single treatment. Its label is then built from every named factor so the one curve still has a name.

A Focus is a rectangular product

A Focus is the **rectangular product** of the levels it names: `Sex: [Female, Male] × Density: [20x, 40x]` implies four treatments. If the data never contained one of them (an

unbalanced design that ran `Male/40x` nowhere), that cell is **absent**. An absent cell is not an error. The run reports it ("Cells the levels imply but the data never held (absent, not an error)") and carries on with the cells that exist. A diagonal (two of four existing cells, without the other two) cannot be expressed as one Focus. Make two Focuses instead.

Individuals with no recorded level for a factor the Focus names belong to no treatment. The run counts them, writes a line to the log ("N individual(s) have no recorded level of ... and belong to no treatment — outside this Focus") and repeats it in the report's Focus section. They are never dropped without a mention.

How a Focus is written in `survival_config.yaml`

Focuses live in the `focuses:` block. You normally edit them in [the Focus window](#), but the block can also be edited by hand:

```
focuses:
  Crowding20v40:
    factors:                # levels in display order: legends, facets, cells
      Sex: [Female, Male]
      Density: ["20x", "40x"]
    reference:              # optional; omitted -> first level listed
      Density: "40x"
    display_names: {Female/20x: "F, 20x"}
    colours: {Female/20x: "#b2182b"}
```

- `factors:` – each named factor with its kept levels, **in display order**. The order controls legends, facet panels and plot cells.
- `reference:` – the [Reference Level](#) of each varying factor (the Cox baseline). Optional.
- `display_names:` and `colours:` – what each treatment is called and drawn in, keyed by treatment label. These are used by the Publication Figures drawn in the Plot Editor. They never change a number, so editing them never puts results [Out of Date](#).

Unfiltered: the Focus every experiment has

A config with no `focuses:` block still has one Focus, **Unfiltered**: every discovered factor at every level. It is written into the file (with the levels in their discovered order) the first time the experiment is analysed or its Focuses are opened in the Focus window. After that you can see it, edit it and rename it, and re-sorting the Design sheet later cannot move a Reference Level. Its results go to `analysis/Unfiltered/`.

A config written before Focuses existed that still has a top-level `factors:` block is migrated to one Focus named **Interaction** with exactly those factors and level order, so every Reference Level is preserved. No Unfiltered Focus is added next to it.

Names, folders and files

The Focus name is shown exactly as typed. Folder and file names use a file-system-safe version of it: any run of characters other than letters, digits, `.`, `_` and `-` becomes `_`. A Focus called `Crowding 20v40` writes to `analysis/Crowding_20v40/`, and its files end in `_Crowding_20v40` (for example `plots/kaplan_meier_Crowding_20v40.png` and `run_summary_Crowding_20v40.json`). See [Where results are written](#).

Two Focuses in one experiment cannot share a name or reduce to the same folder name. A Focus also cannot be named `plots`, `statistics` or `data_output`, because those names are reserved for results from before Focuses existed.

The Active Focus

The **Active Focus** is the one the Hub currently shows. QC, Analyze, Plots and AI act on it. It is

chosen in the Focus card at the top of [the Experiment panel](#). Switching it is navigation, not configuration: every Focus's outputs coexist under their own names, so switching changes nothing on disk. (The Exclusion Group, by contrast, is configuration, shared by every Focus in the directory.)

Unattended runs have no Active Focus. A Batch Run or an Experiment Script with no `run_in_focuses` step analyses **every** Focus, and `run_in_focuses` lets a script name which ones. See [Experiment and Project Scripts](#).

Focuses and Projects

Focuses belong to one Member Experiment. They are **never** inherited from `project.yaml`'s `defaults:`, because members rarely share factors and an inherited Focus would be blocked in every member it did not fit. To reuse a design, use *Copy Focuses from...* in the Focus window. It checks each Focus against the receiving member's data before adding it.

Pitfalls

- A Focus that pools over a factor (for example sex) mixes those individuals into every curve. If sexes differ, analyse them in separate Focuses, or keep the factor varying.
- Changing a Focus's factors, levels, level order or Reference Level after it has been analysed makes its saved results **Out of Date** until it is re-run.
- Renaming a Focus by hand in the YAML leaves its old results folder behind as **Orphaned Results**. Rename in the Focus window instead, which moves the results with the name.

See also

- [The Focus window](#)
- [Focus Shape — what a Focus is offered](#)
- [Reference Levels](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Batches, Projects, Experiments and Focuses](#)
- [Defined Plots](#)

The Focus window

The Focus window is where you create and edit an experiment's [Focuses](#). You tick the factors a Focus names, tick and order each factor's levels, pick each varying factor's [Reference Level](#), and name and colour the treatments. A live preview shows N, the treatments, the [Focus Shape](#) and anything that would block the Focus while you edit, so you know before analysing what the Focus will be offered. Nothing you edit is written until you press **Save** (opening the window can write an implicit Focus into the file; see below).

Opening it

Load an experiment (double-click its row in the Project panel's members table, or select a standalone Experiment Directory). The Experiment panel opens with the **Focus** card at the top:

- **New...** opens the window and immediately adds a new Focus.
- **Edit...** opens the window with the Active Focus selected.

The window title is `Focuses — <experiment name>`. If the data file cannot be read, the window does not open and the Hub says why ("The Focus window could not read this experiment's data").

Opening the window writes any *implicit* Focus into `survival_config.yaml` before you edit

anything: Unfiltered, or a migrated Interaction Focus. This makes it a declared Focus you can see and rename, and freezes the data file's level order so a later re-sort of the Design sheet cannot move a Reference Level. It happens even if you then press Cancel: it is part of opening the Focuses for editing, not one of your edits.

The layout

The intro line at the top names the data file the factors come from. Below it the window is split in two:

- **Left:** the list of this experiment's Focuses, with the buttons that add or remove them.
- **Right:** the editor for the Focus selected in the list (its name, one box per discovered factor plus one per factor the data file lacks, the preview, and the treatment table).

Save and **Cancel** are at the bottom.

The Focus list and its buttons

Button	What it does
New	Adds a Focus over every discovered factor at every level, named <code>New focus</code> (then <code>New focus 2, ...</code>).
Duplicate	Copies the selected Focus, including its Reference Levels, display names and colours, as <code><name> copy</code> .
Delete	Removes the selected Focus. Its results stay on disk and are listed in the Hub as Orphaned Results until you adopt or delete them. An experiment must keep at least one Focus ("An experiment needs at least one Focus.").
Import from DefinedPlots...	Proposes a Focus for each plot in the workbook's <code>DefinedPlots</code> sheet (see below). Enabled only for an <code>.xlsx</code> data file.
Copy Focuses from...	Copies Focuses from another member of the same Project (see below). Enabled only when the experiment is a Member Experiment.

Name

The **Name:** field renames the selected Focus. The list updates as you type. If you clear the field, the Focus keeps its previous name. Renaming here is safe. When you save, the Focus's results folder `analysis/<old>/` is moved to `analysis/<new>/` and the name suffix of every file in it is renamed, so nothing becomes orphaned. The save is refused if the new folder already exists (`analysis/<new>/` already exists; delete or adopt it before renaming...).

The factor boxes

There is one box per factor discovered in the data file, titled with the factor's name and with a checkbox in its title:

- **Box unticked** – the factor is **pooled over**.
- **Box ticked, one level ticked** – the factor is a **filter**.
- **Box ticked, two or more levels ticked** – the factor **varies** and labels the treatments.

A ticked box with *no* level ticked is neither: the preview shows *Blocked: factor '...' is ticked but none of its levels is* — tick the levels to keep, or untick ... to pool over it, and **Save** refuses until you do one or the other.

Inside each box:

- **Level list** – every level the data file holds for this factor, each with a tick box. Tick the levels the Focus keeps. The list shows four rows and scrolls if there are more.
- **Order** – drag a level, or select it and use ▲ / ▼, to set the display order. The order controls legends, the facet panels of the faceted KM, and the cells of the interaction plot. It is saved as the order of the level list in the YAML.
- **Reference:** – a drop-down of the ticked levels, giving this factor's Cox baseline. It defaults to the first ticked level, and once chosen it stays with its level when you reorder the list. It is enabled only when the box is ticked and at least two levels are ticked. **Save** writes every varying factor's reference under `reference:`, the default included, so reordering the levels later (here or by hand in the YAML) never moves the baseline. Writing the default down changes no number and does not put saved results Out of Date.

If a Focus names something the data file no longer has (a **stale** Focus, which is blocked), the window shows it so you can remove it:

- A stale **level** appears in its factor's list in red, marked "(not in the data file)", ticked. Untick it to drop it from the Focus.
- A stale **factor** gets a box of its own above the discovered ones, titled "*factor* — not in the data file", ticked and listing the levels the Focus names. Untick the box to drop the factor.

The preview line

Under the factor boxes, a few lines update with every change:

- `<name>: <description>` – each varying factor with its levels and reference, any filters (only `Sex = Female`) and the factors pooled over.
- `N = ... · k treatment(s) · shape ...` – the individuals the Focus selects after the active Exclusion Group and the workbook's chamber flags are applied, the number of populated treatments, and the Focus Shape (for example `2×2`, or `2×2 (3 of 4 cells)`).
- `This Focus offers ...` – which conditional analyses the Focus is offered (a comparison between treatments; a figure crossing factors; the Factorial Battery), with (*not computable: ...*) after any that the data cannot support.
- `Absent cells (never in the data – not an error): ...` – cells the levels imply that the data never held.
- `Blocked: ...` (in red) – each reason the Focus cannot run as declared: a factor or level the data does not contain, or a cell the exclusions have emptied. A Focus with no factors ticked shows *Blocked: Tick at least one factor*.

If the data could not be loaded for the preview, only the description and the stale-name check are shown.

The treatment table

The table lists the Focus's treatments: the populated ones if the preview could be computed, otherwise every implied one.

Column	Meaning
Treatment	The treatment label (levels of the varying factors joined with /). Not editable.

Display name	What the treatment is called in Publication Figures. Leave it blank, or equal to the label, to use the label.
Colour	A swatch. Click it to pick the treatment's curve colour (the dialog has an alpha channel; alpha 0 means transparent). A swatch reading <i>auto</i> means no colour is set.

Display names and colours are stored in the Focus (`display_names:` and `colours:`) and used by the Plot Editor and the Publication Figures. Curve colours changed in the Plot Editor are written back into the Focus, so both places agree. Neither changes any number, so editing them never makes results Out of Date.

Import from DefinedPlots...

This reads the workbook's `DefinedPlots` sheet and lists every plot. A plot can be imported only if its treatments form a **rectangular product** of levels. That means the Focus built from the levels it uses would contain exactly the listed treatments among those the data holds, and no others. Plots that cannot be imported are greyed out, with the reason in the tooltip (for example "its treatments are not a rectangular product of levels (the product would also include ...)"). Tick the ones you want and press OK. Each becomes a new Focus named after the plot. Import is offered rather than automatic because a Focus is an independent analysis with its own report section, not just a figure. If the workbook has no `DefinedPlots` sheet, the window says so. See [Defined Plots](#).

Copy Focuses from...

Choose another member of the Project. Every one of its Focuses is checked against **this** member's data and exclusions before anything is added. A Focus is not copied if a Focus of the same name already exists here, if it names a factor or level this member's data file does not contain, or if one of its treatments that this member's data holds would have no individuals once this member's active Exclusion Group (and ChamberFlags) are applied. The skipped ones are listed with the reason ("Not copied — they would be blocked here or clash by name"). The copied Focuses appear in the list and are written only when you Save. Focuses are never inherited from `project.yaml`, so this is the way to share a design between members.

Save and Cancel

Save first takes the selected Focus exactly as the window shows it, then checks the whole set. Any problem is shown in a *Cannot save* message and the window stays open. Problems include a Focus with no factors, a factor ticked with no level ticked, a level listed twice, a `reference:` naming a level the factor does not keep, two Focuses with the same name or output folder, and a reserved name (`plots`, `statistics`, `data_output`). If any Focus still names a factor or level the data file does not contain, Save asks *Save blocked Focuses?*, listing them; Cancel keeps the window open so you can drop them. If all is well it:

1. moves the results of every renamed Focus to its new folder,
2. writes the complete `focuses:` block to `survival_config.yaml` (removing an old pre-Focus `factors:` block if there was one), and
3. makes the Focus selected in the list the Active Focus.

The log records `[focus] <experiment>: saved N Focus(es) and any renames`.

Cancel discards your edits in the window.

Pitfalls

- Changing a Focus's levels, level order or Reference Level after analysing it makes its

saved results [Out of Date](#).

- Deleting a Focus does not delete its results. Clear them from the Focus card in the Hub (**Delete...**).

See also

- [What a Focus is](#)
- [Reference Levels](#)
- [Focus Shape — what a Focus is offered](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [The Experiment panel](#)
- [Defined Plots](#)

Reference Levels

Each varying factor of a [Focus](#) has a **Reference Level**: the level the factorial models compare the other levels against. It sets the sign of every coefficient, including the interaction terms, in the [Cox factorial model](#) and the [RMST factorial model](#). The report's text uses it as the baseline too. It is stored separately from the level *order*, so you can list densities as 20x, 40x in every legend and still make 40x the control the model compares against.

Why it is separate from level order

Level order is presentation: it controls legend order, the panels of the faceted KM and the cells of the interaction plot. The Reference Level is statistics. If the first level listed were always the baseline, re-ordering a legend would silently flip every hazard ratio in the model. Keeping the two separate means a presentation choice never changes a result.

Setting it

In the Focus window: each factor box has a **Reference:** drop-down listing the ticked levels. It defaults to the first ticked level and is enabled once the factor varies (two or more levels ticked). Saving writes every varying factor's reference explicitly, the default included. See [The Focus window](#).

In `survival_config.yaml`: add a `reference:` mapping to the Focus, naming a level for each factor you want to set:

```
focuses:
  Crowding20v40:
    factors:
      Genotype: [wCS, mDilp235bx]
      Density: ["20x", "40x"]
    reference:
      Density: "40x"           # Genotype not named -> wCS (first listed)
```

The rules:

- With no `reference:` entry, a factor's Reference Level is the **first level listed** in `factors:`.
- The reference must be one of the levels the Focus keeps for that factor, and the factor must be named in the Focus. Otherwise validation reports it ("reference level '...' is not one of ...'s levels", "reference names factor '...', which the Focus does not name") and the Focus window refuses to save.
- The Focus window writes `reference:` for **every** varying factor when it saves, even when the reference is the first level. A reference left out of a hand-written file means

"the first level listed", and so moves if the levels are reordered; a written one does not.

What it affects

Output	Effect
Cox factorial model	Each factor is dummy-coded with the Reference Level as the dropped baseline. Every main-effect hazard ratio is "this level vs the reference", and each interaction term is relative to the reference cell.
RMST factorial model	The same coding: differences in restricted mean survival time relative to the Reference Levels.
Report	The "Focus definition" table has a <i>Reference level</i> column. The coefficient tables are captioned "Relative to the Reference Levels: Density = 40x, ...".
Hub and Focus window	The Focus description reads <code>Density: 20x, 40x (reference 40x)</code> .
Run Summary	The Reference Levels are recorded as part of the Focus's analytic definition.

The Reference Level does **not** change Kaplan-Meier curves, life tables, the log-rank and Gehan-Wilcoxon tests, the pairwise hazard ratios, the parametric AFT fits or any figure. Those depend on the treatments and the display order only.

Discovered order protects the default

Levels are discovered in **first-appearance order in the Design sheet**, never alphabetically. You usually enter the control first, so the default reference is usually the control. Alphabetical order would, for example, make `mDilp235bx` the reference over `wCS`. When the Unfiltered Focus (or a migrated Interaction Focus) is first written into the config, that order is frozen, so re-sorting the Design sheet later cannot move a Reference Level.

Changing it after analysing

The Reference Level is part of the Focus's analytic definition. Changing it makes the Focus's saved results **Out of Date** ("a Reference Level changed"). The comparison is of the baseline the model actually uses, so writing down a reference that was already the default (as the Focus window's Save does) is not a change. The Hub and the Project Report then stop presenting those results until the Focus is re-run. See [Analysed, Out of Date, Blocked and Orphaned results](#).

Pitfalls

- In a hand-written Focus with no `reference:` for a factor, the reference **follows the order**: moving another level to the top of the YAML list makes that level the reference. Write `reference:` (or save the Focus once in the Focus window, which writes it for you) to pin the baseline whatever the order.
- A one-level factor (a filter) and a pooled factor have no Reference Level. Only varying factors do.
- When reading a coefficient table, check its caption for the baseline before interpreting the sign of a hazard ratio.

See also

- [What a Focus is](#)
- [The Focus window](#)

- [The Cox factorial model](#)
- [The RMST factorial model](#)
- [Interaction analyses \(Factorial Battery\)](#)

Focus Shape — what a Focus is offered

Which analyses and figures you get depends on the **Focus**, not on the experiment. A [Focus](#) that compares three genotypes gets log-rank tests but no interaction model. A Focus crossing genotype with diet also gets the faceted KM, the interaction plot and the Factorial Battery. A single-treatment Focus gets the survivorship battery only. The **Focus Shape** is what decides this: the factors the Focus varies and how many levels each has, which of the implied treatments actually hold individuals, and where the crossing of two varying factors has a hole.

Reading a shape

A shape is written as the level counts of the varying factors:

Shape	Meaning
single cell	nothing varies — one treatment
3	one varying factor with three levels
2×2	two varying factors with two levels each, all four cells populated
2×2 (3 of 4 cells)	the same, but one implied cell holds no individuals
2×3×2	three varying factors

Two questions: relevant, then computable

Each conditional analysis or figure declares a **Requirement**, and each Requirement is checked in two steps:

1. **Relevance** is decided by the Focus's *definition* alone (how many factors it varies and how many treatments its levels imply). An action that is not relevant is **not offered**: no checkbox in the Hub, no figure, no report section, no note in the report. It is not a question this slice asks.
2. **Computability** is decided by the *data*: which treatments hold individuals once the Exclusion Group is applied. A relevant action the data cannot support is **Not Applicable**. It is greyed out in the Hub with the reason, and the run records it in the log, the Run Summary and the report. See [Not Applicable and Left Out](#).

The three Requirements

Requirement	Offered (relevant) when	Computable when	Used by
Comparison	the levels imply 2 or more treatments	2 or more treatments hold individuals	log-rank pairwise and omnibus, Gehan-Wilcoxon, pairwise hazard ratios, the hazard-ratio forest, the log-log diagnostic
Factorial plot	the Focus varies 2 or more factors	2 or more treatments hold individuals	the faceted Kaplan-Meier and the lifespan interaction plot

Factorial model	the Focus varies 2 or more factors	as above, and every pair of varying factors is fully crossed	the interaction analyses: the Cox factorial model and the RMST factorial model
------------------------	------------------------------------	---	--

Everything else needs nothing and is always offered: life tables, summary statistics, median and mean survival, Kaplan-Meier curves (with and without the at-risk table), Nelson-Aalen, the smoothed hazard, mortality (qx), number at risk, the lifespan distribution and the parametric AFT fits.

The factorial figures need only two populated treatments, because a missing cell is just a missing curve or point. The factorial models need full crossing, because an interaction term cannot be estimated across an empty cell.

Crossing gaps

For the factorial models, the app checks every **pair** of varying factors. For each pair it looks for any combination of their levels with no individuals. Each such combination is a *crossing gap*, written like `Genotype=mut × Diet=DR`. One gap makes the models Not Applicable, with the gap named in the reason:

Focus 'GxD' has no individuals for `Genotype=mut × Diet=DR`, so the Factorial Battery's interaction terms cannot be estimated

The models fit main effects plus **pairwise** interactions, so with three varying factors a missing three-way cell is not a gap as long as every pair is still covered.

An *absent* cell (one the data never contained) and a cell *emptied by exclusions* both count as unpopulated for computability. They differ in another way: a cell that QC emptied **blocks** the whole Focus. See [Analysed](#), [Out of Date](#), [Blocked](#) and [Orphaned results](#).

Where you see the shape

- **Focus window** – the preview line `N = 409 · 4 treatment(s) · shape 2×2`, followed by `This Focus offers a comparison between treatments; a figure crossing factors; the Factorial Battery`. Any offered item the data cannot compute is marked (*not computable: ...*). A single-treatment Focus reads "offers the survivorship battery only — no comparison or factorial analysis applies to a single treatment".
- **Hub, Focus card** – the shape line under the Focus selector: the Focus description, `shape ...`, the same "offers ..." text, then any Not Applicable items from the last run and any Blocked or Out of Date notice.
- **Analyze and Plots panels** – one checkbox per offered item. A greyed box is relevant but not computable (hover for the reason). An italic line at the bottom names what is *not offered* for this Focus (hover for why).
- **Plots sub-tile** – the shape of the last run (`focus shape: 2×2`).
- **Report** – the Focus section states the shape, the treatments and any absent cells. The Run Summary records the shape.

Estimated before anything runs

The Hub and the Focus window know the shape before you analyse. The Hub reads the workbook's Design sheet (one row per chamber) and removes the chambers excluded by the active Exclusion Group and the workbook's chamber flags; it does not read the census data. The Focus window computes N and the shape from the loaded data. A run then measures the shape exactly from the data it loaded. For a CSV, which has no chambers, the estimate is simply the

cells the file contains.

The Headline Figure follows the shape

When the Focus can draw the faceted KM (two or more varying factors and two populated treatments), the faceted KM is the [Headline Figure](#) that leads the report. Otherwise the headline is the KM with at-risk table.

Scripts

A script step can name any action, whatever the Focus. When a step's Requirement is not admitted (not relevant, or not computable) for the Focus it runs under, the step is **Not Applicable**: it is logged and recorded, and the script carries on. Only an action name the app does not know at all is a hard error that stops the script from starting. See [Script action reference](#).

See also

- [What a Focus is](#)
- [Not Applicable and Left Out](#)
- [Interaction analyses \(Factorial Battery\)](#)
- [The Analyze panel and Run analysis](#)
- [The Plots panel and Generate plots](#)

Not Applicable and Left Out

An analysis or figure can be missing from a run for three different reasons. The app keeps them apart, because a missing table means something different in each case:

Word	Why it is missing	Recorded?
Not offered	The Focus never asks this question (an interaction model for a one-factor Focus).	No. It is simply not part of this Focus's run.
Not Applicable	The question is relevant, but the <i>data</i> cannot answer it (a crossing with an empty cell).	Yes: log, Run Summary, report.
Left Out	You chose to leave it out (its box is unticked on the Analyze or Plots panel).	Yes: log, Run Summary, report.

The rule behind this: a report must never look complete when it is not. A real gap is always stated with its reason. A question the slice never asked is not mentioned, because a "not run" note in every one-factor report would teach readers to skip the note that matters.

Not offered

Whether an item is offered depends only on the Focus's definition (see [Focus Shape](#)). Items that are not offered have no checkbox. Instead, one italic line at the bottom of the Analyze or Plots panel names them, for example *Not offered for Focus Females: Interaction analyses*. Hover over it for the reason (for example "Focus 'Females' varies only Genotype; the Factorial Battery needs 2 or more varying factors"). Nothing about them appears in the run log or the report.

Not Applicable

An offered item is Not Applicable when the populated treatments cannot support it. Common

causes:

- **Comparisons with one populated treatment** – "only 1 treatment(s) of Focus 'X' hold individuals; a comparison between treatments needs 2 or more". The four comparison analyses are recorded together as *Survival comparisons*.
- **Factorial models with a crossing gap** – "Focus 'X' has no individuals for Genotype=mut × Diet=DR, so the Factorial Battery's interaction terms cannot be estimated". This is recorded as *Factorial Battery*.
- **A figure the data cannot draw** – recorded under the figure's name with "could not be drawn (...)" or "the data cannot support it".
- **A Defined Plot that cannot be drawn in full** – for example "not in Focus 'X': Male/40x", or "Focus 'X' pools over Sex". A Defined Plot is drawn with all the curves its author listed or not at all. See [Defined Plots](#).
- **A script step the Focus does not admit** – for example a `cox_interaction` step run under a one-factor Focus.

In the Hub, before running

The Hub estimates the shape from the Design sheet and the active exclusions. An offered item that is estimated not to be computable has its checkbox **greyed out**, and its tooltip ends with *Not applicable to Focus X: <reason>*. The Focus card's shape line marks the same items (*not computable: ...*).

Where a run records it

- **Run log** – `Not applicable – Factorial Battery: <reason>`
- **Run Summary** – an entry `{"action": ..., "reason": ...}` in the `not_applicable` list of `analysis/<focus>/run_summary_<focus>.json`. See [The Run Summary](#).
- **Experiment report** – a status line on the cover ("N action(s) not applicable to this Focus — listed in the Focus section"), a *Not applicable to this Focus* table (Not applicable / Why) in the Focus section, and, for the battery, "**Factorial Battery: not run** — *reason*" in the Factorial analysis section.
- **Hub** – the Focus card's shape line adds "not applicable last run: ...".
- **Project Report** – the Focus Inventory's *Not applicable* column.

Not Applicable is never an error. The rest of the run carries on, and in a script the next step runs.

Left Out

Each Analyze and Plots checkbox is saved to the experiment's configuration, not kept as Hub state. Unticking a box adds its id to the `omit:` block of `survival_config.yaml`:

```
omit:
  analyses: [parametric_aft]
  plots: [mortality, smoothed_hazard]
```

The same items are then left out of **every** run of this experiment: **Run analysis** in the Hub, a `run_analysis` step in an Experiment Script, and a Batch Run. Each produces the same report. Re-ticking the box removes the id again. The block lists what is left *out*, so an untouched config runs everything, and an analysis added in a later version is included until someone unticks it. See [The omit: section](#).

Rules:

- Only items the Focus is **offered** can be left out of its run. An unticked *Interaction analyses* box means nothing to a one-factor Focus and is not recorded there.

- A figure that draws from a left-out analysis is left out with it. The **hazard-ratio forest** draws from the **pairwise hazard ratios**: while that analysis is unticked, the forest's box is greyed out ("Draws from Pairwise hazard ratios — tick it on the Analyze panel to draw this figure"), and the run records the forest as left out because it "draws from Pairwise hazard ratios, which is unticked".
- Files an earlier run wrote for an item that is now left out (its figure, its statistics table) are **deleted** by the next run, so the folder never shows an old result as current.

Where a run records it

- **Run log** – a line such as *Left out — Mortality (qx): unticked in the Hub (omit: in survival_config.yaml)*
- **Run Summary** – an entry {"id": ..., "item": ..., "reason": ...} in the left_out list.
- **Experiment report** – a cover status line ("N analysis or figure(s) left out of this run by choice"), a *Left out of this run* table in the Focus section, and, for the interaction analyses, "**Interaction analyses: left out of this run**" in the Factorial analysis section.

Generate plots on the Plots panel writes its Not Applicable and Left Out lines to the log only. The Run Summary and the report keep what the last **Run analysis** recorded.

Left Out vs Not Applicable

Left Out is a choice, and you can reverse it by ticking the box and re-running. Not Applicable is a property of the data, and only different data, a different Exclusion Group or a different Focus changes it. That is why the report lists them in separate tables.

See also

- [Focus Shape — what a Focus is offered](#)
- [The omit: section — leaving analyses and figures out](#)
- [The Analyze panel and Run analysis](#)
- [The Plots panel and Generate plots](#)
- [The Run Summary](#)
- [The experiment report](#)

Analysed, Out of Date, Blocked and Orphaned results

Every [Focus](#) has a state that the app reads from disk, from the Focus's saved Run Summary, without re-analysing anything. The state tells you whether the results you are looking at describe the Focus as it is now declared. There are four states for a Focus, plus two kinds of leftover result folder.

State	Meaning	What to do
analysed	Saved results exist and match what a run would use now: the Focus's definition, the exclusions, the data file and the settings below.	Nothing.
not analysed	No Run Summary for this Focus yet.	Run analysis.
out of date	Saved results exist but were produced from something that has since changed (see below).	Re-run the Focus.

blocked	The Focus cannot run as declared.	Fix the Focus or the exclusions (below).
----------------	-----------------------------------	--

If a Focus is both blocked and analysed, it is shown as **blocked**.

Where the state is shown

- **Focus card** (Experiment panel) – each entry in the Focus selector reads <name> – <state>. The shape line under it appends **BLOCKED** – <reasons> or out of date – <reasons> for the Active Focus.
- **Analyze sub-tile** – the Active Focus's state, with N once it has been analysed.
- **Project panel, members table** – the *Analysed* column counts Focuses that are analysed and current (2/3), or shows the date of the latest run when all are. It reads *re-run needed* when any Focus is out of date, and *1/3 · 1 blocked* when any is blocked. Rows with a blocked or out-of-date Focus are coloured, and the tooltip lists each one with its reasons.
- **Project Report** – the Focus Inventory's *State* column, and each Focus's section (see below).
- **Batch Preflight** – lists every Blocked Focus before a Batch Run. See [The Batch preflight](#).

Out of Date

When a run finishes it writes into the Run Summary what it was run from: the Focus's **analytic definition** (factors, ordered levels and Reference Levels), the Exclusion Group's name and the chambers actually removed, a fingerprint (sha256) of the data file, the censoring policy, and what the run left out. Afterwards the app compares that record with what a run would use now. A difference gives one of these reasons:

- "the Focus's factors or levels changed" – a level added or removed, a factor added or removed, or the **level order** changed
- "a Reference Level changed" – compared as the baseline the model actually uses, so writing down the default reference (as the Focus window does on Save) is not a change
- "analysed under exclusion group 'A', config now asks for 'B'"
- "the excluded chambers changed — now also excludes chamber(s) ...; no longer excludes ..." – the active group's rows were edited (or the workbook's ChamberFlags changed). Only chambers inside this Focus count: excluding a chamber of another slice changes none of this Focus's numbers.
- "the data file's contents changed since the analysis" – the workbook or CSV was edited or replaced
- "analysed with assumed censoring on, config now has it off" (or the reverse) – workbooks only, since the policy changes nothing for CSV data
- "the analyses selection changed — now leaves out ..." / "the plots selection changed — now includes ..." – boxes ticked or unticked on the Analyze or Plots panels (the *omit*: block)
- "the saved results predate Focuses and record no definition"

Each of the newer checks applies only when the Run Summary recorded the value. Results from an older version of the app are not marked Out of Date merely for lacking it.

Display names and colours are **not** compared. They change no number.

An Out of Date Focus's section in the Project Report says its results "are out of date — they

predate a change and are not shown" with the reasons, and presents **none** of the old numbers. The report never analyses on your behalf, and it never presents results that no longer describe the Focus. Re-run the Focus (Run analysis with it active, or a script or Batch Run) and rebuild the report.

What Out of Date does not detect

These changes do **not** mark results Out of Date, so re-run yourself after any of them:

- changing `input`: settings (time or event column, layout) or `global`: settings other than `assume_censored`, such as `time_label` or `min_n_per_chamber`
- changing a Project Default that reaches the member only through settings in the previous point
- an upgrade of the app that changes how something is computed

Blocked

A **Blocked Focus** cannot run as declared. Its member is not blocked: the member's other Focuses still run, and a Batch Run continues. Run analysis on a blocked Focus stops before writing anything, with a message like:

```
Focus 'Crowding' is blocked:  
- Focus 'Crowding' names level 'mDilp' of Genotype, which the data file does no
```

There are two reasons a Focus can be blocked:

Stale – the Focus names a factor or level the data file no longer contains, typically because the Design sheet was corrected after the Focus was written. The message suggests the closest name in the data if there is one ("rename it to '...?'"), otherwise "edit the Focus". To fix it:

- In the **Batch preflight**, select the Blocked Focus row and press **Fix selected...** (enabled only when every stale name has a close match). It rewrites every stale name to its close match after asking you to confirm ("Rewrite Focus ... as: ... Its saved results will then be out of date until it is re-run").
- In the **Focus window**, drop or replace the stale names. A stale *level* is listed in red, marked "(not in the data file)", so untick it and tick the right one. A stale *factor* gets a box of its own above the others, titled "*factor* — not in the data file", so untick the box to drop it. Saving with stale names still in place asks you to confirm first.

Data holding levels the Focus does not name is normal, since selecting a subset is what a Focus is for. Only the reverse blocks.

Empty – a treatment the data file *does* contain has no individuals once the active Exclusion Group and the workbook's chamber flags are applied ("treatment 'Male/40x' is in the data file but has no individuals once exclusions are applied — change the Exclusion Group or drop that level from the Focus"). QC has removed a whole cell from under the Focus. Either change the Exclusion Group or untick that level. A cell the data *never* had is merely absent, and absence does not block. A Focus that selects no individuals at all is blocked with "Focus '...' selects no individuals".

The Hub and the preflight estimate blocking from the Design sheet and the excluded chambers. A run confirms it from the loaded data.

A blocked Focus's section in the Project Report says it "is blocked — it cannot run as declared, so nothing was analysed for it", lists the reasons, and notes that the member's other Focuses are unaffected.

Orphaned Results

An **orphaned result** is a folder under `analysis/` that no declared Focus names. It appears

when a Focus was deleted, or renamed by hand in the YAML. (Renaming in the Focus window moves the results along with the name, so it cannot orphan anything.) Orphaned results are never bound into a report.

When an experiment has orphans, the Focus card shows `Orphaned results` (no Focus names them): `<folders>` with two buttons:

- **Adopt...** – pick an orphaned folder, then a declared Focus that has no results of its own. The folder is moved to that Focus's place, the name suffix of every file in it is renamed, and the Run Summary is updated to the new name. The adopted results are then judged like any others, so they show as **out of date** if their recorded definition differs from the Focus that adopted them. If every Focus already has results, the Hub asks you to create or rename one first.
- **Delete...** – pick a folder and confirm ("Delete ... from analysis/? This cannot be undone.").

Pre-Focus results

Results written before Focuses existed sit directly in `analysis/` (`run_summary.json`, `plots/`, `statistics/`, `data_output/`, `report.md`, `*_report.pdf`). They record no Focus and no definition, so they are **never adopted**. Adopting them would vouch for a slice nobody can check. The Focus card notes "Pre-Focus results in `analysis/` — never adopted and never replaced by a re-run; Delete... removes them." A re-run writes new results under `analysis/` `<focus>/` and does not touch the old files. Remove them with **Delete...** → *pre-Focus results*.

See also

- [What a Focus is](#)
- [The Focus window](#)
- [The Batch preflight](#)
- [The Run Summary](#)
- [The Project Report](#)
- [Exclusion Groups](#)

Defined Plots

A **Defined Plot** is a named group of treatments that you write into the DLife workbook's `DefinedPlots` sheet while designing the experiment. For example, "Diet series" might list the four diets fed to females. Each analysis run draws every Defined Plot that is about the Focus being analysed, as a Kaplan-Meier figure with exactly the curves you listed and titled with the plot's name. A Defined Plot is a **figure**, not an analysis. It adds no statistics, though it can be turned into a Focus (see below).

The DefinedPlots sheet

Defined Plots are read only from an Excel (`.xlsx`) DLife workbook with a sheet named exactly `DefinedPlots`. CSV inputs have none.

- **One column per plot.**
- **Row 1:** the plot's name. It is used as the figure title, the report heading and in the file name.
- **Rows 2–5:** ignored.
- **Row 6 downward:** one treatment per cell. Only the text **before the first comma** is used, so `Female/20x, high-dose group` reads as `Female/20x`. The first empty

cell ends the list for that column.

- A column with no treatments from row 6 is ignored. If the sheet cannot be read, there are simply no Defined Plots.

Each treatment must be written as a **full treatment of the data file**: one level of **every** factor in the file, in the file's factor order, joined by /, e.g. `Female/20x` for a file whose factors are Sex then Diet. Spaces around each part are ignored.

Which Focus draws it

Defined Plots are matched against the Focus of each run:

- **Drawn** when the Focus pools over no factor, and every listed treatment that the data file contains falls inside the Focus. Every listed treatment must also hold individuals in this Focus after exclusions. The curves are relabelled with the Focus's own treatment names and drawn in the Focus's order.
- **Not drawn, and not mentioned**, when the plot is about a different slice. A plot comparing the sexes is not about a females-only Focus, so it does not appear there or in its report. The same goes for any Focus that pools over a factor.
- **Not Applicable** when the plot is about this Focus but a listed treatment is missing from it. The treatment may be absent from the data, emptied by the active Exclusion Group, or a typo the file never had. The run records `Defined Plot '<name>'` with the reason, e.g. "not in Focus 'Females': Female/40x", in the log, the Run Summary and the report's "Not applicable to this Focus" table. A Defined Plot is **all or nothing**: it is never drawn with fewer curves than you listed.

Every factor in a file must be named in a label. A plot that cannot be about **any** Focus is a mistake in the sheet, so rather than vanishing it is recorded as **Not Applicable under every Focus**, with the reason:

- a label with the wrong number of /-separated parts (for example, only the diet in a Sex × Diet file): "Female — not a full treatment of this data file (one level of each of Sex/Diet, joined by '/')";
- labels none of which is a treatment the file contains (every one a typo): "none of ... is a treatment of this data file".

What is drawn

The same figure as [Kaplan-Meier curves](#), restricted to the listed treatments:

- KM step curves with 95% confidence bands and "|" censor ticks;
- the plot's name as the title; y axis "Survival Probability"; x axis labelled with your configured time label (for example "Age (days)");
- the curves in the Focus's order, with its display names and colours. A treatment without a Focus colour takes the fixed cycle by its position among **all** the Focus's treatments, so it keeps the same colour as in the Focus's main KM figure.

Where it appears

- File: `analysis/<focus>/plots/defined_<name>_<focus>.png` (150 dpi). In `<name>`, any run of characters other than letters, digits, `.`, `_` and `-` becomes `_`.
- The experiment report's figures section, after the Plot Set figures, titled `Defined Plot — <name>` with the caption "Treatments the experimenter listed in the workbook's DefinedPlots sheet."
- The Run Summary's `defined_plots` entry.

Defined Plots are drawn by **Run analysis**, by scripts and Batch runs that run the analysis, and by **Generate plots** on the Plots panel. They have no checkbox there, and the `omit:` block does not affect them. A full run deletes a `defined_*` PNG an earlier run wrote that this run did not draw (the plot was renamed or removed from the sheet, or is now Not Applicable), so the folder never shows a stale one.

Promoting a Defined Plot to a Focus

Experimenters often use Defined Plots to mark the separate experiments within one file. The [Focus window](#) offers **Import from DefinedPlots...**:

1. It reads the sheet and, for each plot, proposes a Focus with the plot's name that selects exactly the listed treatments.
2. A proposal is possible only when the listed treatments form a **rectangular product of levels**. For example, Female and Male $\times$ 1x and 20x, all four listed. If the product of the levels used would also include a treatment you did not list, the plot is shown as not importable with the reason ("its treatments are not a rectangular product of levels (the product would also include ...)"). The same happens for a level the data file does not contain.
3. You tick the proposals you want. They are added as new Focuses (renamed if the name is taken) and saved when you close the Focus window with OK.

Import is offered rather than automatic because a Focus is an **analysis**, with its own statistics, report section and output folder, whereas a Defined Plot is only a figure.

See also

- [What a Focus is](#)
- [The Focus window](#)
- [Kaplan-Meier curves](#)
- [Not Applicable and Left Out](#)
- [Input data formats](#)
- [The experiment report](#)

Quality control

The QC panel

The QC panel is where you decide which chambers an experiment's analyses leave out. Here you choose the experiment's **active Exclusion Group** (a named list of chambers to remove) and open the **Chamber QC viewer**, where those lists are made. QC comes first among the Experiment sub-tiles because you decide what to exclude before you analyse.

Opening it

Load an experiment by double-clicking its row in the Project panel's members table, or select a standalone Experiment Directory. Then click **QC** under the Experiment tile. Its tile shows a two-line summary:

- `group: <name>` — the active Exclusion Group, or none.
- `N chamber(s) excluded` — how many of this data file's chambers the current configuration removes: the active group plus the workbook's ChamberFlags. It describes what every Focus's next run will leave out, so it changes as soon as you set a group or

save one in the viewer. Chambers a group lists that the file does not have are not counted, and a CSV (which has no chambers) shows 0.

Controls

Active group is an editable drop-down listing every group in this experiment's `qc/remove_chambers.csv`, in the order they first appear in the file. The current active group is pre-selected.

- If the file has no groups yet, a note says so. Open the Chamber QC viewer, flag chambers, and use **Save Exclusions...** under a name, and the group will appear here — the list is re-read every time the viewer saves, and again when it closes.
- If the active group's name is not in the file, a warning says it "currently excludes no chambers". A misspelt name is not an error, but it removes nothing.

Set active group writes the chosen name into `survival_config.yaml` as:

```
exclusions:  
  group: default
```

The log confirms the change. From then on, every run of this experiment (Hub, script or Batch Run) drops that group's chambers and stamps the group's name on its report and Run Summary.

Clearing the box and clicking **Set active group** means *no group*. Normally that removes the `exclusions:` key from the experiment's config. When the Project's `project.yaml` sets a default group (`defaults: exclusions: group:`), removing the key would hand the member straight back to that group, so the panel instead writes an explicit empty group, which overrides the default:

```
exclusions:  
  group: ''
```

The log then says it is overriding the Project default.

Chamber QC viewer... opens the viewer on this experiment, starting on the Hub's Active Focus. See [The Chamber QC viewer](#).

What changes when you set a group

- **Nothing is re-analysed automatically.** Every Focus that was analysed under a different group becomes **Out of Date** ("analysed under exclusion group 'x', config now asks for 'y'"). Its report section stops presenting the old numbers until you run it again.
- **All Focuses share one group.** The group belongs to the experiment directory, not to a Focus: whether a chamber is valid data is a fact about the chamber.
- **A Focus can become Blocked.** If the new group removes every chamber of a treatment the Focus names, that Focus is Blocked (`treatment '...' ... has no individuals once exclusions are applied`) until you change the group or drop the level from the Focus.
- **The Focus shape line updates.** The Experiment panel's shape line and the Analyze/Plots boxes are estimated from the Design sheet minus the active group's chambers, so they show the effect before anything runs.

Pitfalls

- **Editing the YAML by hand.** Deleting a member's `exclusions:` key yourself makes it inherit the Project's default group, if there is one. Use `group: ''` to mean "no group" in a member of such a Project — what clearing the box writes for you.

- **CSV data has no chambers.** For CSV/TSV experiments a group can be set, but it removes nothing. Reports record the group with zero chambers removed.
- **Workbook flags always apply.** Chambers marked `Excluded = 1` on the workbook's ChamberFlags sheet are removed from every run, on top of the active group.

See also

- [Exclusion Groups](#)
- [The Chamber QC viewer](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [The Experiment panel](#)
- [The Analysis Hub](#)

Exclusion Groups

An **Exclusion Group** is a named set of chambers to remove from analysis: vials that were contaminated, mis-scored, dropped, or otherwise not valid data. Groups are stored per experiment in `qc/remove_chambers.csv`. One group at a time is **active**, and the active group is part of the experiment's configuration (`exclusions: {group: ...}` in `survival_config.yaml`). Every run removes its chambers and records the group's name. The same data and the same config always give the same result, and every report says what was removed.

Why named groups

You may want to compare results with and without a questionable set of chambers, or keep a strict and a lenient QC policy side by side. Each is a separate group in the same file. Switching the active group is a deliberate, recorded change of configuration, never a hidden setting. Results produced under a different group are flagged **Out of Date**.

The file: `qc/remove_chambers.csv`

A plain CSV with three columns:

```
group,chamber,note
default,3,low N - early deaths only
default,7,
strict,3,
strict,7,
strict,12,suspicious survival
```

Column	Meaning
group	The group's name. Rows with a blank group are ignored.
chamber	A chamber id, as on the workbook's Design sheet. A whole number is read as that number however it is written (12, 12.0, 12). Anything else is read as text.
note	Optional free text. The app keeps notes it finds but does not display them.

A chamber may appear in several groups. The file lives in the experiment's `qc/` folder. An older experiment may still have the file at the directory root. That copy is read while `qc/` has none, but nothing is ever written back to it: the first save copies all its groups into `qc/remove_chambers.csv`, which is used from then on. The old root file is left in place and can be deleted.

You can edit the file by hand, in any editor or spreadsheet (a "CSV UTF-8" save from a spreadsheet is read correctly). The usual way to write it is the Chamber QC viewer's **Save Exclusions...**, which replaces every row of the named group and leaves other groups untouched. Notes already in the file are kept for every chamber that stays in the group. Saving a group with no chambers flagged removes that group's rows. If the file cannot be read at all, the save is refused rather than rewriting the file without its other groups.

Making a group active

On the Hub's QC panel, pick the group in **Active group** and click **Set active group**. This writes:

```
exclusions:
  group: strict
```

In a Project, `defaults: exclusions: group: in project.yaml` gives every member a default active group name. Each member still reads its own `qc/remove_chambers.csv`. A member opts out with `group: ''`, which is what clearing the QC panel's group box writes in such a Project. See [The QC panel](#).

What a run does

1. It removes every chamber listed under the active group, plus every chamber the workbook's **ChamberFlags** sheet marks `Excluded = 1`.
2. It analyses what remains under the chosen Focus.
3. It records the result:
 - The report's cover line **Exclusion group** reads, for example, `strict - 3 chamber(s) removed`. Other forms are `strict - nothing listed`, `strict - no chambers removed (N listed, none present in this data)`, `none declared (N removed by the workbook)`, or `none`.
 - The report's **Data quality** section lists the excluded chamber ids and the group.
 - The Run Summary (`run_summary_<focus>.json`) stores `exclusion_group`, `excluded_chambers` (the ids removed from this data), `n_excluded` (how many), and `n_excluded_listed` (chambers listed).

The Hub's members table shows each member's active group, and the Project Report's Divergence Note says when members use different groups.

One group per experiment, shared by every Focus

The active group applies to the whole Experiment Directory. A chamber's validity is a fact about the chamber, not about which slice you are analysing. If groups were set per Focus, two sections of one report could disagree about whether the same vial is valid data. The Chamber QC viewer can *show* only the Active Focus's chambers, but the groups it saves belong to the directory.

Effects to expect

- **Out of Date.** Changing the active group makes every Focus analysed under the old one Out of Date until it is re-run. So does changing *which chambers* the active group lists, for every Focus whose chambers the change touches.
- **Blocked Focus.** If the group removes every chamber of a treatment a Focus names, that Focus is Blocked. The message names the treatment and suggests changing the group or dropping the level from the Focus. A cell the data never contained is simply absent and does not block.
- **Shape changes.** Removing chambers can empty a cell of a two-factor Focus. The interaction models then become Not Applicable, because the crossing has a hole.

In scripts

An Experiment Script's `run_analysis` step uses the active group from the config, like the Hub. The separate `apply_exclusions` step (parameter `group`, default `default`) adds a named group's chambers on top for the rest of that script; the runs it changes name both groups (`default + extra`) and show as Out of Date against the config. See [Script action reference](#).

Pitfalls

- **CSV data has no chambers.** Exclusion Groups remove nothing from CSV/TSV experiments. Reports record the group with zero chambers removed rather than claiming exclusions that could not happen.
- **A name with no rows excludes nothing.** The QC panel warns when the active group is not in the file.
- **Text ids must match exactly.** Numeric ids match however they are written (12.0 is chamber 12), but text ids are compared as written, case included: A12 and a12 are different chambers.
- **Saving is not activating.** Saving a group in the viewer does not make it active. Set it on the QC panel.

See also

- [The QC panel](#)
- [The Chamber QC viewer](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Input data formats](#)
- [The Run Summary](#)

The Chamber QC viewer

The Chamber QC viewer shows each chamber's survival curve, one tab per treatment, so you can spot chambers that behave unlike their siblings: a vial where everyone died in the first days, one that never lost anyone, a curve that falls off a cliff. You can click curves to mark chambers as excluded, then save the selection as a named **Exclusion Group**.

It works only for DLife workbooks, because individual-level CSV data has no chambers.

Opening it

On the Hub, load an experiment, open **QC** under the Experiment tile, and click **Chamber QC viewer....** The viewer opens on that experiment's directory, together with the Hub's Active Focus. It can also be pointed at another directory with **Pick project...** in its side panel. That button takes any Experiment Directory, not a Project.

The window

Side panel (left)

- **Project** — the directory being viewed, and **Pick project...** to choose another.
- **Only Focus name** — shown when the viewer was opened from the Hub. When ticked (the default), only individuals in the Active Focus are shown, and the tabs are that Focus's treatments. Untick it to see every chamber in the file, labelled by the full combination of all factors. This only changes what you see. Saved groups always apply to the whole directory, every Focus alike.

- **Active exclusion group** — an editable list of the groups in `qc/remove_chambers.csv`, or `default` if there are none yet. The viewer opens on the experiment's **active** group (the one `exclusions: group: names`, which every run applies), listed even if it has no rows yet. Choosing another group from the list loads its chambers as the current selection. If you have clicked chambers since the last save, the viewer first asks whether to **Save** them to the group you are leaving, **Discard** them, or **Cancel** and stay. Typing a new name does not change the selection: it only names the group that **Save Exclusions...** will offer.
- **Excluded chambers** — the current selection, as `Chamber <id>`. Double-click an entry to un-exclude it.
- **Clear all** — empty the selection.
- An amber note lists the chambers with fewer individuals than `global.min_n_per_chamber` (see below).

Tabs (right) — one per treatment, in display order: the Focus's level order when **Only Focus** is ticked, otherwise the order the data file lists the levels in (the order every Focus starts from). Never alphabetical. Each tab draws every chamber of that treatment as a Kaplan-Meier step curve over the same axes (survival probability against time):

- An included chamber is a thin, translucent **blue solid** line.
- An excluded chamber is a **red dashed** line.
- A chamber with fewer individuals than `global.min_n_per_chamber` (and not excluded) is an **amber solid** line. It is flagged for your decision, never excluded automatically. The tab's tooltip lists that treatment's small chambers with their `N. 0` turns the flag off.
- **Hover** over a curve to see `Chamber <id> - <n> events, last t=<time>, with N=<n>, below the minimum of <min> per chamber added for a small chamber`.
- **Click** a curve to toggle that chamber between included and excluded. The change shows in every tab and in the side list.
- The matplotlib toolbar above each plot zooms and pans. This helps when many curves overlap.

Save Exclusions... (top bar) asks for a group name, pre-filled with the name in the drop-down. It writes the current selection as that group's complete list in `qc/remove_chambers.csv`, replacing any rows that group already had and leaving other groups untouched. A chamber that stays in the group keeps the `note` it already had in the file. A message confirms how many chambers were saved.

Closing the viewer with unsaved clicks asks the same Save / Discard / Cancel question as switching groups.

A typical session

1. Open the viewer from the QC panel.
2. Go through the treatment tabs. Hover over outlying curves to identify them, and click each one you judge invalid.
3. Click **Save Exclusions...** and name the group (e.g. `default` or `strict`).
4. Back on the Hub's QC panel, choose that group and click **Set active group**. Saving alone does not make a group active.

5. Run the analysis again. Focuses analysed under the previous group show as Out of Date until you do.

What the curves are

Each curve is a Kaplan-Meier estimate computed from that chamber's individuals alone, read exactly as an analysis reads them: the experiment's own data file, its `input:` options and its censoring policy (`global.assume_censored`, with the Project's defaults applied for a member). The one difference is that the viewer loads **every** chamber. It never pre-removes the active group's chambers or the workbook's ChamberFlags chambers, so that you can see all of them and decide.

The time axis carries the experiment's time label (`global.time_label`, or Age (`<time_unit>`)).

Pitfalls

- **Saving is not activating.** Saving a group writes it to the file; only **Set active group** on the QC panel makes runs apply it. The exception is saving under the group that is already active: runs apply the new list from then on, and Focuses analysed with the old list show as Out of Date.
- **Notes are kept, not shown.** The viewer has no notes column. Notes you type into `qc/remove_chambers.csv` survive a save for every chamber that stays in the group; a chamber you un-exclude loses its row, note included.
- **CSV experiments.** The viewer says the data has no chamber-level information and shows nothing.
- **Several data files.** The viewer never guesses. For an experiment with several candidate files and no `data_file:` key it shows the same message as a run ("Name one with `data_file:...`"). For a plain folder with no `survival_config.yaml` it asks which file to show.

See also

- [Exclusion Groups](#)
- [The QC panel](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [What a Focus is](#)
- [Censoring policy](#)

Statistical analyses

The Analyze panel and Run analysis

The Analyze panel is where you choose which statistical analyses a run includes and then start the run. It holds one checkbox per analysis in the experiment's **Analysis Set** that the **Active Focus** is offered, and below them a single **Run analysis** button. Run analysis computes everything under the Active Focus and writes the tables, figures, report and Run Summary to `analysis/<focus>/`.

In the app

1. Load an experiment. For a Member Experiment, double-click its row in the Project panel's members table; this opens the Experiment panel.

2. On the Experiment panel, choose (or create) the **Active Focus**. The shape line under the selector tells you, before anything runs, which analyses this Focus is offered and whether any of them cannot be computed from its data.
3. Open the **Analyze** sub-tile. Tick the analyses you want and untick the ones you don't.
4. Click **Run analysis**. Progress and results go to the output log; when the run finishes the log reports where it wrote the results.

Each checkbox has a ? button that opens that analysis's help page.

The checkboxes

Checkbox	What it runs	Offered when
Log-rank pairwise	Mantel-Cox log-rank test for every pair of treatments	the Focus implies 2 or more treatments
Log-rank omnibus	One K-sample log-rank test across all treatments	the Focus implies 2 or more treatments
Gehan-Wilcoxon pairwise	Gehan-weighted log-rank test for every pair	the Focus implies 2 or more treatments
Pairwise hazard ratios	A hazard ratio with 95% CI for every pair (also feeds the hazard-ratio forest)	the Focus implies 2 or more treatments
Parametric AFT models	Weibull, log-normal and log-logistic fits per treatment	always
Interaction analyses	The Factorial Battery's models: the Cox factorial model and the RMST factorial model together	the Focus varies 2 or more factors

Analyses that the Focus is not offered get no checkbox. A quiet italic line under the button names them ("Not offered for Focus ..."); hover over it to see why. For example, a one-factor Focus is never offered Interaction analyses, and a single-treatment Focus is offered no comparisons.

A checkbox that is offered but **greyed out** means the analysis is relevant but the Focus's data cannot support it. Its tooltip gives the reason, for example an empty cell in the crossing of two factors. See [Not Applicable and Left Out](#).

What is always computed

The survivorship core is not in the Analysis Set and has no checkbox, because every figure and report section depends on it. Every run computes:

- the life tables and Kaplan-Meier estimates ([Life tables](#)),
- the per-treatment summary counts, median survival, mean survival (RMST), the lifespan statistics and the survival quantiles ([Summary statistics](#)).

Ticks are saved in the config

A tick is not just a setting on the screen. Unticking a box writes the analysis id to the `omit:` block of the experiment's `survival_config.yaml`, and ticking it removes the id again:

```
omit:
  analyses:
    - parametric_aft
    - gehan_wilcoxon
```

The ids are `logrank_pairwise`, `logrank_omnibus`, `gehan_wilcoxon`, `hazard_ratios`, `parametric_aft` and `interaction`. Because the choice is stored in the config, every run of this experiment leaves the same analyses out: Run analysis in the Hub, an Experiment Script's `run_analysis` step, and a Batch Run alike. See [The omit: section](#).

An unticked analysis is **Left Out**. It is not computed, and the run records it in the log, in the Run Summary's `left_out` list, and in the report's Focus section under "Left out of this run". Left Out is your choice; Not Applicable is a property of the data.

Unticking **Pairwise hazard ratios** also leaves out the hazard-ratio forest figure, because the forest is drawn from those hazard ratios. On the Plots panel its checkbox greys out and its tooltip says why.

What Run analysis does

Run analysis runs the whole pipeline once, under the Active Focus:

1. Loads the data file with the experiment's censoring policy ([Censoring policy](#)) and active Exclusion Group.
2. Applies the Focus: keeps its rows and labels treatments by its varying factors. A Focus that names factors or levels the file lacks, or whose cells the exclusions emptied, is **Blocked**, and nothing is written.
3. Computes the survivorship core, then every ticked analysis the Focus admits.
4. Draws the ticked figures of the Plot Set (the same figures Generate plots draws).
5. Writes the CSV tables, the figures, the experiment report (PDF and Markdown) and the Run Summary.

If one analysis is relevant but cannot be computed, it is recorded as Not Applicable and the run continues. A parametric fit that fails to converge, or a factorial model that fails to fit, does not stop the run either; the report says which and why.

Run analysis does not open figure tabs. To view figures in the Hub, use **Generate plots** on the Plots panel, or open the report.

Where results are written

Everything goes into the Focus's own folder, and every file name ends with the Focus's name (`<focus>`), so analysing a second Focus never overwrites the first:

Location	Contents
<code>analysis/<focus>/data_output/</code>	<code>lifetables_<focus>.csv</code> , <code>individual_data_<focus>.csv</code> , <code>summary_<focus>.csv</code> , <code>median_survival_<focus>.csv</code> , <code>mean_survival_<focus>.csv</code>
<code>analysis/<focus>/statistics/</code>	<code>logrank_pairwise_<focus>.csv</code> , <code>gehan_wilcoxon_pairwise_<focus>.csv</code> , <code>hazard_ratios_<focus>.csv</code> , <code>survival_quantiles_<focus>.csv</code> , <code>lifespan_treatment_stats_<focus>.csv</code> , <code>lifespan_factor_stats_<focus>.csv</code> , <code>factorial_01_coefficients_<focus>.csv</code> , <code>factorial_01_ph_test_<focus>.csv</code> , <code>factorial_02_coefficients_<focus>.csv</code> ,

	parametric_aic_<focus>.csv
analysis/<focus>/plots/	the Plot Set figures
analysis/<focus>/	report_<focus>.md, <experiment>_report_<focus>.pdf, run_summary_<focus>.json

A file appears only when its analysis ran and produced rows. When a later run leaves an analysis out, the pipeline deletes the stale comparison, parametric and factorial tables (and figures) an earlier run of the same Focus wrote, so the folder never shows an out-of-date table as if it were current.

The omnibus log-rank result has no CSV of its own: it is stored in the Run Summary and shown in the report. The parametric AFT comparison is in `parametric_aic_<focus>.csv` and also in the Run Summary.

Pitfalls

- **Check the Active Focus first.** Run analysis always runs under the Active Focus shown on the Experiment panel.
- **One run at a time.** If a task is already running, the Hub refuses to start another.
- **Changing the Focus afterwards** (its levels, filters or Reference Levels) makes the saved results **Out of Date**; re-run to refresh them. See [Analysed, Out of Date, Blocked and Orphaned results](#).

See also

- [Not Applicable and Left Out](#)
- [The omit: section — leaving analyses and figures out](#)
- [Focus Shape — what a Focus is offered](#)
- [Where results are written](#)
- [The experiment report](#)
- [Reading p-values and multiple comparisons](#)

Life tables and the Kaplan-Meier estimator

Every run starts by building a life table for each treatment in the Focus. The life table holds the Kaplan-Meier (KM) survivorship estimate with its confidence band, the interval mortality and hazard, and the Nelson-Aalen cumulative hazard. Almost everything else in the app is computed from it: the KM, hazard, mortality, cumulative-hazard and number-at-risk figures, median survival, mean survival (RMST) and the survival quantiles.

In the app

There is nothing to tick. The life tables are part of the survivorship core, so **Run analysis** on the Analyze panel always computes them, and **Generate plots** recomputes them for the figures. They are computed separately for each treatment of the Active Focus, after the Exclusion Group has removed its chambers. All the chambers of a treatment are pooled.

How it works

The data are first expanded to one row per individual, each with a time and an event flag: 1 means an observed death, 0 means right-censored. See [Censoring policy](#) for how censored individuals arise. The calculation is the app's own code; it does not call a survival library, though its survival estimate and band agree with lifelines'. Treatments appear in the Focus's display

order.

For each treatment, the table has **one row for each distinct time at which any individual of that treatment died or was censored**. At each such time t :

- n_i is the number at risk: everyone still alive and uncensored just before t .
- d_i is the number of deaths at t , and c_i is the number censored at t .
- Individuals censored at t are counted as at risk at t and removed afterwards, so $n_{i+1} = n_i - d_i - c_i$.

The Kaplan-Meier estimate is the running product of the interval survival probabilities:

$$q_i = d_i / n_i \quad \text{probability of dying at } t_i$$

$$p_i = 1 - q_i \quad \text{probability of surviving } t_i$$

$$\hat{S}(t) = \prod p_i \quad \text{over all } t_i \leq t$$

Confidence band. The variance uses Greenwood's formula, and the 95% band is the **log-log** ("exponential Greenwood") interval, built on the $\log(-\log \hat{S})$ scale and transformed back:

$$G(t) = \sum d_i / (n_i (n_i - d_i)) \quad \text{Greenwood sum, over } t_i \leq t$$

$$SE(\hat{S}(t)) = \hat{S}(t) \cdot \sqrt{G(t)}$$

$$v = \log \hat{S}(t)$$

$$95\% \text{ CI} = \exp(-\exp(\log(-v) \pm 1.96 \cdot \sqrt{G(t) / v}))$$

The interval always lies between 0 and 1 without clipping, and it is asymmetric where $\hat{S}$ is close to either end, which is where the tail is read. It is the interval R's `survfit` uses with `conf.type = "log-log"` and the one lifelines' `KaplanMeierFitter` reports, and the app's values match lifelines exactly. Where $\hat{S} = 1$ (before the first death) the band is 1, and where $\hat{S} = 0$ it is 0. A time at which everyone at risk dies ($n_i = d_i$) adds nothing to the Greenwood sum.

Interval hazard. The hazard in each interval uses the actuarial approximation, where Δt is the gap to the next time in the table (for the last row, the gap from the previous time):

$$h_i = 2 q_i / ((1 + p_i) \cdot \Delta t)$$

Nelson-Aalen cumulative hazard, with a normal-approximation 95% interval (the lower bound is clipped at 0):

$$H(t) = \sum d_i / n_i$$

$$\text{Var } H = \sum d_i / n_i^2$$

$$95\% \text{ CI} = H \pm 1.96 \cdot \sqrt{\text{Var } H}$$

Reading the results

The life table is saved as `analysis/<focus>/data_output/lifetables_<focus>.csv`, with one block of rows per treatment:

Column	Meaning
treatment	Treatment label (levels of the Focus's varying factors joined with /)
time	A distinct death or censoring time, in the data's time unit
n_at_risk	Number at risk at this time (before its deaths and censorings are removed)
n_deaths	Deaths at this time
n_censored	Individuals censored at this time
qx	Interval probability of death, d/n
px	Interval probability of survival, $1 - qx$

lx	Survivorship: the cumulative product of px (identical to km_lx)
hx	Interval hazard, actuarial approximation adjusted for interval width
km_lx	Kaplan-Meier survival estimate $\hat{S}(t)$
se_km	Greenwood standard error of $\hat{S}(t)$ (0 before the first death)
km_ci_lo, km_ci_hi	95% log-log confidence band for $\hat{S}(t)$
na_H	Nelson-Aalen cumulative hazard $H(t)$
na_se	Standard error of $H(t)$
na_ci_lo, na_ci_hi	95% confidence band for $H(t)$

The same folder holds `individual_data_<focus>.csv`, the one-row-per-individual data the life tables were built from (time, event, chamber, treatment and the factor columns), so you can check any number by hand or reanalyse it in other software.

Where it appears

- `data_output/lifetables_<focus>.csv`, as above. The life table itself is not printed in the report.
- The figures drawn from it: [Kaplan-Meier curves](#), [KM curves with at-risk table](#), [Nelson-Aalen cumulative hazard](#), [Hazard rate](#), [Mortality \(qx\)](#) and [Number at risk](#).
- The report's "Lifespan statistics" section, through the median, mean and quantile tables derived from it ([Summary statistics](#)).

Pitfalls

- **Census data has coarse time.** With census-scored chambers, many deaths share each census time, so the table has one row per census age, and the KM curve drops in steps at those ages.
- **The tail is noisy.** Late in the experiment few individuals are at risk, so `qx` and `hx` jump about and the confidence band is wide. The log-log band stays inside 0...1 but becomes lopsided there, wider on the side away from the boundary.
- **Censoring at the end.** With assumed censoring on, individuals unaccounted for are censored at their chamber's last census. The KM curve then stays above zero at the end of the experiment, and the band does not close.
- **Pooling.** Chambers are pooled within a treatment; the life table does not model chamber-to-chamber variation. Use the [Chamber QC viewer](#) to look for outlying chambers.

See also

- [Summary statistics, median and mean survival](#)
- [Censoring policy](#)
- [Kaplan-Meier curves](#)
- [Nelson-Aalen cumulative hazard](#)
- [Mortality \(qx\)](#)

Summary statistics, median and mean survival

Alongside the life tables, every run computes descriptive statistics for each treatment: how

many individuals, deaths and censored individuals it has, its median survival, its mean survival (as a restricted mean, RMST), its survival quantiles, and a lifespan table that can also pool by factor level. These are part of the survivorship core, so they are always computed and have no checkbox.

In the app

Click **Run analysis** on the Analyze panel. All of the tables below are computed for every treatment of the Active Focus. The results go to CSV files under `analysis/<focus>/` and to the report's "Experiment summary" and "Lifespan statistics" sections.

What is computed

Per-treatment summary

Counts of the individuals each treatment contributes after the Focus and the Exclusion Group are applied. Saved as `data_output/summary_<focus>.csv` and shown in the report as

Per-treatment summary:

Column	Meaning
treatment	Treatment label
n_individuals	Individuals in the treatment
n_deaths	Observed deaths (event = 1)
n_censored	Right-censored individuals (event = 0)
pct_censored	$100 \times n_{\text{censored}} / n_{\text{individuals}}$, to one decimal

The report cover also gives the Focus-wide totals (treatments, individuals, deaths, censored and % censored), and the summary section gives the observation window (the earliest and latest time in the data) and the number of chambers.

Median survival

The median is read off the Kaplan-Meier curve: it is the **first time at which $\hat{S}(t) \leq 0.5$** . When the curve never reaches 0.5, for example because many individuals are censored, the median is undefined and shown as blank or "—".

Its **95% confidence interval** is read off the KM curve's 95% confidence band the same way: the lower limit is the first time the band's lower edge reaches 0.5, and the upper limit the first time its upper edge does. The band is the log-log interval described in [Life tables](#), so this is the interval R's `survfit` and `lifelines` report. When the upper edge never falls to 0.5, the upper limit is "not reached" and shown as blank or "—", even when the median itself exists.

Saved as `data_output/median_survival_<focus>.csv` and shown in the report as

Median survival:

Column	Meaning
treatment	Treatment label
median_survival	First time $\hat{S}(t) \leq 0.5$
median_ci_lo, median_ci_hi	The 95% confidence interval for the median

Mean survival (RMST, common restriction time)

A plain mean of death times is biased when some individuals are censored. The app therefore reports the **restricted mean survival time (RMST)**: the area under the KM curve from 0 up to a restriction time τ . You can read it as the average lifespan over the window from 0 to τ .

- **τ is common to all treatments**, so their RMSTs can be compared. It is the smallest of the treatments' last observed times: $\tau = \min$ over treatments of (the largest time, death or censoring, in that treatment).
- **The area** is the exact area under the KM **step function**: $\hat{S}$ is 1 from 0 to the first death, then holds each value until the next death, and the last step is cut off at τ itself. This is the same number as lifelines' `restricted_mean_survival_time`.

Saved as `data_output/mean_survival_<focus>.csv` and shown in the report as **Mean survival**:

Column	Meaning
<code>treatment</code>	Treatment label
<code>rmst</code>	Area under the KM curve up to the restriction time
<code>restriction_time</code>	τ , the same for every row

The same τ rule is used by the [RMST factorial model](#), so `restriction_time` here also tells you which τ that model used.

Lifespan statistics

A second table gives each group's mean and median, and can also pool by factor level. Saved as `statistics/lifespan_treatment_stats_<focus>.csv` and, when the Focus varies two or more factors, `statistics/lifespan_factor_stats_<focus>.csv`. In the report they are **Lifespan statistics by treatment** and **Lifespan statistics by factor level**.

Column	Meaning
<code>group</code>	A treatment, or <code>Factor=level</code> in the factor-level table
<code>n, n_deaths, n_censored</code>	Counts for the group
<code>mean_rmst</code>	Area under the group's KM step function from 0 to the common τ , computed as above
<code>tau</code>	The common τ : the same restriction time as the Mean survival table, in every row of both tables
<code>median</code>	First time the group's KM curve is at or below 0.5
<code>t_max</code>	The group's own last observed time
<code>top_10pct_mean, top_5pct_mean</code>	Mean age at death of the longest-lived 10% / 5% of the individuals that died

Notes:

- **`mean_rmst` uses the common τ** , so for a treatment it is the same number as the `rmst` in the Mean survival table (rounded to 2 decimals), and rows observed for different lengths of time are still compared over the same window.
- **The factor-level rows pool across the other factors**. For example, `Genotype=wt` combines every `wt` treatment of the Focus, whatever its diet. The factor-level table is produced only when the Focus varies two or more factors; with one, it would repeat the treatment table.
- **The top-percentile columns appear only when assumed censoring is off** (`assume_censored: false`) **and the group has more than 10 deaths**. They use

deaths only, so they are meaningful only when there is little censoring. The number kept is the fraction of deaths rounded up, and at least one.

Survival quantiles

The time at which each treatment's KM curve first falls to or below a set of survival levels. Saved as `statistics/survival_quantiles_<focus>.csv` and shown in the report as **Survival quantiles**:

Column	Meaning
treatment	Treatment label
S=90%	First time $\hat{S}(t) \leq 0.90$ (10% have died)
S=75%	First time $\hat{S}(t) \leq 0.75$
S=50%	First time $\hat{S}(t) \leq 0.50$ (the median)
S=25%	First time $\hat{S}(t) \leq 0.25$
S=10%	First time $\hat{S}(t) \leq 0.10$ (90% have died)

A blank cell means the curve never fell that far.

Reading the results

- All times are in the data's time unit (`global.time_unit`, for example days).
- Every table lists treatments in the Focus's display order (the order of its levels), never alphabetically.
- With census data, medians, their confidence limits and quantiles always fall on census ages, because the KM curve and its band only drop at those times. Two treatments can share a median even though their curves differ.
- The report rounds numbers to 3 decimal places. For full precision, use the CSV files.

Pitfalls

- **Heavy censoring.** If a treatment's curve stays above 0.5, its median is missing. Its RMST still exists, but it describes survival only up to τ .
- **A short treatment shortens everyone's τ .** One treatment observed for less time pulls the common restriction time down for every treatment, in the Mean survival table and the lifespan tables alike.
- **Results from before this version.** Earlier versions joined the KM points with straight lines (trapezoids) and stopped at the last time before τ , which gave slightly lower means, and restricted each lifespan-table row to its own last time. Re-run the analysis to get the exact values.

See also

- [Life tables and the Kaplan-Meier estimator](#)
- [The RMST factorial model](#)
- [Censoring policy](#)
- [Lifespan distribution](#)
- [The experiment report](#)

Log-rank pairwise

The pairwise log-rank test asks, for each pair of treatments in turn, whether the two survival curves differ. It is the standard Mantel-Cox log-rank test: it compares the deaths seen in each group with the deaths you would expect if both groups shared one hazard, and it weights every death time equally. The app runs it on every pair of treatments in the Focus and adds a Bonferroni-adjusted p-value.

In the app

On the Analyze panel, tick **Log-rank pairwise** and click **Run analysis**. The checkbox is offered when the Active Focus implies two or more treatments. If fewer than two of those treatments contain individuals (for example, the exclusions emptied one), the run records "Survival comparisons" as Not Applicable, with the reason.

The analysis id for omit: is `logrank_pairwise`. In Experiment Scripts, the `log_rank_pairwise` action runs the same test and prints the table to the log.

What is computed

The test is the app's own implementation; it does not call lifelines, and the p-value comes from SciPy's χ^2 distribution. For a pair of treatments A and B, take every distinct time at which at least one individual of A or B died. At each such time t_j :

- n_A , n_B are the numbers at risk in each group (individuals whose time is $\geq t_j$, so those censored at t_j are still at risk), with $n = n_A + n_B$.
- d_A , d_B are the deaths at t_j , with $d = d_A + d_B$.

Then:

$$\begin{aligned} O_A &= \sum d_A && \text{observed deaths in A} \\ E_A &= \sum n_A \cdot d / n && \text{expected deaths in A} \\ V &= \sum n_A \cdot n_B \cdot d \cdot (n - d) / (n^2 (n - 1)) && \text{hypergeometric variance} \\ \chi^2 &= (O_A - E_A)^2 / V && \text{on 1 degree of freedom} \\ p &= P(\chi^2_1 \geq \chi^2) \end{aligned}$$

Ties (several deaths at the same census time) are handled by the hypergeometric variance above. Times with only one individual at risk add nothing to V. If V is 0 (for example, no deaths at all), the app reports $\chi^2 = 0$ and $p = 1$.

Which pairs. Every unordered pair of treatments is tested once: k treatments give $k(k - 1)/2$ tests. Pairs follow the Focus's display order (the order of its levels, never alphabetical), and within each pair `group1` is the treatment that comes earlier in that order. The χ^2 and p-value do not depend on which group is called group1.

Multiple comparisons. The app adds a **Bonferroni** adjustment over all the pairs in the table: $p_{\text{bonferroni}} = \min(1, p \times \text{number of pairs})$. The `significant_0.05` column is True when the *adjusted* p-value is below 0.05. See [Reading p-values and multiple comparisons](#).

Reading the results

Saved as `analysis/<focus>/statistics/logrank_pairwise_<focus>.csv`, one row per pair:

Column	Meaning
<code>group1, group2</code>	The two treatments compared (group1 earlier in the Focus's order)
<code>observed_1</code>	O: observed deaths in group1
<code>expected_1</code>	E: deaths expected in group1 if the two groups had the same hazard
<code>chi2</code>	Log-rank χ^2 statistic (1 df), rounded to 4 decimals

p_value	Unadjusted p-value
df	Degrees of freedom (always 1)
p_bonferroni	Bonferroni-adjusted p-value, capped at 1
significant_0.05	True when p_bonferroni < 0.05

Direction. The test has no sign, but `observed_1` and `expected_1` tell you which way the difference goes. If group1 has more deaths than expected ($O > E$), group1 dies faster than group2. If it has fewer ($O < E$), group1 lives longer. For a size of effect, see [Pairwise hazard ratios](#).

Where it appears

- The CSV above.
- The report's "Survival comparisons" section, as the **Pairwise log-rank** table, with the same columns. Rows are highlighted when the *Bonferroni-adjusted* `p_bonferroni` is below 0.05, exactly the rows where `significant_0.05` is True. p-values are shown to 4 decimals, or < 0.0001 ; other numbers to 3 decimals. Use the CSV for exact values.
- The run log, when it is run as a script action.

When it is not offered

- A single-treatment Focus is not offered any comparison.
- A Focus that implies two or more treatments but has fewer than two populated records "Survival comparisons" as Not Applicable.

Pitfalls

- **Crossing curves.** The log-rank test is most powerful when one group's hazard is a constant multiple of the other's (proportional hazards). If the curves cross, early and late differences can cancel out and the test can miss a real difference. Look at the KM curves and the [Log-log diagnostic](#), and compare with [Gehan-Wilcoxon](#), which weights early deaths more.
- **Bonferroni is conservative** when there are many treatments. With 6 treatments there are 15 pairs, so each raw p-value is multiplied by 15.
- **Individuals, not chambers.** The test treats every individual as independent and pools chambers within a treatment. Strong chamber effects make p-values look more convincing than they are.

See also

- [Log-rank omnibus](#)
- [Gehan-Wilcoxon pairwise](#)
- [Pairwise hazard ratios](#)
- [Reading p-values and multiple comparisons](#)
- [Kaplan-Meier curves](#)

Log-rank omnibus

The omnibus log-rank test asks one question about the whole Focus: do *any* of the treatments' survival curves differ? It is the K-sample extension of the log-rank test. It gives a single χ^2 and p-value for all treatments together, so it is the natural first look before you go through the

pairwise comparisons.

In the app

On the Analyze panel, tick **Log-rank omnibus** and click **Run analysis**. The checkbox is offered when the Active Focus implies two or more treatments. If fewer than two treatments contain individuals, the run records "Survival comparisons" as Not Applicable.

The analysis id for `omit` is `logrank_omnibus`. In Experiment Scripts, the `log_rank_omnibus` action runs the same test and prints the result to the log.

What is computed

The test is the app's own implementation; the p-value comes from SciPy's χ^2 distribution. With K treatments, at every distinct death time t in the Focus:

- n_k is the number at risk in treatment k (time $\geq t$), and $N = \sum n_k$.
- d_k is the deaths in treatment k at t, and $D = \sum d_k$.
- The expected deaths are $e_k = n_k \cdot D / N$.

The observed-minus-expected deaths are summed over time for K – 1 of the treatments (one is redundant, because the O – E values sum to zero over all K). Their covariance matrix V is also summed over time:

$$\begin{aligned} V_{ii} &+= n_i (N - n_i) D (N - D) / (N^2 (N - 1)) \\ V_{ij} &-= n_i n_j D (N - D) / (N^2 (N - 1)) \quad i \neq j \\ \chi^2 &= (O - E)^T V^{-1} (O - E) \quad \text{on } K - 1 \text{ degrees of freedom} \\ p &= P(\chi^2_{\{K-1\}} \geq \chi^2) \end{aligned}$$

Times with one or no individuals at risk, or with no deaths, are skipped. If V cannot be inverted (for example, no deaths at all), the app reports $\chi^2 = 0$ and $p = 1$.

This is a single test, so **no multiple-comparison adjustment** applies to it.

Reading the results

Value	Meaning
χ^2	The omnibus log-rank statistic, rounded to 4 decimals
df	K – 1, one less than the number of treatments
p	Probability of a χ^2 at least this large if all treatments had the same hazard

A small p (below 0.05) says that at least one treatment's survival differs from the rest. It does not say which one: use [Log-rank pairwise](#) for that. With exactly two treatments, the omnibus test is the same as the single pairwise test.

Where it appears

- **Report cover.** A status line reads "Omnibus log-rank: treatments differ (p = ...)" when $p < 0.05$, or "no overall difference detected" otherwise.
- **Report, "Survival comparisons" section.** The **Overall comparison** table has columns Test, χ^2 , df, p and a significance marker (***) $p < 0.001$, (**) $p < 0.01$, (*) $p < 0.05$, ns otherwise). p-values below 0.0001 are shown as `<0.0001`.
- **Run Summary.** The `omnibus_lr` entry of `analysis/<focus>/run_summary_<focus>.json` holds `chi2`, `p_value`, `df` and `groups`, the treatment labels tested, in the Focus's display order. The Project Report and the Hub read it from there.

There is no separate CSV for the omnibus test.

When it is not offered

- A single-treatment Focus is not offered any comparison.
- A Focus that implies two or more treatments but has fewer than two populated records "Survival comparisons" as Not Applicable.

Pitfalls

- **Many treatments dilute one difference.** If a Focus has eight treatments and only one differs, the omnibus test spreads that one difference over 7 df and may miss it. A pairwise test, or a narrower Focus, can find it.
- **The proportional-hazards caveat applies here too.** Curves that cross can cancel out. Check the KM curves and the [Log-log diagnostic](#).
- **No structure.** The test treats the treatments as unordered categories. It does not use the factorial layout: a 2×2 is tested as four unrelated groups. To separate main effects from an interaction, use the [Interaction analyses](#).

See also

- [Log-rank pairwise](#)
- [Reading p-values and multiple comparisons](#)
- [Interaction analyses \(Factorial Battery\)](#)
- [The Run Summary](#)
- [The experiment report](#)

Gehan-Wilcoxon pairwise

The Gehan-Wilcoxon test is a weighted log-rank test. Like the log-rank test, it compares observed and expected deaths between two treatments. The difference is that each death time is weighted by the number of individuals still at risk. Early deaths, when many are alive, count for much more than late deaths, when few are left. It therefore picks up differences early in life that the ordinary log-rank test can dilute. The app runs it on every pair of treatments and adds a Bonferroni-adjusted p-value.

In the app

On the Analyze panel, tick **Gehan-Wilcoxon pairwise** and click **Run analysis**. The checkbox is offered when the Active Focus implies two or more treatments. If fewer than two of those treatments hold individuals, the run records "Survival comparisons" as Not Applicable.

The analysis id for `omit:` is `gehan_wilcoxon`. In Experiment Scripts, the `gehan_wilcoxon` action runs the same tests and prints the table to the log.

What is computed

The test is the app's own implementation, with the p-value from SciPy's χ^2 distribution. For treatments A and B, at each distinct death time t_j of the pair, with the same notation as the [log-rank test](#) (n_A , n_B at risk, d deaths in total, $n = n_A + n_B$), the weight is $w_j = n$, the total number at risk:

$$\begin{aligned}U &= \sum w_j \cdot (d_A - n_A \cdot d / n) \\V &= \sum w_j^2 \cdot n_A \cdot n_B \cdot d \cdot (n - d) / (n^2 (n - 1)) \\ \chi^2 &= U^2 / V \quad \text{on 1 degree of freedom} \\ p &= P(\chi^2_1 \geq \chi^2)\end{aligned}$$

This is the Gehan (Breslow) generalisation of the Wilcoxon rank-sum test to censored data. If V is 0, the app reports $\chi^2 = 0$ and $p = 1$.

Pairs and ordering. Every unordered pair is tested once, in the Focus's display order, and `group1` is the treatment that comes earlier in that order (never alphabetical). The result does not depend on which group is `group1`.

Multiple comparisons. As for the log-rank test, the app adds a **Bonferroni** adjustment over all the pairs: $p_{\text{bonferroni}} = \min(1, p \times \text{number of pairs})$. `significant_0.05` is `True` when the adjusted p is below 0.05. The Gehan-Wilcoxon and log-rank tables are adjusted separately; nothing corrects for running both. See [Reading p-values and multiple comparisons](#).

Reading the results

Saved as `analysis/<focus>/statistics/gehan_wilcoxon_pairwise_<focus>.csv`:

Column	Meaning
<code>group1, group2</code>	The two treatments compared
<code>chi2</code>	Gehan-Wilcoxon χ^2 (1 df), rounded to 4 decimals
<code>p_value</code>	Unadjusted p-value
<code>df</code>	Always 1
<code>p_bonferroni</code>	Bonferroni-adjusted p-value, capped at 1
<code>significant_0.05</code>	<code>True</code> when $p_{\text{bonferroni}} < 0.05$

Unlike the log-rank table, this table has no observed/expected columns. To see the direction of a difference, look at the KM curves or the hazard ratios.

Comparing with the log-rank test:

- Both significant: the curves differ throughout.
- Gehan-Wilcoxon significant, log-rank not: the difference is concentrated early in life, or the curves converge or cross later.
- Log-rank significant, Gehan-Wilcoxon not: the difference is mainly late in life, where the log-rank test gives deaths more relative weight.

Where it appears

- The CSV above.
- The report's "Survival comparisons" section, as the **Pairwise Gehan-Wilcoxon** table. As with the log-rank table, rows are highlighted when the *Bonferroni-adjusted* p is below 0.05 (the `significant_0.05` rows), and p-values are shown to 4 decimals, or <0.0001 .

When it is not offered

The same as the log-rank test: a single-treatment Focus is not offered it, and one with fewer than two populated treatments records "Survival comparisons" as Not Applicable.

Pitfalls

- **The weights depend on censoring.** The weight is the number at risk, which falls with censoring as well as death. If two treatments have very different censoring patterns, the Gehan weights can distort the comparison.
- **Late differences carry little weight.** By late life few individuals remain, so a big late separation barely moves this test.

See also

- [Log-rank pairwise](#)
- [Reading p-values and multiple comparisons](#)
- [Checking proportional hazards](#)
- [Kaplan-Meier curves](#)

Pairwise hazard ratios

A hazard ratio (HR) says how much faster one treatment dies than another. HR = 2 means that, at any age, an individual of the first group has twice the risk of dying of one in the second. The app estimates an HR with a 95% confidence interval for every pair of treatments in the Focus. These are the numbers the [Hazard-ratio forest](#) draws.

In the app

On the Analyze panel, tick **Pairwise hazard ratios** and click **Run analysis**. The checkbox is offered when the Active Focus implies two or more treatments. If fewer than two treatments hold individuals, the run records "Survival comparisons" as Not Applicable.

The analysis id for `omit:` is `hazard_ratios`. **Unticking it also leaves out the hazard-ratio forest**, because the forest has nothing else to draw. When only the plots are regenerated (**Generate plots**), the hazard ratios are recomputed for the forest if it is ticked.

What is computed

The estimate is the **log-rank observed/expected (O/E) hazard ratio**, computed by the app's own code from the same quantities as the [log-rank test](#). It is not fitted by a Cox regression. For treatments A (`group1`) and B (`group2`), at each distinct death time of the pair:

$$\begin{aligned} O_A &= \sum d_A & E_A &= \sum n_A \cdot d / n \\ O_B &= \sum d_B & E_B &= \sum n_B \cdot d / n \\ \\ HR &= (O_A / E_A) / (O_B / E_B) \\ SE(\log HR) &= \sqrt{1/E_A + 1/E_B} \\ 95\% \text{ CI} &= \exp(\log HR \pm 1.96 \cdot SE) \end{aligned}$$

Here n is the number at risk in both groups together, and d the deaths in both. The HR is reported as missing when E_A , E_B or O_B is zero.

- **Direction.** `group1` is the numerator. HR > 1 means group1 has the higher hazard: it dies faster and lives shorter than group2. HR < 1 means group1 lives longer.
- **Which group is group1.** Within each pair, group1 is the treatment that comes earlier in the Focus's display order (the order of its levels), never the alphabetically first label. It is not necessarily the Reference Level: reordering the levels in the Focus window changes which treatment is group1. The report's table caption says so. To read a pair the other way round, take 1/HR and swap and invert the CI limits.
- **No p-value and no adjustment.** The table has no p-value column, and the confidence intervals are ordinary (unadjusted) 95% intervals for each pair. With many pairs, some intervals will exclude 1 by chance. For a test of each pair, use the [log-rank table](#), which carries a Bonferroni-adjusted p-value.
- **Unadjusted.** Each HR compares two treatments on their own, ignoring the rest of the design. For effects of each factor relative to its Reference Level, with interactions, see [The Cox factorial model](#).

Reading the results

Saved as `analysis/<focus>/statistics/hazard_ratios_<focus>.csv`, one row per pair:

Column	Meaning
group1	Numerator treatment (earlier in the Focus's order)
group2	Denominator treatment
hazard_ratio	O/E hazard ratio of group1 relative to group2, to 4 decimals
hr_ci_lo	Lower 95% confidence limit
hr_ci_hi	Upper 95% confidence limit

An interval that excludes 1 (both limits above 1, or both below) shows a difference at roughly the 5% level for that pair on its own. Because the scale is multiplicative, $HR = 0.5$ is as large an effect as $HR = 2$.

Where it appears

- The CSV above.
- The report's "Survival comparisons" section, as the **Pairwise hazard ratios** table.
- The [Hazard-ratio forest](#) figure, one line per pair.

When it is not offered

The same as the other comparisons: not offered to a single-treatment Focus, and Not Applicable when fewer than two treatments hold individuals.

Pitfalls

- **One number assumes proportional hazards.** A single HR summarises the whole lifespan only if the ratio of hazards stays roughly constant with age. If the KM curves cross or converge, the HR is an average of an effect that changes with age. Check the [Log-log diagnostic](#) and [Checking proportional hazards](#), and consider [RMST](#) as a summary that does not need the assumption.
- **The O/E estimate is approximate.** It agrees closely with a Cox estimate when the effect is modest, but it can differ for large effects or very unequal censoring. For publication-grade HRs, fit a Cox model.
- **Independence.** Like the tests, the CI treats individuals as independent and ignores chamber clustering, so it can be too narrow.

See also

- [Hazard-ratio forest](#)
- [Log-rank pairwise](#)
- [The Cox factorial model](#)
- [Checking proportional hazards](#)
- [Reading p-values and multiple comparisons](#)

Parametric AFT models

Kaplan-Meier makes no assumption about the shape of the survival curve. A parametric model instead fits a named lifespan distribution to the data. The app fits three common ones (Weibull, log-normal and log-logistic) to each treatment separately and compares them by AIC. This tells you which distribution describes each treatment's lifespans best, and gives a model-based median lifespan.

In the app

On the Analyze panel, tick **Parametric AFT models** and click **Run analysis**. This analysis has no Focus requirement: it is offered for every Focus, including a single treatment.

The analysis id for omit: is `parametric_aft`. In Experiment Scripts, the `parametric_aft` action fits the same models and writes the results to the log.

What is computed

For **each treatment on its own**, the app fits three accelerated-failure-time (AFT) models with the **lifelines** library: `WeibullAFTFitter`, `LogNormalAFTFitter` and `LogLogisticAFTFitter`. Each is fitted to that treatment's times and event flags only, with no covariates, so each fit is simply the best two-parameter distribution of that family for that treatment, estimated by maximum likelihood. Censored individuals enter the likelihood correctly: they contribute the probability of surviving past their censoring time.

Model	Shape of the hazard it can describe
Weibull	Hazard that rises (or falls) steadily with age as a power of age
Log-normal	Hazard that rises, peaks and then falls
Log-logistic	Like the log-normal, with heavier tails

For each fit the app records:

- **AIC**, the Akaike information criterion: $AIC = -2 \log L + 2k$, where k is the number of parameters. All three models have two parameters, so the ranking by AIC is the same as the ranking by log-likelihood.
- **log-likelihood** of the fit.
- **median survival** predicted by the fitted distribution.

The **best model** for a treatment is the one with the lowest AIC.

Treatments with **fewer than 5 individuals or fewer than 2 deaths are not fitted**, and a model that fails to converge is not fitted either. Neither disappears: each gets a row in the comparison saying why (for example "not fitted: only 3 individual(s) — needs at least 5" or "not fitted: fit failed: ..."), and the run log says the same. If the whole step fails, the run continues and the table holds a single row with the reason.

No test is performed, so there are no p-values and no multiple-comparison adjustment. The models do not compare treatments with each other.

Reading the results

The AIC comparison has one row per treatment and model, treatments in the Focus's display order, plus one row for each treatment that was too small to fit:

Column	Meaning
<code>treatment</code>	Treatment label
<code>model</code>	Weibull, Log-Normal or Log-Logistic; empty for a treatment too small to fit
<code>aic</code>	AIC of the fit (lower is better), to 2 decimals
<code>delta_aic</code>	AIC minus the lowest AIC of that treatment, so 0 marks the best fit
<code>log_likelihood</code>	Maximised log-likelihood

median_survival	Median lifespan predicted by the fitted distribution
best	True for the treatment's lowest-AIC model
note	lowest AIC for the best model; for a row with no fit, not fitted: and the reason

How to read it:

- Compare AICs **within a treatment**, never across treatments. Each treatment's AIC depends on its own sample size and data.
- A difference of less than about 2 AIC units means the models fit about equally well. A difference of 10 or more is strong support for the lower one.
- Compare the model's `median_survival` with the KM median in [Summary statistics](#). Close agreement is a sign of a good fit. Note that the KM median always falls on a census age, while the model median does not.
- If one family wins for every treatment, it is a reasonable choice for further parametric modelling.

Where it appears

- **The report**, in the "Data quality" section, as the table **Parametric model fits (AFT)**: Treatment, Model, AIC, Δ AIC, Median (model) and Note. Each treatment's best model is highlighted, and rows that were not fitted are highlighted as warnings.
- **The CSV analysis**/`<focus>/statistics/parametric_aic_<focus>.csv`, with the columns above. A later run that leaves the analysis out deletes it, so it never reads as current.
- **The Run Summary**, as `parametric_models`: the same rows as a list of records. The [Project Report](#) rebuilds the report's table from it.
- **The run log**, when run as the `parametric_aft` script action, including the AIC comparison table.

The fitted parameters themselves (for example the Weibull scale and shape) are not tabulated.

Pitfalls

- **Census data is interval data**. Deaths are recorded at census ages, so many share a time. The models treat each recorded age as exact.
- **Assumed censoring adds a block of censored individuals at the last census**, which mostly affects the upper tail of the fit.
- **Per-treatment fits only**. The app does not fit an AFT regression with treatment or factor effects. To model factor effects, use the [Cox](#) or [RMST](#) factorial models.

See also

- [Summary statistics, median and mean survival](#)
- [Lifespan distribution](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [Censoring policy](#)
- [The experiment report](#)

Interaction analyses (Factorial Battery)

When a Focus crosses two or more factors (for example Genotype × Diet), the key question is often not just whether each factor matters, but whether they **interact**: does the effect of diet depend on genotype? The **Interaction analyses** checkbox runs the two models of the **Factorial Battery** that answer this:

- the **Cox factorial model**, which estimates factor effects as hazard ratios, tests all interactions together with a likelihood-ratio (LR) test, and checks proportional hazards;
- the **RMST factorial model**, which estimates the same factorial design as differences in restricted mean lifespan and does not assume proportional hazards.

Both are fitted relative to each factor's **Reference Level**.

In the app

1. On the Experiment panel, make the Active Focus one that varies two or more factors. The shape line (for example 2×2) tells you whether the battery will be offered.
2. Optionally, set each factor's **Reference Level** in the **Focus window** (the *Reference*: box beside each factor's levels). See [Reference Levels](#).
3. On the Analyze panel, tick **Interaction analyses** and click **Run analysis**.

One checkbox runs both models; you cannot run one without the other from the Analyze panel. The analysis id for `omit`: is `interaction`.

The battery's two figures, the **Faceted Kaplan-Meier** and the **Lifespan interaction plot**, are separate Plot Set items on the Plots panel and are not controlled by this checkbox.

In Experiment Scripts, the actions `cox_interaction` and `rmst_interaction` run each model on its own under the step's Focus. `rmst_interaction` has a τ parameter (0 = automatic). They write their results to the log. Placed after a `run_analysis` step, each refitted model **replaces** the run's model of the same name, so a later `report` step shows every model once.

When it is offered, and when it is Not Applicable

Situation	What happens
The Focus varies fewer than two factors	Not offered: no checkbox, no report section
It varies two or more factors, and every pair of them is fully crossed in the populated cells	Both models are fitted
It varies two or more factors, but some combination of levels has no individuals	Not Applicable : the run records "Factorial Battery" with the empty cells named (for example <code>Genotype=mut × Diet=DR</code>), and the checkbox is greyed with that reason

Any number of levels per factor is allowed. A "factor" here is a *varying* factor of the Focus: one named with two or more levels. A factor the Focus filters to one level, or pools over, does not enter the model.

The crossing must be complete because an interaction term compares a combination of levels with the others. If that combination has no individuals, its term cannot be estimated.

The model design

Both models use the same design, built from the Focus's varying factors:

- **Main effects**: each factor is dummy-coded with its Reference Level as the baseline. A factor with L levels gets L – 1 terms, named `Factor_level` (for example `Diet_DR`).

- **Pairwise interactions:** for every pair of factors, the product of each of their dummies, named $A_a:B_b$ (for example `Genotype_mut:Diet_DR`). A pair with L^A and L^B levels gets $(L^A - 1)(L^B - 1)$ terms.
- **Only pairwise interactions** are included. With three factors, the model has the three two-way interactions but no three-way term.

The design is fitted to individuals, not to treatment means: every individual in the Focus is one row.

Reading the results

How to interpret the coefficients when interactions are in the model:

- A **main-effect** coefficient is the effect of that level **at the Reference Level of every other factor**. In a Genotype $\times$ Diet model with references wt and AL, `Genotype_mut` is the effect of mut *on the AL diet only*. It is not an average over diets.
- An **interaction** coefficient is how much that effect changes at the other level. In the Cox model it is a ratio of hazard ratios; in the RMST model it is a difference of differences in mean lifespan.
- The **Cox LR test** asks whether the interaction terms, taken together, improve the fit. A small p-value means the factors' effects are not additive (on the log-hazard scale).

The [Lifespan interaction plot](#) shows the same idea graphically: lines that are not parallel suggest an interaction.

Where it appears

- **Report, "Factorial analysis" section:** one subsection per model, giving the formula, the model line (n and events; concordance for Cox; τ , R^2 and the robust F test for RMST; AIC), the Cox LR interaction table, the coefficient table (captioned with the Reference Levels), the Cox model's Schoenfeld check, and a highlighted table of any warnings raised while fitting that model. If the battery was Not Applicable, the section is a single line, "Factorial Battery: not run", with the reason. If it was unticked, it reads "Interaction analyses: left out of this run".
- **CSV files** in `analysis/<focus>/statistics/`:
 - `factorial_01_coefficients_<focus>.csv`: Cox coefficients
 - `factorial_01_ph_test_<focus>.csv`: Cox Schoenfeld PH check
 - `factorial_02_coefficients_<focus>.csv`: RMST coefficients
- **Run Summary:** `factorial_models` in `run_summary_<focus>.json` lists each model's title, type, formula, any error, n, events, concordance, AIC, Reference Levels, its warnings, and, for Cox, the LR interaction test; for RMST, `tau`, `r_squared` and the F test.

If a model fails to fit, the run continues. The report then says "*model* could not be fitted", with the error.

Pitfalls

- **Changing a Reference Level changes the coefficients, but not the fit.** Main effects then refer to a different baseline cell, and interaction signs can flip. The LR test and the fitted survival do not change. Changing a Reference Level marks the Focus's results Out of Date.
- **Unbalanced or sparse cells** give wide intervals. A cell with very few deaths can make the Cox fit unstable.

- **Display order is not the baseline.** Reordering levels for figures does not change the model unless you also change the Reference Level.

See also

- [The Cox factorial model](#)
- [The RMST factorial model](#)
- [Checking proportional hazards](#)
- [Reference Levels](#)
- [Focus Shape — what a Focus is offered](#)
- [Lifespan interaction plot](#)

The Cox factorial model

The Cox factorial model is the hazard-ratio half of the [Factorial Battery](#). It fits a Cox proportional-hazards regression with every varying factor of the Focus and every pairwise interaction between them. It reports each term as a hazard ratio relative to the Reference Levels, tests whether the interactions are needed with a likelihood-ratio test, and checks the proportional-hazards assumption term by term.

In the app

Tick **Interaction analyses** on the Analyze panel and click **Run analysis**. This fits the Cox model together with the [RMST factorial model](#). It is offered only when the Active Focus varies two or more factors, and it is computed only when every pair of them is fully crossed in the data. Otherwise the battery is recorded as Not Applicable. In an Experiment Script, the `cox_interaction` step fits it on its own and writes the formula, the LR test and the coefficients to the log.

The baseline of each factor is its **Reference Level**, set in the [Focus window](#) or with `reference:` in the Focus's config:

```
focuses:
  Genotype x Diet:
    factors:
      Genotype: [wt, mut]
      Diet: [AL, DR]
    reference:
      Diet: AL
```

A factor with no `reference:` entry uses its first listed level. See [Reference Levels](#).

What is computed

Two Cox models are fitted with the **lifelines** library (`CoxPHFitter`, default settings: no penalty, Efron's method for tied death times) to every individual in the Focus:

1. **Main-effects model:** the dummy-coded factors only.
2. **Interaction model:** the main effects plus every pairwise interaction term (see [the model design](#)).

The model for an individual's hazard at age t is

$$h(t) = h_0(t) \cdot \exp(\beta_1 x_1 + \beta_2 x_2 + \dots)$$

where $h_0(t)$ is an unspecified baseline hazard (the hazard of the reference cell) and the x are the 0/1 dummies. **All reported coefficients come from the interaction model.**

The LR interaction test compares the two models:

```
LR = 2 * (log L_interaction - log L_main)
df = number of interaction terms
p = P( $\chi^2_{df} \geq LR$ )
```

It asks whether all the interaction terms together significantly improve the fit. It is one test, so no adjustment is applied. If the design has no interaction terms, the test is not produced.

The proportional-hazards check runs a Schoenfeld-residual test on each term of the interaction model; see [Checking proportional hazards](#).

Reading the results

Model line

The report gives **n** (individuals), **events** (deaths), **concordance** (Harrell's C of the interaction model: the probability that, of two individuals, the model ranks the one that died first as higher-risk; 0.5 is chance, 1 is perfect) and **AIC** (the partial-likelihood AIC of the interaction model, $-2 \log L + 2 \times \text{number of terms}$). The formula lists every term, for example `Genotype_mut + Diet_DR + Genotype_mut:Diet_DR`.

Interaction test

The report's interaction table has the row "Interaction (LR, vs main-effects model)", with χ^2 , df, p and a significance marker. The statistic is recorded as `statistic` in the Run Summary, under `factorial_models → lr_interaction`, together with `df`, `p_value`, the two log-likelihoods (`ll_main`, `ll_interaction`) and the main-effects model's concordance (`concordance_main`). The same value is also kept under its older name `lr_stat`, so tools written against earlier Run Summaries still find it.

Coefficients

Saved as `analysis/<focus>/statistics/factorial_01_coefficients_<focus>.csv` and shown in the report's **Coefficients** table, whose caption names the Reference Levels:

Column	Meaning
<code>covariate</code>	The term: <code>Factor_level</code> (main effect) or <code>A_a:B_b</code> (interaction)
<code>coef</code>	β , the log hazard ratio
<code>HR</code>	$\exp(\beta)$, the hazard ratio
<code>se</code>	Standard error of β
<code>coef_lo</code> , <code>coef_hi</code>	95% Wald confidence interval for β ($\beta \pm 1.96 \cdot se$)
<code>HR_lo</code> , <code>HR_hi</code>	The same interval on the hazard-ratio scale
<code>z</code>	Wald statistic, β / se
<code>p_value</code>	Two-sided Wald p-value for $\beta = 0$, unadjusted
<code>-log2(p)</code>	The p-value as "bits of evidence against the null" (lifelines' S-value)
<code>term_type</code>	<code>main_effect</code> or <code>interaction</code>

Interpretation, with references `wt` and `AL`:

- `Genotype_mut` HR = 0.6: on the **AL** diet, `mut` flies have 40% lower hazard than `wt`.
- `Diet_DR` HR = 0.5: in **wt** flies, `DR` halves the hazard compared with `AL`.
- `Genotype_mut:Diet_DR` HR = 1.3: the `DR` effect in `mut` is 1.3 times the `DR` effect in `wt`. Equivalently, the hazard ratio of `mut/DR` against `wt/AL` is $0.6 \times 0.5 \times 1.3$. An

interaction HR of 1 means no interaction.

- HR < 1 means longer life than the reference; HR > 1 means shorter.

The report highlights coefficient rows with an unadjusted p below 0.05. It shows p-values to 4 decimals, or <0.0001, and other numbers to 3 decimals; use the CSV for exact values.

Model warnings

If lifelines warns while fitting either model (for example that the fit did not converge cleanly, or that a column has very low variance), or the PH check cannot be computed, the message is kept with the model. The report lists it in a highlighted **Cox factorial model: warnings** table under the model, and the Run Summary stores it in that model's `warnings` list. Treat the numbers above such a warning with caution.

Where it appears

- Report, "Factorial analysis" section, under the heading **Cox factorial model**.
- `statistics/factorial_01_coefficients_<focus>.csv` and `statistics/factorial_01_ph_test_<focus>.csv`. The Cox model is always model 01.
- Run Summary: `factorial_models[0]`.
- In an Experiment Script, a `cox_interaction` step after `run_analysis` refits this model and **replaces** the run's copy, so a later `report` step shows it once.

Pitfalls

- **Read main effects at the reference.** With an interaction in the model, a main effect is a *simple* effect at the other factors' Reference Levels, not an average effect. If the interaction is negligible and you want average effects, the main-effects model would be the one to report. The app fits it for the LR test but tabulates only the interaction model.
- **Proportional hazards.** Every HR assumes the effect is constant across age. Check the PH table and the [Log-log diagnostic](#). If PH fails, prefer the [RMST model](#).
- **Several p-values per model.** The coefficient p-values are not adjusted for each other. The LR test is the one designed to answer "is there any interaction?".
- **Clustering.** Individuals in a chamber share conditions, but the model treats them as independent and has no chamber term, so standard errors can be too small.

See also

- [Interaction analyses \(Factorial Battery\)](#)
- [The RMST factorial model](#)
- [Checking proportional hazards](#)
- [Reference Levels](#)
- [Reading p-values and multiple comparisons](#)

The RMST factorial model

The RMST factorial model is the companion to the [Cox factorial model](#) in the [Factorial Battery](#). It fits the same factorial design (main effects plus every pairwise interaction, relative to the Reference Levels), but on the **restricted mean survival time (RMST)**: the average lifespan up to a cut-off age τ . Its coefficients are differences in mean lifespan, in your time unit, and it **does not assume proportional hazards**. Use it when the Cox model's PH check fails, or whenever "days of life gained" is easier to explain than a hazard ratio.

In the app

Tick **Interaction analyses** on the Analyze panel and click **Run analysis**. The RMST model is fitted alongside the Cox model. It has the same requirement: the Focus must vary two or more factors, every pair fully crossed. In a full run, τ is always chosen automatically.

In an Experiment Script, the `rmst_interaction` step fits this model on its own under the step's Focus. Its τ parameter lets you set the restriction time yourself (0 = automatic). Its results go to the log. After a `run_analysis` step, the refitted model also **replaces** the run's RMST model, so a later `report` step shows it once, with the τ the step used.

What is computed

This is the pseudo-value regression method (Andersen and colleagues). It turns a censored outcome into one number per individual, which can then go into an ordinary regression.

1. Choose τ . By default, τ is the smallest of the treatments' last observed times: $\tau = \min$ over treatments of (the largest time, death or censoring, in that treatment). Every treatment is therefore followed for at least τ . This is the same rule as the `restriction_time` in the [Mean survival](#) table, and on the same data it gives the same τ . The report prints the τ used on the model line and in a sentence under it, and the Run Summary records it as `tau`.

2. RMST of the whole Focus. θ is the area under the Kaplan-Meier curve of **all individuals in the Focus pooled**, from 0 to τ . The curve is a step function, flat between death times, and the area is computed exactly: each step's height times its width, with the last step cut off at τ itself (the same calculation as the Mean survival table, and the same number as lifelines' `restricted_mean_survival_time`).

3. Jackknife pseudo-values. For each individual i , θ_{-i} is recomputed with that individual left out, and

$$\text{pseudo}_i = n \cdot \theta - (n - 1) \cdot \theta_{-i}$$

Pseudo-values average to the RMST of the group they come from, and censored individuals get sensible values without being treated as deaths.

4. Regression. The pseudo-values are regressed on the factorial design with ordinary least squares (**statsmodels** OLS), with an intercept, and with **heteroskedasticity-robust (HC1) standard errors**:

$$\text{pseudo}_i = \beta_0 + \sum \beta \cdot (\text{main-effect dummies}) + \sum \beta \cdot (\text{interaction dummies}) + \text{error}$$

z-statistics, two-sided p-values and 95% confidence intervals use the normal distribution with the robust standard errors. No multiple-comparison adjustment is applied.

Reading the results

Saved as `analysis/<focus>/statistics/`

`factorial_02_coefficients_<focus>.csv` and shown in the report's **Coefficients** table under **RMST factorial model**:

Column	Meaning
<code>covariate</code>	<code>const</code> (intercept), <code>Factor_level</code> (main effect) or <code>A_a:B_b</code> (interaction)
<code>coef</code>	Estimate, in the data's time unit
<code>HR</code> , <code>HR_lo</code> , <code>HR_hi</code>	Always blank: not applicable to an RMST model (kept so the table matches the Cox table's layout)
<code>se</code>	Robust (HC1) standard error
<code>z</code>	<code>coef</code> / <code>se</code>

p_value	Two-sided p-value for coef = 0, unadjusted
coef_lo, coef_hi	95% confidence interval for coef
term_type	intercept, main_effect or interaction

Interpretation, with Reference Levels wt and AL and time in days:

- `const = 46`: the estimated mean lifespan up to τ in the **reference cell** (wt on AL).
- `Genotype_mut = +11`: on **AL**, mut lives on average 11 days longer than wt, within the window up to τ .
- `Diet_DR = +14`: in **wt**, DR adds 14 days.
- `Genotype_mut:Diet_DR = -8`: the DR benefit is 8 days smaller in mut than in wt (difference of differences). So mut/DR is estimated at $46 + 11 + 14 - 8 = 63$ days. An interaction of 0 means the effects add up.

The report highlights coefficient rows whose unadjusted p is below 0.05, except the intercept: its test (is the reference cell's mean lifespan zero?) answers no question anyone asked.

The model line in the report gives n, events, τ , R^2 (the share of the pseudo-values' variance the design explains), AIC and the **robust F** test of all terms together, with its p-value. This AIC belongs to the least-squares fit of the pseudo-values: it is not comparable with the Cox model's AIC and has no direct survival interpretation. There is no concordance, no LR interaction test and no PH check for this model. Use the interaction coefficients and their p-values; with more than one interaction term, judge them together.

If statsmodels warns while fitting, the message is listed under the model in a highlighted **RMST factorial model: warnings** table and stored in the model's `warnings` list in the Run Summary.

Where it appears

- Report, "Factorial analysis" section, under the heading **RMST factorial model**.
- `statistics/factorial_02_coefficients_<focus>.csv`. The RMST model is always model 02.
- Run Summary: `factorial_models[1]` (title, formula, n, events, AIC, Reference Levels, `tau`, `rmst_overall` (θ), `r_squared`, `f_statistic`, `f_p_value` and `warnings`).

Pitfalls

- **Everything is "up to τ ".** The RMST says nothing about survival after τ . If one treatment ends early, τ is short for every treatment, and differences late in life are not seen. Check `restriction_time` in `mean_survival_<focus>.csv`.
- **Main effects are simple effects** at the other factors' Reference Levels, exactly as in the Cox model.
- **Large Focuses take longer.** The jackknife refits the Kaplan-Meier curve once for each distinct (age, died or censored) pair, since leaving out either of two identical individuals gives the same curve. Census data has few distinct ages, so this is usually quick, but a Focus with thousands of distinct exact ages takes noticeably longer.
- **Clustering.** The robust standard errors allow unequal variances, but they do not account for chambers. Individuals are still treated as independent.

See also

- [Interaction analyses \(Factorial Battery\)](#)

- [The Cox factorial model](#)
- [Summary statistics, median and mean survival](#)
- [Checking proportional hazards](#)
- [Reference Levels](#)

Checking proportional hazards

Hazard ratios, the Cox model, and to a degree the log-rank test all rest on the **proportional-hazards (PH) assumption**: the ratio of two groups' hazards stays the same at every age. If treatment A halves the risk of death at 10 days, it should also halve it at 60 days. When the assumption fails (curves that cross, or an effect that fades or appears late), a single hazard ratio is an average of an effect that changes with age. The app gives you one formal test and one visual check.

The formal test: Schoenfeld residuals

Where it is run

The test is part of the [Cox factorial model](#), so it runs whenever **Interaction analyses** is ticked on the Analyze panel and the Focus admits the [Factorial Battery](#) (two or more varying factors, fully crossed). It is not run for the pairwise hazard ratios, the log-rank tests, or one-factor Focuses.

What is computed

The app calls the **lifelines** function `proportional_hazard_test` on the fitted Cox **interaction** model, with `time_transform="rank"`. For each term of the model, this tests whether the term's scaled Schoenfeld residuals trend with the rank of the death times. A trend means the term's effect, its log hazard ratio, drifts with age. Each term gets a χ^2 statistic on 1 degree of freedom and a p-value.

There is **one row per model term**: every main-effect dummy and every interaction dummy. No global (whole-model) test is reported, and **no multiple-comparison adjustment** is applied across the rows.

Reading the results

Saved as `analysis/<focus>/statistics/factorial_01_ph_test_<focus>.csv` and shown in the report's "Factorial analysis" section, under the Cox model, as **Proportional-hazards check (Schoenfeld residuals)**:

Column	Meaning
covariate	The model term, e.g. Diet_DR or Genotype_mut:Diet_DR
test_statistic	χ^2 statistic (1 df) for a time trend in that term's effect
p_value	p-value for that trend

How to read it:

- **A small p-value (below 0.05)** means that term's hazard ratio is not constant with age, so the PH assumption is doubtful for it. The report caption says the same: small p-values indicate violation.
- **A large p-value** means no evidence against PH. It does not prove PH holds, especially with few deaths.
- With large samples, the test can flag departures too small to matter. Look at the curves before deciding.
- With many rows, expect about one in twenty to fall below 0.05 by chance.

If the test itself fails to run, the PH table is missing from the report and no `ph_test` CSV is written for that run. The failure is not silent: the report lists it, as "PH assumption test failed: ...", in the highlighted **Cox factorial model: warnings** table under the model, and the Run Summary keeps it in that model's `warnings` list. Warnings lifelines raises while running the test are listed there too.

The visual check: the log-log plot

The [Log-log diagnostic](#) on the Plots panel plots $\log(-\log \hat{S}(t))$ against log time for each treatment. Under proportional hazards the curves are roughly **parallel**, a constant vertical distance apart. Curves that converge, diverge or cross suggest non-proportional hazards. The plot is offered whenever the Focus has two or more treatments, so it is the check to use for one-factor Focuses, and for the pairwise hazard ratios, which have no formal test. The [Kaplan-Meier curves](#) help too: KM curves that cross are a clear sign that hazards are not proportional.

What to do when PH fails

- Use the [RMST factorial model](#). It estimates effects as differences in mean lifespan up to τ and needs no PH assumption. It is fitted alongside the Cox model.
- For pairwise comparisons, compare the **log-rank** test with **Gehan-Wilcoxon**, which weights early deaths more. If they disagree, the difference is concentrated at one end of life. See [Gehan-Wilcoxon pairwise](#).
- Report hazard ratios as averages over the observed period, or describe the curves directly: medians, survival quantiles and RMST ([Summary statistics](#)).

Pitfalls

- **Census data have many ties.** Many deaths share each census age, which makes the residual trend coarse. Treat borderline p-values with caution.
- **Interaction terms are tested too.** A PH failure in an interaction row means the *interaction* changes with age, not necessarily either factor alone.

See also

- [The Cox factorial model](#)
- [The RMST factorial model](#)
- [Log-log diagnostic](#)
- [Pairwise hazard ratios](#)
- [Reading p-values and multiple comparisons](#)

Reading p-values and multiple comparisons

A run can produce dozens of p-values: one per treatment pair for two different tests, one per model coefficient, one per PH check. This page explains what each p-value tests, which ones the app adjusts for multiple comparisons (only two tables are), and how the report displays them, so you can judge what is genuinely significant.

What a p-value means here

A p-value is the probability of seeing a difference at least as large as the one observed if there were really no effect. A p below 0.05 is the conventional threshold for "statistically significant". It says nothing about how large or important an effect is. Read it alongside the effect size: the hazard ratio, the difference in RMST or median, or the KM curves themselves. All p-values in the app are two-sided, and the threshold is always 0.05; it cannot be changed.

Which p-values are adjusted

Analysis	p-value tests	Adjustment
Log-rank pairwise	two treatments' survival curves are equal	Bonferroni across all pairs in the table (<code>p_bonferroni</code>)
Gehan-Wilcoxon pairwise	the same, weighting early deaths	Bonferroni across all pairs in the table (<code>p_bonferroni</code>)
Log-rank omnibus	all treatments are equal	none needed (a single test)
Pairwise hazard ratios	no p-value; 95% CI per pair	none: each CI is an ordinary 95% interval
Cox factorial model, LR interaction	all interaction terms are zero	none needed (a single test)
Cox coefficients	each $\beta = 0$ (Wald test)	none
RMST factorial model coefficients	each coefficient = 0 (robust z-test)	none
Schoenfeld PH check	each term's effect is constant with age	none
Parametric AFT models	no p-values (models are compared by AIC)	not applicable

Bonferroni, as the app applies it

For the two pairwise tables, with m = the number of pairs (k treatments give $m = k(k - 1)/2$):

```
p_bonferroni = min(1, p * m)
significant_0.05 = p_bonferroni < 0.05
```

The adjustment is made separately within each table. The log-rank and Gehan-Wilcoxon tables are each corrected for their own pairs, but nothing corrects for running both tests, for running several Focuses, or for looking at both the pairwise tables and the factorial models. Bonferroni controls the chance of *any* false positive among the pairs. It is simple and safe but conservative: with many treatments it can miss real differences.

Unadjusted p-values

For the model coefficients and PH checks, the p-values are as the fitting library reports them (lifelines for Cox, statsmodels for RMST). If you test many coefficients, expect about one in twenty to fall below 0.05 by chance when nothing is going on. You can apply your own correction to the values in the CSV files. Where a single combined test exists, prefer it: the Cox **LR interaction test** answers "is there any interaction?" in one p-value, instead of reading each interaction coefficient separately.

How p-values are displayed

In the CSV files (`analysis/<focus>/statistics/`), p-values are stored at full precision. Use them for anything you publish.

In the report:

- **Single-test tables** (the omnibus log-rank "Overall comparison" and the Cox "Interaction (LR, vs main-effects model)" row) show p to 4 decimals, or <0.0001 , followed by a marker: *** for $p < 0.001$, ** for $p < 0.01$, * for $p < 0.05$, *ns* otherwise.
- **Multi-row tables** (pairwise log-rank, pairwise Gehan-Wilcoxon, model coefficients, PH check) print p-value columns (`p_value`, `p_bonferroni`) the same way, to 4 decimals

or <0.0001 , so a tiny p-value is never shown as 0.000 . Other numbers are printed to 3 decimals.

- **Row highlighting.** In the pairwise tables, a row is highlighted when its **Bonferroni-adjusted** `p_bonferroni` is below 0.05, so the highlight always agrees with the `significant_0.05` column. In the coefficient tables, which have no adjusted p-value, a row is highlighted when its unadjusted `p_value` is below 0.05 (never the RMST model's intercept row).
- **Report cover.** The omnibus log-rank p-value appears as a status line: "treatments differ" when $p < 0.05$, "no overall difference detected" otherwise.

Pitfalls

- **Individuals are treated as independent.** Every test treats each individual as an independent observation. Individuals in the same chamber share conditions, so if chambers differ, the effective sample size is smaller than the number of individuals and p-values are too small. Check chamber consistency in the [Chamber QC viewer](#).
- **Choosing a Focus after looking at the data** is a hidden multiple comparison. Decide your main comparisons before you look.
- **Non-significant is not "no effect".** With few deaths, real differences can give large p-values. Look at confidence intervals: a wide interval that includes 1 (for an HR) or 0 (for an RMST difference) means the data cannot tell.
- **Very small p-values in large cohorts** can come with tiny effects. Always check the effect size.

See also

- [Log-rank pairwise](#)
- [Gehan-Wilcoxon pairwise](#)
- [The Cox factorial model](#)
- [Checking proportional hazards](#)
- [The experiment report](#)

Figures

The Plots panel and Generate plots

The Plots panel lists the **Plot Set** of the Active Focus — every standard figure this Focus can get — with one checkbox per figure, and one **Generate plots** button below them. A ticked box means "draw this figure"; an unticked box means "leave it out of every run". The same ticks govern **Generate plots**, **Run analysis** on the Analyze panel, Experiment Scripts and Batch runs.

In the app

Open the Experiment tile and choose the **Plots** sub-tile. With no experiment loaded the panel only says "Load an experiment to see its plots."

- **The checkboxes.** One per figure in the Plot Set, in the order the report uses. Hover over a box to see its caption. Each box has a **?** button that opens that figure's page in this manual.
- **Ticking and unticking** saves at once. An unticked figure is written to the `omit:` block of `survival_config.yaml` (under `plots:`), and the log says so, e.g. `Log-log`

diagnostic: left out of <experiment>'s runs —
 survival_config.yaml omit:. Ticking it again removes it from the block.

- **Generate plots** draws the ticked figures for the Active Focus. Each figure opens in its own tab, titled <figure> — <Focus name>, and is saved as a PNG under analysis/<focus>/plots/.
- **The "Not offered" line.** Under the button, a grey italic line names figures this Focus is not offered (for example "Not offered for Focus Females: Faceted Kaplan-Meier, Lifespan interaction plot."). Hover over it to see why.

Which figures are listed

The Standard Lifespan Plot Set, in order:

Figure	Plot id	Needs
KM curves with at-risk table	km_risk_table	—
Kaplan-Meier curves	km_curves	—
Cumulative deaths	cumulative_events	—
Lifespan distribution	survival_distribution	—
Mortality (qx)	mortality	—
Hazard rate	hazard	—
Smoothed hazard	smoothed_hazard	—
Nelson-Aalen cumulative hazard	nelson_aalen	—
Number at risk	number_at_risk	—
Hazard-ratio forest	hazard_ratio_forest	a comparison; Pairwise hazard ratios ticked
Log-log diagnostic	log_log	a comparison
Faceted Kaplan-Meier	km_faceted	a figure crossing factors
Lifespan interaction plot	interaction_lifespan	a figure crossing factors

A figure's requirement decides how it appears (see [Focus Shape](#)):

- **Not offered.** The Focus's definition does not ask the question. A single-treatment Focus gets no forest and no log-log diagnostic. A Focus that varies fewer than two factors gets no faceted KM and no interaction plot. These figures have no checkbox and are not recorded anywhere.
- **Not Applicable (greyed).** The figure is relevant but the data, after exclusions, cannot support it. For example, the Focus implies two treatments but only one holds individuals. The box is greyed and its tooltip gives the reason. The Hub works this out from the Design sheet and the active Exclusion Group before anything runs.
- **Greyed because an analysis is unticked.** The hazard-ratio forest draws from **Pairwise hazard ratios**. If that box is unticked on the Analyze panel, the forest box is greyed and its tooltip says "Draws from Pairwise hazard ratios — tick it on the Analyze panel to draw this figure." Runs record the forest as Left Out with the reason "draws from Pairwise hazard ratios, which is unticked".

What Generate plots does, and does not do

- It loads the Active Focus's slice, with exclusions applied, and computes the lifetables afresh.
- If the forest is ticked (and Pairwise hazard ratios is ticked), it computes the pairwise hazard ratios. It computes no other statistic.
- It writes each figure to `analysis/<focus>/plots/<file>_<focus>.png` at 150 dpi, overwriting the earlier version. The file names are listed on each figure's page.
- It does **not** rewrite the statistics, the report or the Run Summary. They stay as the last **Run analysis** left them. To get new figures into the report, run the analysis again.
- It also draws the workbook's [Defined Plots](#) that are about the Active Focus, each in its own tab titled `Defined Plot - <name> - <Focus name>`, exactly as a full run does.
- It logs every figure it leaves out or finds Not Applicable ("Left out — ...", "Not applicable — ...") and ends with "N figure(s) saved to ...".

A Blocked Focus (one that names factors or levels the data file lacks, or whose cells were all excluded) cannot be drawn. Generate plots stops with the reason.

In a full run

Run analysis draws the same ticked figures the same way, because both buttons share one drawing routine. A full run also:

- records each left-out figure under "Left out of this run" in the report and under `left_out` in the Run Summary, and each Not Applicable figure under "Not applicable to this Focus";
- deletes a Plot Set or Defined Plot PNG that an earlier run wrote and this run did not, so a folder never shows a stale figure as if it were current;
- places the figures in the report's figures section with the **Headline Figure** first. That is the [KM curves with at-risk table](#), or the [Faceted Kaplan-Meier](#) when the Focus crosses factors and the data can draw it. Where a figure has been curated in the [Plot Editor](#), the report shows the curated version in its place.

If one figure fails to draw, it is recorded as Not Applicable ("could not be drawn (...)") and the other figures are still drawn.

How the figures look

- Treatments are drawn in the **Focus's order**, labelled with its **display names**, in its **colours**: the same order, names and per-curve colours the Publication Figures use. A treatment the Focus gives no colour takes a fixed colour cycle by its position in the Focus's order, so it keeps one colour across every analysis figure of the Focus. (A Publication Figure's fallback is its Style's own cycle instead.)
- The time axis carries your configured time label (`global: time_label`, or Age (`<time_unit>`)).

See also

- [The omit: section — leaving analyses and figures out](#)
- [Not Applicable and Left Out](#)
- [The Analyze panel and Run analysis](#)
- [Defined Plots](#)

- [Where results are written](#)
- [The Plot Editor](#)

Kaplan-Meier curves

The Kaplan-Meier (KM) figure draws one survival curve per treatment of the Focus. Each curve shows the estimated fraction of individuals still alive at each age. A shaded band shows the 95% confidence interval, and tick marks show where individuals were censored. This is the basic survivorship figure and appears in the Standard Lifespan Plot Set for every Focus.

In the app

- **Plots panel:** the **Kaplan-Meier curves** checkbox (plot id `km_curves`). Tick it and click **Generate plots**, or let **Run analysis** draw it.
- **Scripts:** the **KM curves** action draws the same figure. You can pick treatments, turn the CI band off, or switch to the at-risk-table version.
- **Plot Editor:** `km_curves` is the figure to curate for publication. There the at-risk counts are a Style option, so this one Spec stands in for both KM figures. See [Publication Figures, Specs and Styles](#).

What is drawn

- **x axis:** age, from 0, labelled with your configured time label: `global: time_label`, or `Age (<time_unit>)` (for example "Age (days)"). The numbers are in your data's own unit.
- **y axis:** "Survival Probability", from 0 to 1.
- **One step curve per treatment**, starting at $S(0) = 1$ and dropping at every age where deaths were recorded.
- **95% CI band:** a shaded staircase around each curve.
- **Censor ticks:** a short vertical mark "|" on the curve at every age where at least one individual was censored.
- **Legend:** the treatments in the **Focus's order**, labelled with its display names. Each curve takes the colour the Focus gives it (its `colours:`, set in the Focus window or the Plot Editor). A treatment without one takes the fixed 12-colour cycle by its position in the Focus's order, so it keeps the same colour in every figure of the Focus, Publication Figures and [Defined Plots](#) included.
- **Size and title:** 10 × 6 inches, titled "Kaplan–Meier Survival Curves", with a light grid.

How it is computed

The curves come from the Focus's lifetable (`lifetables_<focus>.csv`), computed separately for each treatment and pooled over chambers:

$$\hat{S}(t) = \prod \text{ over observed times } t_i \leq t \text{ of } (1 - d_i/n_i)$$

where d_i is the number of deaths at age t_i and n_i is the number at risk just before t_i . Every distinct observed age (a death or a censoring) is a row of the lifetable.

The confidence band is the **log-log** ("exponential Greenwood") interval, built from Greenwood's sum on the log(–log) scale:

$$\begin{aligned} \sigma(t) &= \sqrt{(\sum d_i / (n_i (n_i - d_i))) / |\ln \hat{S}(t)|} \\ \text{CI} &= [\hat{S}(t)^{\exp(+1.96 \cdot \sigma)}, \hat{S}(t)^{\exp(-1.96 \cdot \sigma)}] \end{aligned}$$

It always lies inside [0, 1] and is asymmetric where survival is near either end, which is where a plain $\pm 1.96 \cdot \text{SE}$ interval would have to be clipped. It is the interval lifelines reports, computed in

the app's own lifetable code. See [Life tables and the Kaplan-Meier estimator](#) for the full lifetable.

Whether individuals unaccounted for at the final census count as censored depends on the [Censoring policy](#).

Reading it

- The **vertical drop** at an age is the fraction of the remaining cohort that died then.
- The **median lifespan** is the age where a curve crosses 0.5.
- **Curves that separate early and stay apart** suggest a consistent difference in risk. **Curves that cross** suggest the hazards are not proportional; check the [Log-log diagnostic](#).
- **Overlapping bands** are a visual guide only. Use the [log-rank tests](#) to judge differences.
- **Wide bands at old ages** reflect the few individuals still at risk.

Where it appears

- File: `analysis/<focus>/plots/kaplan_meier_<focus>.png` (150 dpi).
- The experiment report's figures section, titled "Kaplan-Meier curves". If `km_curves` is curated in the Plot Editor, the report shows the curated figure instead, and it also replaces the at-risk-table version.

When it is not drawn

This figure has no requirement, so every Focus is offered it, even a single-treatment one. It is missing only when it is unticked (**Left Out**).

See also

- [KM curves with at-risk table](#)
- [Faceted Kaplan-Meier](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [Summary statistics, median and mean survival](#)
- [The Plots panel and Generate plots](#)

KM curves with at-risk table

This figure shows the Kaplan-Meier survival curves with a **number-at-risk table** underneath: for each treatment, how many individuals were still alive and uncensored at a few ages along the time axis. The counts show how many individuals each part of a curve rests on. For Standard Lifespan this is the **Headline Figure** of any Focus that does not cross factors, so it leads the Focus's report section.

In the app

- **Plots panel:** the **KM curves with at-risk table** checkbox (plot id `km_risk_table`). It is the first box in the Plot Set.
- **Scripts:** the **KM curves** action with **With risk table** ticked draws the same figure.
- **Plot Editor:** this figure has no Spec of its own. In a Publication Figure the at-risk counts are a Style option (the **At-Risk Band**) on `km_curves`. A Spec saved under the old id `km_risk_table` is loaded as `km_curves`.

What is drawn

The figure has two stacked panels that share the time axis (12 inches wide, 7 + 0.35 inches per treatment tall).

Upper panel: the KM curves. These are the same estimates as [Kaplan-Meier curves](#):

- step curves starting at $S(0) = 1$, with lines slightly thicker than in the plain KM figure;
- 95% log-log (exponential Greenwood) confidence bands;
- "|" censor ticks;
- legend at upper right; y axis "Survival Probability" from 0 to 1;
- treatments in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).

Lower panel: "Number at Risk".

- One row per treatment, in the same order, labelled at the left with its display name. The numbers are coloured like that treatment's curve.
- Counts are printed at **5 ages: 0, 25, 50, 75 and 100% of the last observed age** of the treatments drawn, evenly spaced in time. These are the same ages the publication [At-Risk Band](#) uses by default.
- The count at age t is the **number at risk at t** : the individuals whose recorded age (death or censoring) is t or later, i.e. alive and uncensored entering age t . It is read from the lifetable: the number at risk of the first observed age at or after t . At age 0 this is the starting group size; after a treatment's last observed age it is 0. The publication band counts the same way, with the same code, so the two always agree.
- The x axis is labelled with your configured time label (for example "Age (days)").

Reading it

- Read a curve together with the count beneath it. A drop late in the experiment, when only a handful are at risk, is a large step for very few deaths.
- Where a treatment's count reaches 0, its curve has no more information. Any later separation between curves is not supported by data.
- Large differences in starting counts between treatments mean their curves are estimated with different precision. The CI bands show this too.

Headline Figure

The Headline Figure is this figure for any Focus except one that varies two or more factors and populates at least two treatments. For those Focuses the [Faceted Kaplan-Meier](#) takes over. The Headline Figure is placed first in the report's figures section and marked "— headline figure".

Where it appears

- File: `analysis/<focus>/plots/km_with_risk_table_<focus>.png` (150 dpi).
- The experiment report's figures section. If `km_curves` is curated in the Plot Editor, the report shows that curated figure in place of **both** KM figures.

When it is not drawn

It has no requirement, so every Focus is offered it. It is missing only when unticked (**Left Out**).

Pitfalls

- A count printed at an age between two observed ages is the number entering the **next** observed age, because everyone who died or was censored at the earlier one is no longer at risk. A column at an age with a death shows the number **before** that death.

See also

- [Kaplan-Meier curves](#)
- [Number at risk](#)
- [Plot Editor — Panels & legend](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [The experiment report](#)

Nelson-Aalen cumulative hazard

The Nelson-Aalen figure draws the **cumulative hazard** $H(t)$ for each treatment: the total risk of death built up from age 0 to age t . The KM curve shows how many survive. This curve shows how much mortality has accumulated, and because it is cumulative it is often easier to read for differences in the **rate** of dying than a KM curve.

In the app

- **Plots panel:** the **Nelson-Aalen cumulative hazard** checkbox (plot id `nelson_aalen`).
- **Scripts:** the **Nelson-Aalen** action (optionally for a chosen list of treatments).
- **Plot Editor:** `nelson_aalen`, with the confidence band and point options available.

What is computed

For each treatment, at every observed age t_i (from the Focus's lifetable):

$$\begin{aligned} H(t) &= \sum \text{ over } t_i \leq t \text{ of } d_i / n_i \\ \text{Var}(H(t)) &= \sum \text{ over } t_i \leq t \text{ of } d_i / n_i^2 \\ 95\% \text{ CI} &= H(t) \pm 1.96 \cdot \sqrt{\text{Var}(H(t))}, \text{ lower bound clipped at } 0 \end{aligned}$$

Here d_i is the number of deaths at t_i and n_i the number at risk just before it. The interval is symmetric on the linear scale. It is computed in the app's own lifetable code (columns `na_H`, `na_se`, `na_ci_lo`, `na_ci_hi` of `lifetables_<focus>.csv`).

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)").
- **y axis:** "Cumulative Hazard $H(t)$ ", from 0. It has no upper limit and can exceed 1.
- **One step curve per treatment**, starting at $H(0) = 0$, with a shaded 95% CI band.
- Legend at upper left. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- Title "Nelson-Aalen Cumulative Hazard", 10 × 6 inches.

Reading it

- The **slope** of a curve is the hazard (the death rate) at that age. A curve that bends upward shows mortality that accelerates with age, which is typical of ageing cohorts.
- A curve that runs **above** another has accumulated more risk. If two curves keep a constant **ratio** (on a log scale, a constant vertical gap), the hazards are proportional.
- $H(t)$ is closely tied to survival: $-\log \hat{S}(t) \approx H(t)$. The two figures tell the same story on different scales.
- The band widens at old ages, where few individuals remain.

Where it appears

- File: `analysis/<focus>/plots/nelson_aalen_<focus>.png` (150 dpi).

- The experiment report's figures section, or the curated version if `nelson_aalen` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Kaplan-Meier curves](#)
- [Smoothed hazard](#)
- [Log-log diagnostic](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [The Plots panel and Generate plots](#)

Log-log diagnostic

The log-log diagnostic is a visual check of the **proportional-hazards (PH)** assumption behind the hazard ratios and the Cox models. For each treatment it plots $\log(-\log \hat{S}(t))$ against $\log(t)$. If the hazards are proportional, the curves are **parallel**, separated by a constant vertical distance.

In the app

- **Plots panel:** the **Log-log diagnostic** checkbox (plot id `log_log`). Its caption reads "Parallel lines support the proportional-hazards assumption."
- **Plot Editor:** `log_log`. The x label defaults to `log(<time label>)`.

What is computed

From each treatment's Kaplan-Meier estimate $\hat{S}(t)$ (the lifetable column `km_lx`):

$$x = \ln(t)$$

$$y = \ln(-\ln \hat{S}(t))$$

Logarithms are natural logs. Only ages where $t > 0$ and $0 < \hat{S}(t) < 1$ are used, because the transform is infinite at $\hat{S} = 1$ (before the first death) and at $\hat{S} = 0$ (after the last). A treatment with fewer than two such points is not drawn.

What is drawn

- **x axis:** `log(<time label>)`, for example "log(Age (days))": the natural log of age in your data's time unit.
- **y axis:** "log(-log $\hat{S}(t)$)".
- One line with small markers per treatment, joining the transformed KM values. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- In the Plot Editor the values are joined with straight lines too, by default (**Line style** `auto`).
- A dashed grey horizontal line at $y = 0$. This is where $\hat{S}(t) = e^{-1} \approx 0.37$.
- Title "Log(-log $\hat{S}(t)$) vs Log(t) — PH Assumption Check", 10 × 6 inches.

Reading it

- **Roughly parallel lines** support proportional hazards: one treatment's hazard is a constant multiple of the other's at every age. The vertical gap is approximately the log hazard ratio.

- **Lines that converge, diverge or cross** suggest the hazard ratio changes with age, for example a treatment that protects early but not late. Hazard ratios and Cox coefficients then describe an average effect. The [log-rank test](#) loses power, and the [Gehan-Wilcoxon test](#) or [RMST](#) may describe the difference better.
- **Straight lines** suggest a Weibull distribution of lifespans. The slope is the Weibull shape parameter. See [Parametric AFT models](#).
- The early part of each line rests on very few deaths and is noisy. Judge parallelism over the bulk of the data.

This is a visual check. For a formal test of PH in a factorial Focus, see [Checking proportional hazards](#).

Where it appears

- File: `analysis/<focus>/plots/log_log_diagnostic_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `log_log` is curated in the Plot Editor.

When it is offered, Not Applicable or Left Out

This figure **requires a comparison between treatments**:

- **Not offered** to a Focus that implies only one treatment. There is nothing to compare, so the box does not appear.
- **Not Applicable** when the Focus implies two or more treatments but fewer than two hold individuals after exclusions. The box is greyed with the reason, and a run records it under "Not applicable to this Focus".
- **Left Out** when unticked.

See also

- [Checking proportional hazards](#)
- [Pairwise hazard ratios](#)
- [Nelson-Aalen cumulative hazard](#)
- [Kaplan-Meier curves](#)
- [Focus Shape — what a Focus is offered](#)

Cumulative deaths

The cumulative-deaths figure shows mortality building up over age for each treatment, as the **cumulative probability of death**, $1 - \hat{S}(t)$. It is the mirror image of the [Kaplan-Meier curves](#): where a KM curve falls from 1, this one rises from 0. It is in the Standard Lifespan Plot Set and offered to every Focus.

In the app

- **Plots panel:** the **Cumulative deaths** checkbox (plot id `cumulative_events`). Its caption reads "Cumulative probability of death, $1 - S(t)$ from the Kaplan-Meier estimate (so censoring-aware), with its 95% band."
- **Plot Editor:** `cumulative_events`, with the confidence band and point options available.

What is computed

For each treatment, from the Kaplan-Meier estimate $\hat{S}(t)$ in the Focus's lifetable (column

km_lx):

$F(t) = 1 - \hat{S}(t)$

band = [1 - upper KM limit, 1 - lower KM limit]

The band is the KM confidence band reflected (see [Life tables and the Kaplan-Meier estimator](#)). Because it is built on the KM estimate, it accounts for censoring: it estimates the fraction of the cohort that would have died by age t . A raw count of deaths would not. It falls short wherever individuals were censored, and it grows with group size, so treatments of different sizes could not be compared on it. The analysis figure and the Publication Figure draw the same quantity.

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)", from global: `time_label` or `time_unit`).
- **y axis:** "Cumulative probability of death", from 0 to 1.
- **One rising step curve per treatment**, starting at 0, with a shaded 95% band.
- Treatments in the Focus's order, with the Focus's display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- Legend at upper left. Title "Cumulative Deaths ($1 - S(t)$)", 10 × 6 inches.

In the Plot Editor the y axis has fixed breaks at 0, 0.25, 0.5, 0.75 and 1, and a new Spec starts with the 0.5 reference line ticked, as on the KM figures.

Reading it

- A steep rise marks ages of heavy mortality.
- The age where a curve crosses 0.5 is the median lifespan.
- The final height is the estimated fraction that died by the last observed age, not a count of deaths.

Where it appears

- File: `analysis/<focus>/plots/cumulative_events_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `cumulative_events` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Kaplan-Meier curves](#)
- [Nelson-Aalen cumulative hazard](#)
- [Publication Figures, Specs and Styles](#)
- [The Plots panel and Generate plots](#)

Hazard rate

The hazard-rate figure plots the raw, unsmoothed **interval hazard** h_x for each treatment: the estimated instantaneous rate of death at each observed age. It is in the Standard Lifespan Plot Set and offered to every Focus. The raw rate is noisy, so the Plot Set also has the [Smoothed hazard](#); this figure shows what the smoothing starts from.

In the app

- **Plots panel:** the **Hazard rate** checkbox (plot id `hazard`). Its caption reads "Raw interval hazard rate, unsmoothed."
- **Scripts:** the **Hazard rate** action (key `hazard_plot`). Left as is, it draws this raw figure. With **Smoothed** ticked it draws the smoothed hazard instead, using its own **Smoothing σ** parameter (range 0.1–20). The figure opens in a tab titled "Hazard rate".
- **Plot Editor:** `hazard`, drawn with straight lines between values by default.

What is computed

The lifetable has one row per distinct observed age (a death or a censoring) for each treatment. At row i :

$$q_i = d_i / n_i \quad \text{(probability of dying at this age)}$$

$$p_i = 1 - q_i$$

$$\Delta t_i = t_{(i+1)} - t_i \quad \text{(gap to the next observed age; for the last row, the gap to the previous one 1 if there is only one row)}$$

$$hx_i = 2 \cdot q_i / ((1 + p_i) \cdot \Delta t_i)$$

This is the actuarial approximation to the hazard over the interval. It is a rate **per unit of your time axis** (per day for a day-scaled census). The "intervals" are the gaps between successive observed ages. In a census design these are usually the census intervals, not fixed bins chosen by the app. The value is the lifetable column `hx` in `lifetables_<focus>.csv`.

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)").
- **y axis:** "Hazard Rate", from 0.
- One line with small dot markers per treatment, joining the per-row `hx` values. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- Title "Hazard Rate Over Time", 10 × 6 inches.

Reading it

- The hazard is the force of mortality. For ageing cohorts it typically rises with age, often roughly exponentially (Gompertz-like).
- Single-row spikes are common. One death among few survivors, or an unusually short gap between censuses, gives a large `hx`. Look at the trend, or use the smoothed figure.
- Late values rest on very few individuals and are unreliable.
- A constant **ratio** between two treatments' hazards is what the proportional-hazards assumption says.

Where it appears

- File: `analysis/<focus>/plots/hazard_rate_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `hazard` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Smoothed hazard](#)

- [Mortality \(qx\)](#)
- [Nelson-Aalen cumulative hazard](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [Script action reference](#)

Smoothed hazard

The smoothed-hazard figure shows how the **death rate** changes with age in each treatment. It takes the raw interval hazard from the lifetable and smooths it with a Gaussian kernel, so the age trend is visible through the row-to-row noise. It is in the Standard Lifespan Plot Set and is offered to every Focus.

In the app

- **Plots panel:** the **Smoothed hazard** checkbox (plot id `smoothed_hazard`). Its caption reads "Kernel-smoothed hazard rate."
- **Scripts:** the **Hazard rate** action with **Smoothed** ticked draws the same figure, with an adjustable **Smoothing σ** .
- **Plot Editor:** `smoothed_hazard`. It uses the same kernel and bandwidth ($\sigma = 3$), so the curated figure shows the same estimate.

What is computed

1. Start from the raw hazard h_x at each observed age, the quantity the [Hazard rate](#) figure draws unsmoothed (see [Hazard rate](#) for the formula: $h_x = 2q / ((1 + p) \cdot \Delta t)$, using the gap Δt to the next observed age).
2. Smooth the sequence of h_x values, in age order, with `scipy.ndimage.gaussian_filter1d` at **$\sigma = 3$** .

σ is measured in lifetable rows, not in time units. $\sigma = 3$ means the kernel's standard deviation spans about three successive observed ages. With a daily census that is about three days. With a census every other day, about six. The filter treats the rows as evenly spaced even if they are not, and handles the ends of the series by reflecting it.

A treatment with **fewer than 5 rows** in its lifetable is not drawn in the analysis figure. The Publication Figure draws such a treatment unsmoothed.

The Plot Set figure and the Publication Figure always use $\sigma = 3$, one shared setting in the code (`plotting.SMOOTHED_HAZARD_SIGMA`). Only the script action lets you change σ .

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)").
- **y axis:** "Smoothed Hazard Rate", from 0. The units are deaths per individual per unit of your time axis.
- One smooth line per treatment, with a faint fill to 0 beneath it. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one). Legend at upper right.
- Title "Smoothed Hazard Rate", 10 × 6 inches.

Reading it

- A rising curve means mortality accelerates with age. In many lifespan cohorts the rise is roughly exponential.
- A treatment that **lowers** the curve at all ages has a proportional benefit. One that

delays the rise shifts the curve to the right.

- Smoothing blurs sharp features: a brief die-off is spread over neighbouring ages, and the curve near the first and last ages is less reliable.
- At the oldest ages few individuals remain, so the curve can swing widely. Read it together with the [Number at risk](#).
- The figure shows no confidence band. Formal comparisons belong to the log-rank tests and hazard ratios.

Where it appears

- File: `analysis/<focus>/plots/smoothed_hazard_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `smoothed_hazard` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Hazard rate](#)
- [Mortality \(qx\)](#)
- [Nelson-Aalen cumulative hazard](#)
- [Life tables and the Kaplan-Meier estimator](#)
- [The Plots panel and Generate plots](#)

Mortality (qx)

The mortality figure plots **qx**, the probability of dying in each interval, against age for every treatment. It is the classic demographic view of a life table. It is in the Standard Lifespan Plot Set and offered to every Focus.

In the app

- **Plots panel:** the **Mortality (qx)** checkbox (plot id `mortality`). Its caption reads "Interval mortality probability."
- **Scripts:** the **Mortality (qx)** action.
- **Plot Editor:** `mortality`, labelled "Mortality (qx)" on the y axis.

What is computed

The lifetable has one row per distinct observed age (an age at which at least one death or censoring was recorded) for each treatment, pooled over chambers. At each row:

$$qx = dx / nx$$

where `dx` is the number of deaths at that age and `nx` is the number alive and uncensored just before it. This is the lifetable column `qx` in `lifetables_<focus>.csv`. In a census design, each row normally corresponds to one census, so `qx` is the fraction of the survivors that died since the previous census.

`qx` is a probability **per interval**, not per unit time. It depends on the gap between observed ages. If censuses are irregular, a longer gap gives a larger `qx` at the same underlying death rate. For a rate per unit time, see the [Hazard rate](#) and [Smoothed hazard](#).

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)").
- **y axis:** "Probability of Death (qx)", from 0.
- One line with small dot markers per treatment, joining the per-row qx values. It is not smoothed. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- Title "Interval Mortality (qx)", 10 × 6 inches.

Reading it

- qx usually rises with age in an ageing cohort.
- **Late points are noisy.** When only a few individuals remain, one death can give $qx = 0.5$ or 1.0. The last row of a treatment in which everyone died has $qx = 1$.
- Rows with censoring but no deaths have $qx = 0$. These pull the line to the axis between deaths, which is common when censoring happens at times other than deaths.
- For a smoother view of the same information, use the [Smoothed hazard](#).

Where it appears

- File: `analysis/<focus>/plots/mortality_qx_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `mortality` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Life tables and the Kaplan-Meier estimator](#)
- [Hazard rate](#)
- [Smoothed hazard](#)
- [Number at risk](#)
- [The Plots panel and Generate plots](#)

Number at risk

This figure plots, for each treatment, the **number of individuals at risk** (alive and not yet censored) at each age. It shows how much data supports every part of the survival curves, and it makes heavy or uneven censoring easy to see. It is in the Standard Lifespan Plot Set and offered to every Focus.

In the app

- **Plots panel:** the **Number at risk** checkbox (plot id `number_at_risk`). Its caption reads "Individuals at risk over time."
- **Plot Editor:** `number_at_risk`, with the y axis labelled "Individuals at risk".

What is computed

For each treatment, the lifetable column `n_at_risk` at each observed age t :

$$n(t) = (\text{group size}) - (\text{deaths and censorings at all earlier observed ages})$$

It counts the individuals present **just before** the events at t . The starting value is the number of individuals in the treatment (after exclusions and the [Censoring policy](#)).

What is drawn

- **x axis:** age from 0, labelled with your configured time label (for example "Age (days)").
- **y axis:** "Number at Risk", from 0. This is a count of individuals.
- One step curve per treatment, flat at the starting group size from age 0 to the first observed age. Treatments are in the Focus's order, with its display names and colours (the fixed colour cycle, in Focus order, for any treatment without one).
- Title "Number at Risk", 10 × 6 inches.

Reading it

- Each downward step is the deaths **plus** the censorings at the previous observed age. A step much larger than the KM drop at the same age means many individuals were censored there (for example, lost or escaped).
- A large drop at the final census usually reflects individuals unaccounted for being treated as censored.
- Curves that start at very different heights mean the treatments had different sample sizes. Their KM curves and tests rest on different amounts of data.
- Where a curve approaches 0, survival estimates for that treatment become unreliable.

Where it appears

- File: `analysis/<focus>/plots/number_at_risk_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `number_at_risk` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [KM curves with at-risk table](#)
- [Kaplan-Meier curves](#)
- [Censoring policy](#)
- [Life tables and the Kaplan-Meier estimator](#)

Lifespan distribution

The lifespan-distribution figure shows the **spread of individual lifespans** in each treatment as violin plots: their shape, middle and range. Survival curves show only the cumulative picture. This figure shows whether deaths cluster at one age, spread widely, or fall into two groups. It is in the Standard Lifespan Plot Set and offered to every Focus.

In the app

- **Plots panel:** the **Lifespan distribution** checkbox (plot id `survival_distribution`). Its caption reads "Distribution of individual lifespans by treatment — recorded deaths only; censored individuals are excluded, since their lifespan is unknown."
- **Plot Editor:** `survival_distribution`. There it is drawn as density curves (see below).

Which individuals are included

Only recorded deaths, in both versions of the figure. A censored individual's lifespan is

unknown: the age it was last seen is a lower bound, not a lifespan. Under the default censoring policy, individuals unaccounted for at the last census are censored at that age, so including them would pile weight at the end of the experiment. With heavy censoring the figure therefore describes the individuals whose deaths were seen, which may be the shorter-lived part of the cohort. For censoring-aware medians, see [Summary statistics, median and mean survival](#).

What is drawn (analysis figure)

- **One violin per treatment**, side by side, in the Focus's order and labelled with its display names. Each violin is filled with that treatment's colour at 70% opacity: the Focus's colour, or the fixed colour cycle in Focus order.
- Each violin shows a kernel density of the treatment's death ages (matplotlib's `violinplot`), with black bars marking the **median**, the minimum and the maximum. The median is the plain median of the death ages, **not** the Kaplan-Meier median.
- If any treatment has one recorded death or fewer, the figure falls back to **box plots** for all treatments (box = quartiles, line = median, whiskers and outlier points in matplotlib's default style).
- **x axis**: "Treatment", with labels slightly rotated.
- **y axis**: `Lifespan - <time label>`, for example "Lifespan — Age (days)".
- Title "Lifespan Distribution". The width grows with the number of treatments.

The Publication Figure version

In the Plot Editor and in rendered Publication Figures, `survival_distribution` is drawn as one smoothed **density curve** per treatment (plotnine `geom_density`), overlaid on one axis, with the area filled at the Style's **Band opacity**. The **x axis** is lifespan and the **y axis** "Density". It uses the same individuals as the analysis figure: recorded deaths only.

Reading it

- A **narrow, tall** violin means lifespans are tightly clustered. A **long, thin** one means a few individuals lived much longer or died much earlier than the rest.
- **Two bulges** in one violin suggest a mixed population, such as an early die-off followed by normal ageing.
- Compare medians by eye, but test differences with the [log-rank tests](#). For censoring-aware medians, see [Summary statistics, median and mean survival](#).

Where it appears

- File: `analysis/<focus>/plots/survival_distribution_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `survival_distribution` is curated in the Plot Editor.

When it is not drawn

It has no requirement. Every Focus is offered it, and it is missing only when unticked (**Left Out**).

See also

- [Summary statistics, median and mean survival](#)
- [Kaplan-Meier curves](#)
- [Censoring policy](#)
- [Lifespan interaction plot](#)

Hazard-ratio forest

The forest plot shows every **pairwise hazard ratio** of the Focus on one axis, each with its 95% confidence interval. You can see at a glance which pairs of treatments differ in their risk of death, in which direction and by how much. It draws the table produced by the **Pairwise hazard ratios** analysis.

In the app

- **Plots panel:** the **Hazard-ratio forest** checkbox (plot id `hazard_ratio_forest`).
- It **draws from Pairwise hazard ratios**. If that box is unticked on the Analyze panel, the forest box is greyed with the tooltip "Draws from Pairwise hazard ratios — tick it on the Analyze panel to draw this figure". Runs then record the forest as **Left Out** ("draws from Pairwise hazard ratios, which is unticked").
- **Generate plots** computes the hazard ratios itself when the forest is ticked, so it does not need a previous run.
- **Scripts:** the **Hazard-ratio forest** action (`forest_plot`).
- **Plot Editor:** `hazard_ratio_forest`. It reads the saved `hazard_ratios_<focus>.csv` from the last run and never computes ratios itself, so run the analysis first.

What is computed

For every pair of treatments, taken in the Focus's display order (group1 is the one listed earlier), the app estimates the hazard ratio by the **log-rank observed/expected method**. At each observed age, the expected deaths in each group are its share of the at-risk total times the deaths in both groups. Summed over ages:

$$\begin{aligned} \text{HR} &= (O_1 / E_1) / (O_2 / E_2) && \text{group1 is the numerator} \\ \text{SE}(\log \text{HR}) &= \sqrt{1/E_1 + 1/E_2} \\ 95\% \text{ CI} &= \exp(\log \text{HR} \pm 1.96 \cdot \text{SE}) \end{aligned}$$

O is the observed number of deaths and E the expected number under equal hazards. A pair where the estimate cannot be formed (for example, no deaths in group2) has no ratio and is left off the plot. The table is saved as `statistics/hazard_ratios_<focus>.csv` (columns include `group1`, `group2`, `hazard_ratio`, `hr_ci_lo`, `hr_ci_hi`). See [Pairwise hazard ratios](#).

What is drawn (analysis figure)

- One row per pair, labelled "group1 vs group2" at the left with the Focus's display names, in table order from top to bottom.
- A **diamond** at the hazard ratio and a horizontal line spanning its 95% CI.
- **Red** where $\text{HR} > 1$ (group1 has the higher hazard) and **blue** where $\text{HR} \leq 1$.
- The numbers "HR (lower–upper)" to three decimals at the right of each row.
- A dashed vertical line at **HR = 1** (no difference).
- **x axis:** "Hazard Ratio (95% CI)", on a **linear** scale starting at 0.
- Title "Hazard Ratio Forest Plot". The height grows with the number of pairs.

The Publication Figure version uses a **log₁₀ ratio axis**, so 0.5 and 2 sit the same distance from 1. Its rows carry the same display-name labels; it colours each comparison from the Style's cycle and drops any row whose ratio or lower limit is not positive. A new Spec's x label is "Hazard ratio (log scale)".

Reading it

- **HR > 1:** group1 dies at a higher rate than group2 (shorter-lived). **HR < 1:** group1 dies at a lower rate (longer-lived). HR = 2 means twice the risk of death at any age, if hazards are proportional.
- An interval that **does not cross 1** suggests a difference at about the 5% level for that pair, with no correction for multiple comparisons. For p-values, see the [log-rank pairwise table](#) and [Reading p-values and multiple comparisons](#).
- On the linear axis, ratios below 1 are compressed between 0 and 1. A ratio of 0.5 is as large an effect as 2.0.
- The ratio summarises the whole lifespan as one number. If the [Log-log diagnostic](#) shows crossing lines, the ratio averages effects that change with age.

Where it appears

- File: `analysis/<focus>/plots/hazard_ratio_forest_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `hazard_ratio_forest` is curated in the Plot Editor.

When it is offered, Not Applicable or Left Out

- **Not offered** to a Focus that implies only one treatment (it requires a comparison).
- **Not Applicable** when fewer than two of the Focus's treatments hold individuals after exclusions, or when the hazard-ratio table comes out empty ("fewer than two treatments to compare").
- **Left Out** when its own box is unticked, or when **Pairwise hazard ratios** is unticked.

See also

- [Pairwise hazard ratios](#)
- [Log-log diagnostic](#)
- [Reading p-values and multiple comparisons](#)
- [Not Applicable and Left Out](#)
- [The Plots panel and Generate plots](#)

Faceted Kaplan-Meier

The faceted Kaplan-Meier figure splits the survival curves into side-by-side panels: **one panel per level of the first varying factor**, with a curve in each panel for every level (or combination of levels) of the other factors. If one factor's effect differs between panels, that difference is the **interaction**, visible without reading a coefficient. For a Focus that crosses factors, this is the **Headline Figure**.

In the app

- **Plots panel:** the **Faceted Kaplan-Meier** checkbox (plot id `km_faceted`). It appears only when the Active Focus varies two or more factors.
- **Scripts:** the **Faceted KM** action, with a **Show 95% CI** option (off by default).
- **Plot Editor:** `km_faceted`. Its **Facet by** combo chooses which varying factor becomes the panels. See [Plot Editor — Figure](#).

What is drawn

Suppose the Focus varies Sex (Female, Male) then Diet (1x, 20x). Its treatment labels are Female/1x, Female/20x and so on.

- **Panels:** one per level of the **first** varying factor in the Focus's order, here "Sex: Female" and "Sex: Male". Panels appear in the Focus's level order, and a level with no individuals gets no panel.
- **Curves:** in each panel, one KM step curve per level of the remaining factor(s), here 1x and 20x. With three or more factors, the curves are the combinations of the remaining levels joined by "/". A cell with no individuals is simply a missing curve.
- **Colours:** each curve level keeps the same colour in every panel, so 20x is the same colour under Female and Male. It is the colour the Focus gives that level (set it on the faceted figure in the Plot Editor's Colours card), otherwise the fixed cycle in the Focus's level order. The legend shows the Focus's display name for the level where it has one.
- **Censor ticks** ("|") at ages with censoring. **No CI bands** in the Plot Set figure. The script action can add them.
- **Axes:** every panel shares the y axis "Survival probability" (0 to 1). Each x axis uses your configured time label (for example "Age (days)").
- **Legend:** on the right-hand panel, titled with the curve factor(s), e.g. "Diet".
- **Title:** `Survival by treatment, faceted by <first factor>`. Each panel is about 5.5 × 5 inches.

The curves are the same Kaplan-Meier estimates as the [Kaplan-Meier curves](#) figure, computed per treatment from the Focus's lifetable.

Reading it

- Compare the **gap between curves within each panel**. If 20x lies below 1x by about the same amount in both panels, Diet acts the same way in both sexes (no interaction). If the gap is large in one panel and absent or reversed in the other, the effect of Diet depends on Sex, which is an interaction.
- Compare the **same-coloured curve across panels** to see the effect of the panel factor at each level of the other.
- The formal tests of these patterns are the [Cox factorial model](#) and the [RMST factorial model](#), relative to each factor's [Reference Level](#).

Headline Figure

For a Focus that varies two or more factors and populates two or more treatments, this figure is the Headline Figure. It is placed first in the report's figures section and is the default figure the Plot Editor opens on.

Where it appears

- File: `analysis/<focus>/plots/km_faceted_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `km_faceted` is curated in the Plot Editor.

When it is offered, Not Applicable or Left Out

It requires a **figure crossing factors**:

- **Not offered** when the Focus varies fewer than two factors. It has no checkbox, and the Plots panel's "Not offered" line names it.
- **Not Applicable** when the Focus varies two or more factors but fewer than two of its treatments hold individuals after exclusions.
- Unlike the Factorial Battery's models, it does **not** need every combination of levels to be

present. A missing cell is a missing curve.

- **Left Out** when unticked.

See also

- [Lifespan interaction plot](#)
- [Interaction analyses \(Factorial Battery\)](#)
- [Focus Shape — what a Focus is offered](#)
- [Kaplan-Meier curves](#)
- [Plot Editor — Figure](#)

Lifespan interaction plot

The lifespan interaction plot is the classic two-way "interaction plot" for lifespan. It shows the **median lifespan** of every cell of the Focus's crossing, with one factor on the x axis and one line per level of the other. **Parallel lines** mean the factors act independently. **Non-parallel or crossing lines** mean they interact. It is offered whenever the Focus varies two or more factors.

In the app

- **Plots panel:** the **Lifespan interaction plot** checkbox (plot id `interaction_lifespan`). It appears only for a Focus varying two or more factors.
- **Scripts:** the **Lifespan interaction plot** action, with a **Metric** choice: `median` (default) or `mean`.
- **Plot Editor:** `interaction_lifespan`. It computes the same values as the analysis figure, the same way round.

What is drawn

Suppose the Focus varies `Sex` (Female, Male) then `Diet` (1x, 20x).

- **x axis:** the levels of the **first** varying factor, here `Sex`: Female, Male, in the Focus's level order. This is the same factor that makes the panels of the [Faceted Kaplan-Meier](#), so the two figures read the same way round. The axis title is the factor name.
- **Lines:** one per level of the other factor(s), here 1x and 20x. With three or more factors, one line per combination of the other factors' levels, joined by "/". The legend is titled with those factor names and shows the Focus's display names. Line colours are the Focus's colours where it sets them, otherwise the fixed colour cycle in level order.
- **Points:** for each cell, the cell's lifespan statistic, with its 95% confidence interval as error bars, and lines joining a line's points. A cell with no individuals is skipped.
- **y axis:** `Median lifespan — <time label>`, e.g. "Median lifespan — Age (days)".
- Title "Lifespan interaction", 7 × 5 inches.

How the values are computed

For each cell (one combination of levels), from the individual-level data, **censored individuals included as censored**:

- **Median** (the Plot Set figure, and the default): the **Kaplan-Meier median**, the first age at which the cell's survival falls to 0.5 or below. Its 95% interval is read off the KM curve's own confidence band where it crosses 0.5 (lifelines' exponential-Greenwood band, the way lifelines reports the CI of a median). A cell whose survival never falls to 0.5 has **no median**, so it gets no point. It is never replaced by the plain median of the recorded times, which would count every censored individual as dead.

- **Mean** (script option only): the **restricted mean lifespan** (RMST), the area under the cell's KM curve from 0 to a common τ . τ is the earliest of the cells' last observed ages, so no cell is extrapolated, and it is printed in the y-axis title, e.g. "Restricted mean lifespan ($\tau = 62$) — Age (days)". Its interval is $\pm 1.96 \times$ the standard error of the RMST estimate.
- **Open limits:** when the KM band never falls to 0.5, the upper limit of the median's interval is not reached. That side gets no error bar, rather than one running off the axis or stopping at an invented value.

The intervals describe each cell's own uncertainty. They are a visual guide, not a test: the formal test of interaction is the model, below.

Reading it

- **Parallel lines:** moving from Female to Male changes median lifespan by the same amount at 1x and 20x. There is no interaction on this scale.
- **Lines that converge, diverge or cross:** the Sex effect depends on Diet. Crossing lines mean the direction of the effect reverses.
- Medians summarise the middle of each survival curve. Two cells can share a median yet differ early or late. Check the [Faceted Kaplan-Meier](#).
- The formal test of interaction is the likelihood-ratio test in the [Cox factorial model](#). The [RMST factorial model](#) tests interaction on the restricted-mean scale without assuming proportional hazards.

The Publication Figure version

The Plot Editor and rendered Publication Figures draw the same median and interval for each cell, by the same code, with the first varying factor on x and the lines in the Focus's order. Only the median is offered there. A new Spec's x label is the first factor's name and its legend title the other factors' names; the y label is `Median lifespan — <time label>`. Only the y axis can be pinned.

Where it appears

- File: `analysis/<focus>/plots/interaction_lifespan_<focus>.png` (150 dpi).
- The experiment report's figures section, or the curated version if `interaction_lifespan` is curated in the Plot Editor.

When it is offered, Not Applicable or Left Out

It requires a **figure crossing factors**:

- **Not offered** to a Focus varying fewer than two factors.
- **Not Applicable** when fewer than two of the Focus's treatments hold individuals after exclusions.
- It does **not** need a full crossing. An empty cell is just a missing point.
- **Left Out** when unticked.

See also

- [Faceted Kaplan-Meier](#)
- [The Cox factorial model](#)
- [The RMST factorial model](#)
- [Interaction analyses \(Factorial Battery\)](#)

- [Focus Shape](#) — what a Focus is offered

The Plot Editor

The Plot Editor is where you design **Publication Figures**: journal-ready, vector versions of the figures in the Plot Set, drawn with `plotnine` (a `ggplot`-style library). On the left you edit one figure's content (its **Plot Spec**) and its look (its **Plot Style**). On the right a live preview redraws as you change things. What you save becomes the Project's curated figure, which **Render publication figures** writes out for every member and the reports show in place of the default figure.

The editor changes presentation only. It never reruns an analysis or changes which treatments a Focus contains. The one thing it writes outside `plot_specs.yaml` is the Active Focus's per-curve colours, into `survival_config.yaml`, because those belong to the Focus.

Opening it

The editor is opened from the **Project** panel's actions card, with the **Plot editor...** button. It needs one Member Experiment whose data it previews. It picks, in order:

1. the member selected in the members table;
2. otherwise the loaded experiment;
3. otherwise the Project's only member;
4. otherwise it asks which member to use.

The preview draws that member's **Active Focus**, and the window title shows both: `Plot Editor - <member> · Focus <name>`. Press **F1** in the editor to open this page.

The window

Top bar (right-hand side):

- **Save Project default** writes the **current figure's** Spec and its own Style into the container's `plot_specs.yaml`. It also saves any per-curve colours you changed into the Active Focus's definition in `survival_config.yaml`. The status bar confirms the save. Only figures saved this way are rendered and used by the reports.
- **Export SVG** writes the current figure, as previewed, to `<container>/figures/<plot_id>_<focus>.svg`, with text kept editable. There is no dialog. If the name is taken, `_1`, `_2`... is appended rather than overwriting, so earlier exports are kept for comparison.
- **?** opens [Publication Figures, Specs and Styles](#).

The **container** is the Project directory. For a standalone experiment with no Project, it is the experiment's own directory.

Left column, five cards from top to bottom:

Card	What it holds	Saved as
Figure	which figure; title, axis labels, legend title, facet factor, axis limits, reference line; <i>Copy style from...</i>	the Spec
Canvas & type	size in mm, theme, font, text sizes and colour	the Style
Curves & points	curve and axis weights, line join, markers, censor ticks, CI band	the Style

Panels & legend	panel fill and border, gridlines, facet strips, legend position, At-Risk Band	the Style
Colours	a swatch per curve, the fallback cycle, Refresh preview	Focus colours and the Style's cycle

Controls that mean nothing for the current figure are **disabled** rather than silently ignored. For example, censor ticks are disabled on a hazard curve, and the At-Risk Band on anything but a KM figure. Controls a figure does use stay enabled even where the name suggests otherwise: **Band opacity** is the distribution's density fill, and the point shape, size, fill and outline style the forest and interaction markers. Scrolling the mouse wheel over the left column scrolls it and never changes a spin box or combo.

A colour swatch showing an automatic value (**curve, theme, cycle**) can be returned to it after a colour has been picked: **right-click** the swatch and choose **Use curve** (or **Use theme, Use cycle...**).

Right side: the preview. It is rendered at the screen's full resolution for the figure's size in millimetres, and re-rendered (not stretched) when you resize the window. If a figure cannot be drawn, the preview says why instead, for example "No data for hazard_ratio_forest under this Focus." (a forest needs the saved pairwise hazard ratios, the log-rank observed/expected estimates) or "Preview failed: ...".

Which figures it offers

The **Figure** combo lists:

- every figure already saved in `plot_specs.yaml`; and
- a default for each figure in the Active Focus's Plot Set that is relevant to the Focus's definition: `km_curves`, `cumulative_events`, `survival_distribution`, `mortality`, `hazard`, `smoothed_hazard`, `nelson_aalen`, `number_at_risk`, `hazard_ratio_forest` and `log_log` (the last two only with two or more treatments), plus `km_faceted` and `interaction_lifespan` when the Focus varies two or more factors.

There is no separate `km_risk_table` entry, because in a Publication Figure the at-risk counts are a Style option of `km_curves`. The editor opens on the **Headline Figure**: `km_faceted` when it is offered, otherwise `km_curves` (the publication form of the KM with at-risk table).

Where the data comes from

The preview uses the Active Focus's **saved results** in `analysis/<focus>/`:

- series figures read `lifetables_<focus>.csv`, or compute the lifetables from the data if there is none;
- the distribution and interaction figures read `individual_data_<focus>.csv`, or load the data;
- the forest reads `hazard_ratios_<focus>.csv` only. The editor never fits a model, so run the analysis (with Pairwise hazard ratios ticked) before curating the forest.

Curves follow the Focus's treatment order and display names. A Spec cannot reorder or rename them (see [Publication Figures, Specs and Styles](#)).

A typical session

1. Run the analysis for the Focus you want to show.
2. Open the Plot Editor from the Project panel with that member selected.

3. Choose a figure, set its labels, size, fonts and colours, and watch the preview.
4. Click **Save Project default**. Repeat for each figure you want curated.
5. Back on the Project panel, choose a format and click **Render publication figures**.

See also

- [Publication Figures, Specs and Styles](#)
- [Plot Editor — Figure](#)
- [Plot Editor — Colours](#)
- [Project actions — reports, figures, Plot Editor](#)
- [The Plots panel and Generate plots](#)

Plot Editor — Figure

The **Figure** card is the top card of the Plot Editor. It chooses which figure you are editing and holds that figure's **content decisions**: the **Plot Spec**. Everything on this card except *Copy style from...* is saved in the figure's entry under `plots:` in the container's `plot_specs.yaml` when you click **Save Project default**.

Text fields update the preview when you press Enter or leave the field. Every other control updates it at once.

Fields

Figure: the plot id of the figure being edited (`km_curves`, `mortality`, `km_faceted`, ...). Switching loads that figure's Spec and Style into all the cards and disables the controls that do not apply to it. Unsaved edits to the previous figure stay in the editor's working copy until you close it, but they are written to the file only by **Save Project default** while that figure is selected.

Title: the figure title. It is empty by default, and an empty title draws no title, which suits a journal figure whose caption carries it.

X label: the x-axis title. The default follows what the axis shows. Where it is age, it is your configured time label, e.g. "Age (days)" from `global: time_label` (also the distribution, whose x axis is lifespan). For the log-log diagnostic it is "log(Age (days))". For the forest plot it is "Hazard ratio (log scale)", which is also what a cleared field gives. For the interaction plot it is the name of the first varying factor, whose levels are on that axis.

Y label: the y-axis title. The default is set per figure: "Survival probability" (KM and faceted KM), "Cumulative probability of death" (cumulative deaths), "Cumulative hazard" (Nelson-Aalen), "Individuals at risk", "Mortality (qx)", "Hazard rate", "Smoothed hazard", "log(-log S(t))", "Density" (distribution), "Comparison" (forest) or `Median lifespan - <time label>` (interaction). An empty field falls back to the figure's own label.

Legend title: the title over the curve legend. The default is "Treatment". For the faceted KM it is the second varying factor, because there the curves are that factor's levels, not whole treatments; for the interaction plot, the factor(s) after the first, whose levels make the lines. The forest plot shows no legend. Legend entries follow the Focus's order and show its display names.

Facet by: a list of the Active Focus's **varying factors**, plus "(none)". The factor you pick becomes the **panels**, one per level. The other factor(s) become the curves within each panel. It is enabled only for `km_faceted`, and it defaults to the first varying factor. With "(none)" the faceted KM draws as one panel of whole-treatment curves.

X limits: a checkbox and two numbers (low – high, two decimals). Unchecked, the axis fits the

data. Checked, the axis is pinned to that range, which you need when two figures must sit side by side on the same scale. The two numbers are sorted when saved, so typing them the wrong way round still works. The units are those of the x axis: your time unit, or natural-log time for the log-log diagnostic. On the forest plot the limits are ratios and must both be greater than 0, otherwise they are ignored. On the interaction plot x is a list of levels and cannot be pinned.

Y limits: the same, for the y axis (three decimals; default range 0 – 1 when first ticked). Units are those of the y axis: probability, cumulative hazard, count, density or lifespan. On the forest plot y is the list of comparisons and cannot be pinned. On a KM figure, pinning y replaces the room the editor otherwise reserves below 0 for the At-Risk Band, so set the lower limit below 0 (e.g. –0.3) if you want the band to stay visible.

Reference line: a checkbox and a value (three decimals). Checked, a dashed grey horizontal line is drawn at that y value. The default is **checked at 0.5** for the KM figures and cumulative deaths, marking median survival, and unchecked elsewhere. Negative values are allowed, which is useful on the log-log diagnostic. It is drawn on the series figures (KM, cumulative deaths, Nelson-Aalen, hazard, mortality, number at risk, log-log) but not on the forest, distribution or interaction plots.

Style: Copy style from... opens a list of every other figure's working style ("mortality (figure)") and every named style already in `plot_specs.yaml` ("km_curves (saved style)"). The one you pick is copied over the current figure's **whole Style**: all of Canvas & type, Curves & points, Panels & legend and the colour cycle. Each figure owns its style, so this is the deliberate way to make figures match. The copy is not saved until you click **Save Project default**. The ? beside it opens [Publication Figures, Specs and Styles](#).

What the Spec cannot do here

- **Treatment order and display names** come from the Focus. Set them in the [Focus window](#).
- A Spec can **narrow** a figure to some of the Focus's treatments, but there is no control for it in the editor. Add a `treatments:` list to the figure's entry in `plot_specs.yaml` by hand. Names not in the Focus are ignored, and if none match, all treatments are shown.
- **Facet order** (`facet_order:`) can likewise only be set in the file.

See also

- [The Plot Editor](#)
- [Publication Figures, Specs and Styles](#)
- [Plot Editor — Panels & legend](#)
- [Faceted Kaplan-Meier](#)
- [The Focus window](#)

Plot Editor — Canvas & type

The **Canvas & type** card sets a figure's physical size, its overall theme and all of its text. These are **Plot Style** settings. When you click **Save Project default**, they are saved under the figure's own name in the `styles:` section of the container's `plot_specs.yaml`. Use *Copy style from...* on the Figure card to give another figure the same look.

Every change redraws the preview at once.

Fields

Size (mm): width × height of the finished figure in **millimetres** (whole numbers, steps of 5).

Width ranges from 40 to 400 and height from 30 to 400. The default is **120 × 90 mm**. This is the size the SVG, PDF or PNG is written at, so set it to the column width your journal asks for: typically about 85–90 mm for one column and 170–180 mm for two. Fonts are in points, so they stay the same physical size when you change the figure size.

Theme: the plotnine base theme the rest is laid over:

- `theme_classic` (default): white background with x and y axis lines and no box;
- `theme_bw`: white background with a panel border;
- `theme_minimal`: no axis lines or border;
- `theme_light`: light grey panel border;
- `theme_matplotlib`: matplotlib's look.

The editor switches off the theme's gridlines unless you turn them on under [Panels & legend](#).

Font: the font family for all text, chosen from the fonts installed on this computer. The default is DejaVu Sans. If the chosen font is not installed where the figure is rendered, the first available of Helvetica, Arial, Liberation Sans and DejaVu Sans is used, and the exported SVG names the font actually used.

Base size (pt): the text size every other text size follows (4 – 32 pt in steps of 0.5; default **9 pt**). Change only this and all text scales together.

text: the swatch beside Base size sets **one colour for all text:** title, axis titles, tick labels, legend and facet strips. Default black. Panel borders and boxed facet strips are also outlined in this colour. Axis lines and ticks keep the theme's own colour.

Per-element sizes. Each is 0 – 40 pt in steps of 0.5, and shows **auto** at 0 (the default), meaning "follow the base size":

Field	Applies to	When auto
Title (pt)	the figure title	base + 2 pt
Axis titles (pt)	the x and y axis titles	base
Tick labels (pt)	the numbers and level names on the axes	base
Legend (pt)	legend entries and the legend title	base
Facet strips (pt)	panel titles of a faceted figure	base + 1 pt

Set one of these only for the element that has to differ, such as a long title or cramped tick labels. Leaving the rest on auto keeps the figure scaling as one thing when you change the base size.

The At-Risk Band's numbers have their own size, set under [Panels & legend](#).

See also

- [The Plot Editor](#)
- [Plot Editor — Curves & points](#)
- [Plot Editor — Panels & legend](#)
- [Publication Figures, Specs and Styles](#)

Plot Editor — Curves & points

The **Curves & points** card styles the data itself: the weight of the curves, how points are joined, optional markers, the Kaplan-Meier censor ticks and the confidence band. These are **Plot Style**

settings, saved under the figure's own name in `plot_specs.yaml` by **Save Project default**.

Controls that do not apply to the current figure are greyed out:

Figure	Markers	Censor ticks	CI band	Band opacity	Line style
km_curves, km_faceted	yes	yes	yes	yes	yes
nelson_aalen, cumulative_events	yes	—	yes	yes	yes
mortality, hazard, smoothed_hazard, number_at_risk, log_log	yes	—	—	—	yes
survival_distribution	—	—	—	yes (the density fill)	—
hazard_ratio_forest, interaction_lifespan	shape, size, fill and outline only	—	—	—	—

Fields

Curve width: the thickness of the curves and lines (0.1 – 6.0 in plotnine size units; default **0.9**). On the forest and interaction plots it also sets the interval lines, and on the distribution the density outline.

axes: the weight of the figure's "furniture": axis lines, tick marks, panel borders and boxed strip outlines (0 – 4.0 pt; default **0.8**). It is kept separate from curve width because a heavy curve on light axes is a common journal style.

Line style: how successive values are joined:

- **auto** (default) follows the figure. **Step** for KM, Nelson-Aalen, number at risk and cumulative deaths. **Straight lines** for mortality, hazard, smoothed hazard and the log-log diagnostic (as the analysis figure joins them).
- **step** draws a staircase: each value is held until the next.
- **line** draws straight segments between values.

The CI band follows the same choice, so it always bounds its curve. Not available on the forest, distribution and interaction plots.

Points on the curve

Draw points on the curve: adds a marker at chosen values on every curve. It is **off** by default, because a lifespan cohort has an event at almost every census and a marker per event buries the curve.

Points at: which values get a marker:

- **events** (default): values where the curve actually changed. On a survival curve that means every age where survival fell. The age-0 starting point and censoring-only values

are skipped.

- **censored:** values at ages with at least one censoring.
- **all:** every observed age (except age 0).

Shape / size: the marker shape, one of `o` circle, `s` square, `^` and `v` triangles, `D` diamond, `d` thin diamond, `p` pentagon, `h` hexagon, `*` star, `+`, `x` and `|`. Size ranges from 0.2 to 12 (default **2**). The first nine are *filled* shapes, which can have an outline. On `+`, `x` and `|`, an outline only makes the mark thicker.

Opacity / fill:

- **Opacity:** 0 – 1, default 1 (opaque).
- **Fill swatch:** the marker's interior. **curve** (the default, an empty swatch) uses the curve's own colour. Picking a colour uses that colour. Picking with the dialog's alpha at 0 stores `none`, which gives a **hollow** marker when an outline is set (see below). Partial alpha is stored as an `rrggbbaa` colour. Right-click the swatch and choose **Use curve** to go back to the curve's colour.

Outline:

- **Weight:** 0 – 3.0, default **0**. At 0 the marker has no separate edge and is drawn in one colour: the fill colour if one is picked, otherwise the curve colour. Above 0, and on a filled shape, the marker has an edge of this weight with the fill inside.
- **Colour swatch:** the edge colour. Default black. Picking alpha 0 gives an invisible edge. Right-click it and choose **Use curve** for an edge in the curve's own colour.

So for a hollow circle, choose shape `o`, outline weight above 0, and a fill picked with alpha 0.

Censor ticks

Kaplan-Meier figures only.

Censor ticks: marks on the curve at every age where at least one individual was censored. **On** by default.

Tick shape/size: the mark, from the same shape list (default `|`, a short vertical tick), size 0.2 – 12 (default **2**), and a colour swatch (default **curve**, the curve's own colour; right-click it to return to **curve**).

Confidence band

KM, cumulative-deaths and Nelson-Aalen figures only.

95% CI bands: shades the 95% confidence interval around each curve. **Off** by default in Publication Figures, although the analysis figures draw it. The intervals are the ones in the lifetable: the log-log (exponential Greenwood) band for KM, reflected for cumulative deaths, and $\pm 1.96 \cdot \sqrt{\Sigma d/n^2}$ for Nelson-Aalen (see [Life tables and the Kaplan-Meier estimator](#)).

Band opacity: 0 – 1, default **0.15**. The band takes each curve's colour.

The [Lifespan distribution](#) uses this same opacity for its density fill, so the control stays enabled for that figure (the **95% CI bands** box does not).

Forest and interaction markers

On the forest and interaction plots the marker **is** the estimate, so it is always drawn. Its **shape, size, fill and outline** follow the point settings above, which stay enabled for those figures, at full opacity and a size of at least 2. **Draw points on the curve**, **Points at** and the point opacity do not apply there and are greyed.

See also

- [The Plot Editor](#)
- [Plot Editor — Canvas & type](#)
- [Plot Editor — Panels & legend](#)
- [Plot Editor — Colours](#)
- [Kaplan-Meier curves](#)

Plot Editor — Panels & legend

The **Panels & legend** card controls everything around the curves: the panel background and border, gridlines, the title strips of a faceted figure, where the legend sits, and the **At-Risk Band** of number-at-risk counts under a Kaplan-Meier figure. These are **Plot Style** settings, saved under the figure's own name in `plot_specs.yaml` by **Save Project default**.

Fields

Panel:

- **Swatch:** the plotting area's background. **theme** (the default, an empty swatch) keeps the theme's own fill. Pick a colour to override it.
- **Border:** draws a box around each panel in the text colour, at the **axes** weight from [Curves & points](#). Off by default. A boxed facet strip turns the border on regardless.

Gridlines: `none` (default), `y` or `both`.

- `none` removes all gridlines.
- `y` draws light grey horizontal lines at the y-axis breaks, and no vertical ones.
- `both` draws light grey lines at the breaks of both axes.

Minor gridlines are always off. The lines are drawn whatever the theme, including the default `theme_classic`, which has none of its own. With the At-Risk Band on, `both` keeps the vertical lines out of the band, so they do not read as table rulings. The default is `none` because a survivorship curve is read against its own steps, and rules at 0.25, 0.5 and 0.75 compete with them.

Facet strips: the panel titles of a faceted figure (`km_faceted`, or a distribution given a `facet_by`: by hand in `plot_specs.yaml`).

- `plain` (default): the level name as bare text.
- `boxed`: the level name in a filled, outlined box, ggplot style. The panels get a border too, so the strip does not look detached.
- **Swatch:** the box fill for `boxed` (default light grey `#d9d9d9`). Not used with `plain`.

Legend: where the curve legend goes: `right` (default), `left`, `top`, `bottom` or `none` (no legend). The forest plot never shows a legend.

The At-Risk Band

Kaplan-Meier figures only (`km_curves`, `km_faceted`). The band is the Publication Figure's version of the number-at-risk table. The counts are drawn as text **inside the same plot**, in space reserved below the curves, so the figure stays one object for faceting and vector export.

At-risk band below the curves: turns the band on. **On** by default, so a curated `km_curves` is "KM with at-risk counts" unless you untick it.

Band size:

- **Font size:** 3 – 20 (default **7**), the size of the count numbers.
- **row:** the vertical space given to each curve's row of counts, 0.02 – 0.4 (default **0.075**), in **y-axis units**. On a survival axis running 0 to 1, 0.075 is 7.5% of the axis height. The first curve's counts sit at –0.075, the second at –0.15 and so on, and the y axis is extended down to make room. The band has no axis labels. Each row's numbers are coloured like its curve, which identifies the row.

Band times: the ages the counts are printed at, as numbers separated by commas or spaces in your time unit (e.g. 0, 20, 40, 60). Anything that is not a number is ignored. **Empty** (the default) prints at five evenly spaced ages: 0%, 25%, 50%, 75% and 100% of the last observed age. Pin the times when figures must line up side by side.

What the count means: for each curve and each band time t , the **number at risk at t** : the individuals whose recorded age (death or censoring) is t or later, i.e. alive and uncensored entering age t . It is the number at risk of the first observed age at or after t ; at age 0 it is the starting group size, and after a curve's last observed age it is 0. In a faceted figure each panel gets its own rows. The counts are read from the same data as the curve above them, so the two cannot disagree, and by the same code as the analysis figure's [at-risk table](#), at the same default ages, so the two print the same numbers.

Pitfall: if you pin **Y limits** on the Figure card, the pinned range replaces the room reserved for the band. Set the lower limit below 0 (for two treatments at the default row height, about –0.2) to keep it visible.

See also

- [The Plot Editor](#)
- [KM curves with at-risk table](#)
- [Plot Editor — Canvas & type](#)
- [Plot Editor — Figure](#)
- [Publication Figures, Specs and Styles](#)

Plot Editor — Colours

The **Colours** card sets the colour of each curve in the current figure and the **fallback cycle** that any curve without its own colour takes. Per-curve colours belong to the **Focus**, not to the figure, so every Publication Figure of that Focus agrees: Females are the same red on the KM curves and on the hazard. The cycle belongs to the figure's Style.

Fields

One swatch per curve. The card lists every curve the current figure will draw, in the order it draws them, each labelled with its display name. The list is rebuilt when you switch figure, because the curves depend on the data and the figure:

- for most figures, one swatch per **treatment** of the Focus;
- for the faceted KM, one per **curve level** (the level, or combination of levels, of the non-panel factors), since the same curve appears in every panel;
- for the interaction plot, one per **line**; for the forest, one per **comparison** ("A vs B").

Click a swatch to pick a colour. The colour dialog includes an alpha (transparency) slider. Partial alpha stores a semi-transparent colour, and alpha 0 stores `none`, which makes the curve invisible.

Each swatch first shows, in order of priority:

1. the colour the Focus already assigns to that curve;
2. a colour from an older Style's own per-label palette, if there is one and the Focus names none;
3. otherwise, the next colour of the fallback cycle by curve position.

What gets saved. Only a swatch you changed **away from its cycle colour** becomes an assignment. A swatch still showing its cycle colour is not saved, so opening the editor and touching something does not pin every curve's colour. When you click **Save Project default**, changed assignments are written into the Active Focus's definition (its `colours:`) in `survival_config.yaml`, and the status bar adds `Curve colours saved to Focus <name>`. The same colours can be seen and edited in the Colour column of the [Focus window](#).

Because colours are keyed by the Focus's own treatment labels, a colour set under one Focus does not carry over to another, even for the same individuals.

Fallback cycle. A row of small swatches, the colours in the order unassigned curves take them. The first curve takes the first colour, the second the second, and so on, wrapping round. The default is an 8-colour, colour-blind-safe palette (`#4C72B0`, `#DD8452`, `#55A868`, `#C44E52`, `#8172B3`, `#937860`, `#DA8BC3`, `#8C8C8C`). Click any swatch to change it. The cycle is part of this figure's Style (`palette_cycle`), saved with the figure and copied by *Copy style from....* A journal palette can live here without naming any treatment in advance.

To return **one** curve to the cycle, right-click its swatch and choose **Use cycle**.

Reset curve colours to the cycle. Clears the per-curve assignments for the curves in this figure, so they fall back to the cycle. The cycle itself is unchanged. The reset is written to the Focus only when you next click **Save Project default**.

Refresh preview. Redraws the preview. Changes normally redraw it automatically; use this after editing `plot_specs.yaml` or the Focus elsewhere.

Notes

- The per-curve colours are the **Focus's**, so they apply to every figure of it: the Publication Figures (the preview, Export SVG, Render publication figures, curated figures in the reports) and the analysis PNGs from Run analysis and Generate plots. Only the fallback differs: a Publication Figure takes this card's cycle, an analysis PNG a fixed cycle, each in the Focus's order.
- Display names (the labels beside the swatches and in legends) also come from the Focus. Change them in the Focus window, not here.

See also

- [The Plot Editor](#)
- [The Focus window](#)
- [Plot Editor — Figure](#)
- [Publication Figures, Specs and Styles](#)

Publication Figures, Specs and Styles

A **Publication Figure** is a hand-curated, journal-ready version of a figure from the Plot Set. It is drawn with **plotnine** (a ggplot-style library) and written as SVG or PDF with editable text, or as PNG. It is separate from the matplotlib PNGs that **Run analysis** and **Generate plots** produce. You **author** Publication Figures in the [Plot Editor](#) and **render** them from the Project panel.

Each Publication Figure is defined by two things:

- a **Plot Spec**: its content decisions. These are the title, axis labels, legend title, facet factor, axis limits, reference line, and optionally a narrowing to some of the Focus's treatments;
- a **Plot Style**: its look. This covers size in mm, theme, fonts and text sizes, curve and axis weights, markers, censor ticks, confidence band, panels, gridlines, legend, the At-Risk Band and the colour cycle.

Who decides what

Decision	Lives in	Set in
Which treatments exist, their order	the Focus	the Focus window
Display names and per-curve colours	the Focus (<code>display_names:</code> , <code>colours:</code> in <code>survival_config.yaml</code>)	the Focus window, or the Plot Editor's Colours card
Labels, limits, reference line, facet factor, narrowing	the Spec	the Plot Editor's Figure card
Everything about the look	the Style	the Plot Editor's style cards

A Spec can **narrow** a figure to a subset of the Focus's treatments, but never reorder or rename them. Curves always follow the Focus's order, so a legend never disagrees with the model printed beside it.

plot_specs.yaml

All Specs and Styles live in **one file at the container**: `plot_specs.yaml` in the Project directory, or in the experiment's own directory for a standalone experiment. Every member of a Project renders with the same curation. A shared Spec still works for a member whose treatments differ: names it does not have are ignored, and its curves follow its own Focus.

The file has three keys:

```
default_style: default          # the Style a figure's first edit is seeded from
styles:
  default: {}                  # built-in values unless overridden
  km_curves:                   # each saved figure owns a Style of its own name
    width_mm: 85.0
    height_mm: 65.0
    theme: theme_classic
    font_family: Arial
    base_size: 8.0
    line_width: 0.9
    censor_ticks: true
    ci_band: false
    risk_table: true
    risk_table_times: [0, 20, 40, 60]
    palette_cycle: ['#4C72B0', '#DD8452', '#55A868', '#C44E52']
    # ...every other Style field
plots:
  km_curves:
    style: km_curves
    title: ''
    x_label: Age (days)
    y_label: Survival probability
    series_label: Treatment
```

```
treatments: []          # empty = every treatment of the Focus
facet_by: ''
facet_order: []
x_limits: []           # [low, high] pins the axis
y_limits: []
reference_line: 0.5     # null = no line
```

- **Save Project default** in the Plot Editor writes **one figure at a time**: its Spec under `plots:` and its Style under `styles:`, both under the figure's plot id. Figures you never saved are not in the file.
- Unknown top-level keys are kept when the editor saves, so the file can be edited by hand.
- Valid plot ids under `plots:` are `km_curves`, `km_faceted`, `nelson_aalen`, `number_at_risk`, `cumulative_events`, `mortality`, `hazard`, `smoothed_hazard`, `log_log`, `survival_distribution`, `hazard_ratio_forest` and `interaction_lifespan`. Other ids are ignored. A Spec saved under the retired id `km_risk_table` is read as `km_curves`, because the at-risk counts are a Style option (`risk_table`) there.
- **Legacy member files.** Earlier versions saved Specs in each member's own `plot_specs.yaml`. Those are still read, but only for plot ids the container has no Spec for.

Rendering

Project panel → **Render publication figures**, with the format chosen beside it (svg, pdf or png):

- walks **every member** of the Project. A member is skipped ("nothing curated — skipped") when the file has no saved Specs, and nothing at all is rendered if nothing is curated;
- for each member, renders under **each of its Focuses** whose results are analysed and current. A Focus that is not analysed, or whose results are **Out of Date**, is named in the log and skipped. Run it first;
- renders **only the figures saved in** `plots:`, never defaults for the rest of the Plot Set;
- under each Focus, renders only the saved figures that **Focus is offered**. A saved `km_faceted` or `interaction_lifespan` is skipped under a Focus varying fewer than two factors, and a saved forest or log-log diagnostic under a single-treatment Focus, each with a log line giving the reason (`km_faceted: not offered — Focus 'Females' varies only Diet; a figure crossing factors needs 2 or more varying factors — skipped.`);
- writes each to `<member>/figures/<focus>/<plot id>_<focus>.<format>`, overwriting the previous render;
- names and skips a figure that has no data under a Focus (for example a forest with no saved hazard ratios, or an interaction plot none of whose cells reaches a median), and carries on with the others.

The same rendering is available as the **Render publication figures** script action, at Project level or for one experiment. In an Experiment Script it renders the script's Focus, or every Focus if none is set.

Export SVG in the Plot Editor writes just the current figure to the container's `figures/` folder (`<plot id>_<focus>.svg`, numbered rather than overwritten).

Formats. SVG keeps text as editable text (`svg.fonttype: none`). PDF embeds TrueType fonts (`pdf.fonttype: 42`), so labels can still be retyped in an illustration program. PNG is rasterised at 300 dpi. All are written at the Style's size in millimetres.

Figures are drawn from the Focus's **saved** results in `analysis/<focus>/lifetables`, individual data, and the hazard-ratio table. A Publication Figure never computes a statistic of its own.

In the reports

A curated figure **replaces** the analysis figure in the experiment report and the Project Report. It is drawn through its Spec and Style at 200 dpi and titled `<figure> – curated figure`. Figures with no saved Spec fall back to the default matplotlib PNG. A curated `km_curves` stands in for **both** default KM figures. If a curated figure fails to render, the report shows the default instead.

How these differ from the analysis figures

- Both use the Focus's order, display names and per-curve colours. Where the Focus sets no colour, a Publication Figure takes its Style's cycle and an analysis PNG a fixed cycle, each in the Focus's order.
- The At-Risk Band is text inside the same plot, not a separate table axis. This is a deliberate design choice (ADR-0004): one plot object facets, themes and exports as vectors without stitching two figures together.
- The numbers are the same; some figures are drawn differently. The distribution is a density curve rather than a violin (both over recorded deaths only). The forest uses a log ratio axis. The interaction plot shows the same Kaplan-Meier medians and intervals, the same way round. Cumulative deaths are $1 - S(t)$ in both. See each figure's page.

See also

- [The Plot Editor](#)
- [Plot Editor — Figure](#)
- [Plot Editor — Colours](#)
- [Project actions — reports, figures, Plot Editor](#)
- [The Focus window](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)

Scripts

Experiment and Project Scripts

A script is a saved, named list of steps that the app can re-run at any time. Scripts are how an analysis becomes repeatable: the same steps, in the same order, give the same outputs whether you run them from the Hub, from a Project, or in an overnight Batch Run. There are two levels of script, and they never mix.

Level	Acts on	Saved in	Bridge to the level below
Experiment Script	one Experiment Directory, one Focus at a time	<code>survival_config.yaml</code> → <code>scripts:</code> (or centrally in <code>project.yaml</code> →	<code>run_in_focuses</code> repeats the rest of the script once per Focus

		experiment_scripts:)	
Project Script	a Project and its Member Experiments	project.yaml → scripts:	run_in_experiments runs a named Experiment Script in every member

Each level has its own list of actions (see [Script action reference](#)). An Experiment Script cannot call a project action, and a Project Script cannot call an experiment action directly — it goes through `run_in_experiments`.

What a script looks like

Every script is a `name`, an optional `notes` line, and a list of `steps`. Each step names an `action` plus any parameters that action takes. You normally build scripts in the [Script Editor](#), but the YAML is plain text and can be read or edited by hand.

An Experiment Script in `survival_config.yaml`:

```
scripts:
  - name: Standard analysis
    notes: Runs the standard analysis battery once per Focus.
    steps:
      - action: run_in_focuses
      - action: run_analysis
```

A Project Script in `project.yaml`, plus one central Experiment Script that every member can use:

```
scripts:
  - name: batch
    steps:
      - action: run_in_experiments
        script: Standard analysis
      - action: render_publication_figures
      - action: project_report
experiment_scripts:
  - name: Quick look
    steps:
      - action: load_data
      - action: run_in_focuses
      - action: km_curves
    show_ci: true
```

The scripts every file starts with

- Every `survival_config.yaml` is given an Experiment Script called **Standard analysis** (`run_in_focuses`, then `run_analysis`) the first time the file is written without a `scripts:` key.
- Every `project.yaml` is given a Project Script called **batch**: `run_in_experiments` (script *Standard analysis*), `render_publication_figures`, `project_report`. A Batch Run executes this script in each Project unless another one is designated.

Both are written into the file, not hidden in code, so you can see and edit them. Once a `scripts:` block exists it is never re-seeded — an empty list is treated as a deliberate deletion. A Project whose `scripts:` list is empty does not run in a Batch Run.

The app also has a few **built-in** scripts that are available by name even when no file defines them:

Built-in	Level	Steps
Standard analysis	Experiment	<code>run_in_focuses, run_analysis</code>
Standard pipeline	Project	<code>validate_project, run_in_experiments (Standard analysis), render_publication_figures, project_report</code>
Report pipeline	Project	<code>run_in_experiments (Standard analysis), render_publication_figures, project_report</code>

When **Report pipeline** is run by name and no member has curated figure specs, its figure step is dropped before the run starts and the log says so.

How a script name is found

An Experiment Script name — named by a `run_in_experiments` step, or picked on the Hub's **Experiment scripts** card — is looked up in this order:

1. the Project's central `experiment_scripts`;
2. the member's own `scripts`: in its `survival_config.yaml`;
3. the built-in Experiment Scripts.

The Hub card lists them in the same order: central scripts first, then the experiment's own scripts whose names no central script uses, then the built-ins. So when a member and the Project both define a script of the same name, the **central** copy runs everywhere.

A named Project Script is looked up in the Batch's central `project_scripts`: (Batch Runs only), then the Project's own `scripts`:, then the built-ins.

Three kinds of problem, three behaviours

- **Unknown action — the script refuses to start.** A step naming an action that does not exist at that level (a typo, or a project action in an Experiment Script) is a hard error before the first step runs. A silently skipped step would produce a report that looks complete and is not. In a Batch Run the Project is logged as failed and the Batch continues.
- **Not Applicable — recorded, the run goes on.** A real action that the current Focus's shape does not admit (a log-rank test on a single-treatment Focus, the Cox factorial model on a Focus with an empty cell) is logged as *Not applicable — reason*, written into that Focus's Run Summary when the script analysed that Focus, and the next step runs. See [Not Applicable and Left Out](#).
- **Failure — that Focus or member stops, the others continue.** Any other error stops the current Focus only; the remaining Focuses still run, and the error is raised at the end so the member counts as failed. One level up, `run_in_experiments` carries on with the next member.

A **Blocked** Focus is reported and skipped; the member's other Focuses run normally.

Scripts and the Hub's tick boxes

The `run_analysis` step runs exactly what the Hub's **Run analysis** button runs: the Analysis Set and Plot Set, less anything the config's `omit`: block leaves out (the boxes you unticked on the Analyze and Plots panels). So a script, a Batch Run and the Hub button produce the same report. See [The omit: section](#).

See also

- [Running Experiment Scripts](#)
- [Project Scripts](#)
- [The Script Editor](#)
- [Script action reference](#)
- [Batch runs](#)

Running Experiment Scripts

An Experiment Script runs a list of steps against one Experiment Directory. Every step runs under a **Focus**, so a script can analyse one slice of the data file or repeat itself for every Focus the config declares.

In the app

1. Load an experiment (double-click it in the Project panel's members table, or open an Experiment Directory).
2. Open the Experiment tile and choose the **Scripts** sub-tile. The **Experiment scripts** card appears.
3. Pick a script in the drop-down. The list holds the Project's central `experiment_scripts:`, then the experiment's own scripts (those whose names no central script already uses), then the built-in **Standard analysis**. A name both define runs the **central** script — the same rule `run_in_experiments` and a Batch Run use.
4. Click **Run script**.

The run happens in the background and its progress is written to the log, one line per step ([2/3] Run analysis). Only one task runs at a time; if another is still going, the Hub says so.

Edit scripts... opens the [Script Editor](#) on the loaded experiment. When you save there, the Hub re-reads the experiment's config, so the drop-down — and the next run — use the scripts as saved.

Experiment Scripts also run without the Hub: a Project Script's `run_in_experiments` step runs a named Experiment Script in every member, and a Batch Run does the same across Projects. See [Project Scripts](#).

Which Focus the steps run under

What happens depends on whether the script contains a `run_in_focuses` step.

Without `run_in_focuses`

- From the Hub, every step runs once, under the **Active Focus** chosen in the Experiment panel.
- Unattended (from a Project Script or a Batch Run), there is no Active Focus, so the whole script runs once **per declared Focus**. An unattended run that analysed one Focus and said nothing about the others would be a silent gap.

With `run_in_focuses`

The script is split at that step:

- Steps **before** it run once. From the Hub they run under the Active Focus; unattended they run with no Focus (a `run_analysis` placed here analyses every Focus).
- Steps **after** it run once for each Focus — every declared Focus when its `only` parameter is blank, or just the Focuses it names. A name in `only` that the experiment does not declare is logged and skipped.

When there is more than one Focus, the log marks each with a line such as — Focus 'diet' —. A script may contain at most one `run_in_focuses` step.

What happens per Focus

For each Focus the runner cuts that Focus's slice from the loaded data, then:

- **Blocked Focus** — if the Focus cannot run as declared (for example it names a level the data does not contain), it is logged as `BLOCKED` with the reasons and skipped. The other Focuses still run.
- **Not Applicable step** — a step whose requirement the Focus Shape does not meet is logged as `Not applicable - <action>: <reason>` and the next step runs. If this run analysed the Focus (a `run_analysis` or `report` step produced a result), the entry is also added to that Focus's Run Summary under `not_applicable`.
- **Failing step** — any other error stops this Focus (`FAILED: ...`) and the runner moves to the next Focus. When all Focuses have had their turn, the script reports the failures together, so a Project counts the member as failed.

What a script writes

Only some steps write files:

- `run_analysis` (and `report`, when nothing has been analysed yet) runs the full battery and writes `analysis/<focus>/:` the tables, the figures, the report (`<experiment>_report_<focus>.pdf` and `report_<focus>.md`) and `run_summary_<focus>.json`.
- `render_publication_figures` writes curated figures to `figures/<focus>/`.

The individual analysis and plot steps (`log_rank_pairwise`, `cox_ph`, `km_curves`, `hazard_plot` and so on) are quick looks: tests print their tables to the log and plots open as tabs in the Hub's plot area. **They save nothing to disk and do not appear in the report**, and they do not consult the `omit: block`. Use `run_analysis` for results you want to keep.

Pitfalls

- **Load before you look.** The individual analysis and plot steps need data in memory. Put a `load_data` step before them (before or after `run_in_focuses` both work). `run_analysis` loads the data itself and needs no `load_data` step.
- **filter does not reach run_analysis.** A filter step narrows the in-memory data for the quick-look steps that follow; `run_analysis` reloads from the data file under the Focus. To analyse a subset properly, declare a Focus for it.
- **Exclusions from a script add to the config's.** `apply_exclusions` removes a group's chambers on top of the config's active Exclusion Group, and the run's report and Run Summary name both groups (`base + extra`). Because the config alone no longer describes that run, the Hub then shows its results as Out of Date.
- **Scripts with the same name.** When the Project's central `experiment_scripts:` and a member both define a script of the same name, the central one runs — from the Hub, from `run_in_experiments` and in a Batch Run alike. Rename the member's copy to keep both.

See also

- [Experiment and Project Scripts](#)
- [Script action reference](#)

- [The Script Editor](#)
- [What a Focus is](#)
- [Not Applicable and Left Out](#)
- [The Run Summary](#)

The Script Editor

The Script Editor builds and edits saved scripts without typing YAML. You choose actions from a list, arrange them as steps, fill in each step's parameters in a form, and save. The editor writes the scripts into the right section of the right file and leaves the rest of that file alone.

Opening it

Click **Edit scripts...** on either:

- the **Experiment scripts** card (Experiment tile → Scripts), or
- the Project panel's **Scripts** card.

From the Experiment scripts card the editor opens on the loaded experiment (or, with none loaded, on the selected folder). From the Project panel's Scripts card it always opens on the Project's `project.yaml`, at the *Project scripts* level.

The three levels

The **Level** drop-down at the top chooses which set of scripts you are editing:

Level	File and section	Actions offered	Available when
Experiment scripts	<code>survival_config.yaml</code> → <code>scripts:</code>	experiment actions for the experiment's type	an experiment is open
Project scripts	<code>project.yaml</code> → <code>scripts:</code>	project actions	the directory is, or sits inside, a Project
Central experiment scripts	<code>project.yaml</code> → <code>experiment_scripts:</code>	experiment actions for the Project's type	the directory is, or sits inside, a Project

Central experiment scripts are one recipe that serves every member without being copied into each `survival_config.yaml`. The label at the right of the control row shows the target file and the action set, for example `survival_config.yaml · core U Standard Lifespan`.

Switching level re-reads that level's scripts from disk. If you have unsaved edits, the editor asks before discarding them.

The window

- **Top bar** — **+ New script** (asks for a name), **- Delete script** (asks for confirmation) and **Save**.
- **Active script** — the drop-down of scripts at this level, with **Rename...** and **Validate** beside it.
- **Actions** (left) — every action this level offers, grouped under headers (LOAD, ANALYZE, PLOTS, QC, SCRIPTS, AI, TOOLS) and sorted by title. Hover for a description. **Double-click** an action to append it to the end of the active script as a new step, pre-filled with its default parameter values. You need an active script first.

- **Steps** (centre) — one card per step: its number, icon, action title (as this level names it, in its category's colour) and a short summary of its parameters. A step naming an action this level does not provide shows its raw key in grey, with a tooltip saying the script will refuse to start. Each card has **up**, **down** and **delete** buttons. Click a card to select it.
- **Parameters** (right) — the form for the selected step: the action's title and description, then one field per parameter. Changes apply as you type. A field that depends on another is greyed out until it applies (for example *Smoothing* σ on a Hazard rate step is enabled only when *Smoothed* is ticked).
- **Live YAML preview** (bottom) — the current level's scripts exactly as they will be written. It is read-only.

Parameter fields

Field	Used for	How to fill it
Tick box	on/off options	tick or untick
Number box	numbers (σ , τ)	type or use the arrows; the allowed range is enforced
Drop-down	fixed choices (format, metric, provider)	pick one
Factor drop-down	a single factor (Filter rows)	lists the factors found in the experiment's data file
Text	names, values	type
Comma list	several names (treatments, Focuses, members, factors)	separate with commas; blank means "all"
Path	a folder	type it or click ... to browse

Empty fields are not written to the YAML, so the action's default applies.

Validate and Save

Validate checks that every step names an action this level provides (and, for experiment levels, that `run_in_focuses` appears at most once). It does not check parameter values or whether a step suits a particular Focus — that is decided per Focus at run time, as **Not Applicable**. A script that fails validation would refuse to start.

Save writes every script at the current level into its section of the file — `scripts:` or `experiment_scripts:` — and leaves every other key in that file untouched. A message confirms how many scripts were written and where. When the editor was opened from the Hub, the Hub then re-reads the Project and the loaded experiment, so its **Run script** drop-downs list the saved scripts and the next run uses them.

Closing the window with unsaved edits asks whether to **Save**, **Discard** them, or **Cancel** and keep editing.

Pitfalls

- Saving replaces the whole section with what the editor shows, including scripts you deleted.
- A script edited here is run from the Hub's **Run script** buttons, from `run_in_experiments`, or from a Batch Run — the editor itself does not run scripts.
- An editor opened from the Hub tells only that Hub about its saves. If you edit a config

by hand, re-open the Project (picking it again re-reads it from disk).

See also

- [Experiment and Project Scripts](#)
- [Script action reference](#)
- [Running Experiment Scripts](#)
- [Project Scripts](#)

Script action reference

Every action a script step can name, with its parameters. In YAML a step is the action key plus its parameters:

```
- action: hazard_plot
  smoothed: true
  sigma: 3.0
```

A parameter left out takes its default. "List" parameters accept either a YAML list or a comma-separated string (`treatments: A, B`); blank means "all". Actions marked **Requires** are offered only when the Focus Shape admits them; reached anyway, the step is **Not Applicable** for that Focus — logged, recorded, and the script carries on (see [Not Applicable and Left Out](#)). A step naming an action not listed for its level stops the script before it starts.

Experiment actions

These are available in Experiment Scripts and central experiment scripts. The Standard Lifespan type offers all of them.

Summary

Action	Title in the editor	Category	Parameters	Requires	Writes files?
load_data	Load data	Load	—	—	no
apply_exclusions	Apply exclusions	Load	group	—	no
filter	Filter rows	Load	factor, value	—	no
run_in_focuses	Run in focuses	Scripts	only	—	no
run_analysis	Run analysis	Analyze	—	—	yes
report	Generate report	Tools	output_dir	—	yes
render_publication_figures	Render publication figures	Plots	format	—	yes
chamber_overlay_qc	Chamber QC overlay	QC	treatment	—	no
km_curves	KM curves	Plots	treatments, show_ci, with_risk_table	—	no

nelson_aalen	Nelson-Aalen	Plots	treatments	—	no
hazard_plot	Hazard rate	Plots	smoothed, sigma	—	no
mortality	Mortality (qx)	Plots	—	—	no
forest_plot	Hazard-ratio forest	Plots	—	comparison	no
faceted_km	Faceted KM	Plots	show_ci	factorial plot	no
interaction_plot	Lifespan interaction plot	Plots	metric	factorial plot	no
log_rank_pairwise	Log-rank pairwise	Analyze	—	comparison	no
log_rank_omnibus	Log-rank omnibus	Analyze	—	comparison	no
gehan_wilcoxon	Gehan-Wilcoxon pairwise	Analyze	—	comparison	no
parametric_aft	Parametric AFT models	Analyze	—	—	no
cox_ph	Cox PH (interactions)	Analyze	factors, include_interactions	—	no
rmst	RMST regression	Analyze	factors, tau	—	no
cox_interaction	Cox factorial model	Analyze	—	factorial model	no
rmst_interaction	RMST factorial model	Analyze	tau	factorial model	no

The requirements: **comparison** needs at least two populated treatments in the Focus; **factorial plot** needs a Focus that varies two or more factors (with at least two populated treatments); **factorial model** additionally needs every pair of those factors fully crossed in the populated cells. See [Focus Shape](#).

"Writes files? no" means a quick look: tests print to the log, figures open as tabs in the Hub's plot area (and are discarded in unattended runs). Nothing is saved, nothing goes into the report, and the `omit:` block is not consulted. Only `run_analysis`, `report` and `render_publication_figures` produce lasting output.

Data steps

`load_data` — **Load data**. No parameters. Loads the experiment's data file exactly as its `survival_config.yaml` describes it (format, columns, censoring policy), with the config's active Exclusion Group and any chambers added by earlier `apply_exclusions` steps removed. Factors are discovered from the file. Under a Focus the in-memory data is that Focus's slice; otherwise it is the whole file. Logs the number of individuals and treatments. Needed before the quick-look analysis and plot steps; `run_analysis` does not need it.

`apply_exclusions` — **Apply exclusions.**

Parameter	Type	Default	Meaning
group	text	default	Name of a group in <code>qc/remove_chambers.csv</code>

Adds that group's chambers to the set this run excludes, *in addition to* the config's active Exclusion Group. Every later `load_data`, `run_analysis` and `report` step leaves them out. If data is already loaded and has a chamber column, the chambers are also dropped from it — from the whole loaded file as well as the current Focus's slice, so a Focus cut later under `run_in_focuses` does not get them back. Logs how many chambers the group lists; a name with no saved group excludes nothing.

The report and Run Summary of every analysis the script then runs name **both** groups, joined with `+` (for example `default + extra`), and count the chambers of both. Because the config's active group alone would not reproduce that run, the Hub shows its results as **Out of Date** ("analysed under exclusion group 'default + extra' ..."). To make a group part of every run, set it as the active group on the QC panel instead. See [Exclusion Groups](#).

`filter` — **Filter rows.**

Parameter	Type	Default	Meaning
factor	factor (drop-down)	none — required	Column to filter on
value	text	none	Value to keep (compared as text)

Keeps only the rows where the column equals the value, and logs how many rows remain. Fails if `factor` is blank or not a column of the loaded data. Needs loaded data. It affects only the quick-look steps after it in the same Focus; `run_analysis` reloads from the file and ignores it. To analyse a subset with saved results, declare a Focus instead.

Structure

`run_in_focuses` — **Run in focuses.**

Parameter	Type	Default	Meaning
only	list	blank = every Focus	Focus names to run

Splits the script: steps before it run once, steps after it run once per Focus (continue-on-error). Names in `only` that the experiment does not declare are logged and skipped. At most one per script. See [Running Experiment Scripts](#).

Saved analysis and outputs

`run_analysis` — **Run analysis.** No parameters. The same run as the Hub's **Run analysis** button: the Analysis Set and Plot Set for the Focus, the Factorial Battery when the Focus varies two or more factors, less whatever the config's `omit:` block leaves out. Writes `analysis/<focus>/` — tables, figures, the report and `run_summary_<focus>.json`. Later quick-look steps in the same Focus reuse its data and life tables. Placed where no Focus is in force (before `run_in_focuses` in an unattended run), it analyses **every** declared Focus; a Focus that fails makes the step fail after the others have run. See [The Analyze panel and Run analysis](#).

`report` — **Generate report.**

Parameter	Type	Default	Meaning
output_dir	folder	blank	A folder to write a copy of each report into as well; blank = only <code>analysis/<focus>/</code>

If an earlier `run_analysis` in this Focus produced a result, rewrites that Focus's report (PDF and Markdown) from it. Otherwise it runs the full analysis — which writes the report — for the step's Focus, or for every Focus when none is in force. The report always goes to `analysis/<focus>/`; with `output_dir`, each report is also written there, under the same Focus-suffixed names (`report_<focus>.md`, `<member>_report_<focus>.pdf`), so several Focuses can share the folder. The analysis outputs themselves stay in `analysis/<focus>/`.

`render_publication_figures` — **Render publication figures.**

Parameter	Type	Default	Choices
<code>format</code>	choice	<code>svg</code>	<code>svg</code> , <code>pdf</code> , <code>png</code>

Renders the curated Publication Figures (those saved in `plot_specs.yaml` from the Plot Editor) into `figures/<focus>/<plot>_<focus>.<format>`, for the step's Focus — or, with no Focus in force, for each Focus whose saved results are current (not analysed and Out of Date Focuses are named and skipped). With nothing curated it renders nothing and says so. `svg` and `pdf` keep text editable. See [Publication Figures, Specs and Styles](#).

Quality control

`chamber_overlay_qc` — **Chamber QC overlay.**

Parameter	Type	Default	Meaning
<code>treatment</code>	text	<code>blank = every treatment</code>	One treatment to draw

Draws one figure per treatment, in the Focus's display order, overlaying the Kaplan-Meier curve of each chamber, so an aberrant vial stands out. The time axis carries the experiment's time label. Chambers excluded by earlier `apply_exclusions` steps are marked. If the data has no chamber column (long-format CSV), it logs that and does nothing. See [The Chamber QC viewer](#).

Quick-look figures

These draw a Focus the way the saved figures do: the experiment's time-axis label (`global.time_label`, or `Age (<unit>)`), and the Focus's treatment order, display names and colours. The forest uses the display names too.

`km_curves` — **KM curves.**

Parameter	Type	Default	Meaning
<code>treatments</code>	list	<code>blank = all</code>	Treatments to draw
<code>show_ci</code>	on/off	<code>on</code>	Draw 95% confidence bands
<code>with_risk_table</code>	on/off	<code>off</code>	Add the number-at-risk table beneath

See [Kaplan-Meier curves](#).

`nelson_aalen` — **Nelson-Aalen.** `treatments` (list, `blank = all`). Cumulative hazard by treatment. See [Nelson-Aalen cumulative hazard](#).

`hazard_plot` — **Hazard rate.**

Parameter	Type	Default	Meaning
<code>smoothed</code>	on/off	<code>off</code>	Kernel-smoothed hazard instead of the raw interval hazard
<code>sigma</code>	number, 0.1–20	<code>3.0</code>	Smoothing width (the same default as the Plot Set's smoothed hazard figure); used only when <code>smoothed</code>

			is on
--	--	--	-------

See [Hazard rate](#) and [Smoothed hazard](#).

mortality — **Mortality (qx)**. No parameters. Interval mortality probability. See [Mortality \(qx\)](#).

forest_plot — **Hazard-ratio forest**. No parameters. Requires comparison. Computes the pairwise hazard ratios and draws them; if there are none, logs that and draws nothing.

faceted_km — **Faceted KM**. show_ci (on/off, default off). Requires factorial plot. One panel per level of the Focus's first varying factor, curves coloured by the others. Needs a Focus. See [Faceted Kaplan-Meier](#).

interaction_plot — **Lifespan interaction plot**. metric (choice median or mean, default median). Requires factorial plot. Kaplan-Meier median (or restricted mean) lifespan per cell with its 95% interval, censoring-aware, the first varying factor on x; non-parallel lines suggest interaction. Needs a Focus. See [Lifespan interaction plot](#).

Quick-look statistics

These print their result tables to the log.

- log_rank_pairwise — Mantel-Cox log-rank test for every pair of treatments. Requires comparison. See [Log-rank pairwise](#).
- log_rank_omnibus — one k-sample log-rank test across all treatments. Requires comparison. See [Log-rank omnibus](#).
- gehan_wilcoxon — Gehan-Wilcoxon (early-weighted) test for every pair. Requires comparison. See [Gehan-Wilcoxon pairwise](#).
- parametric_aft — fits Weibull, log-normal and log-logistic accelerated-failure-time models to each treatment and logs the AIC table (one row per treatment × model, with ΔAIC, log-likelihood, the model's median and why a treatment or model was not fitted), then the best model per treatment. See [Parametric AFT models](#).

cox_ph — **Cox PH (interactions)**.

Parameter	Type	Default	Meaning
factors	list of factors	blank = the Focus's varying factors	Factors in the model
include_interactions	on/off	on	Fit the pairwise interaction terms; off fits main effects only

Fits a Cox proportional-hazards model on the Focus's data, with each factor's Reference Level as baseline — main effects plus every pairwise interaction, or main effects alone with include_interactions off. Logs n, events, AIC, concordance and the coefficient table, and, when interactions are fitted, the likelihood-ratio interaction test (main-effects model against the interaction model). See [The Cox factorial model](#).

rmst — **RMST regression**.

Parameter	Type	Default	Meaning
factors	list of factors	blank = the Focus's varying factors	Factors in the model
tau	number ≥ 0	0 (= chosen automatically)	Restriction time τ , in the data's time unit

Restricted mean survival time pseudo-value regression with full-factorial interactions; logs τ and the coefficient table. The model always includes every pairwise interaction. (Older scripts may carry an `include_interactions` key on this step; it never had an effect and is now ignored.) See [The RMST factorial model](#).

`cox_interaction` — **Cox factorial model**. No parameters. Requires factorial model. The Factorial Battery's Cox model on the Focus's varying factors: main effects plus every pairwise interaction, relative to the Reference Levels. Logs the formula, the Reference Levels, the interaction likelihood-ratio test and the coefficients.

`rmst_interaction` — **RMST factorial model**. `tau` (number ≥ 0 , default 0 = automatic). Requires factorial model. The RMST companion to the Cox factorial model, with no proportional-hazards assumption. See [Interaction analyses](#).

Project actions

These are available only in Project Scripts (`project.yaml` $\rightarrow$ `scripts:`).

Action	Title in the editor	Parameters
<code>validate_project</code>	Validate project	—
<code>run_in_experiments</code>	Run in experiments	<code>script</code> , <code>only</code>
<code>render_publication_figures</code>	Render publication figures	<code>format</code>
<code>project_report</code>	Project report	<code>formats</code> , <code>with_narrative</code> , <code>provider</code>
<code>generate_ai_narrative</code>	Generate AI narrative	<code>provider</code>

`validate_project` — **Validate project**. No parameters. Checks the Project and every member: that all members share the Project's Experiment Type, each member's own configuration problems, and `defaults:` keys the Project should no longer carry. Any problem **stops the Project Script**, listing the problems. Otherwise logs how many members were validated and any divergence in data-source settings, which is reported but not a failure.

`run_in_experiments` — **Run in experiments**.

Parameter	Type	Default	Meaning
<code>script</code>	text	none — required	Name of the Experiment Script to run
<code>only</code>	list	blank = every member	Member names to run

Runs the named Experiment Script in each member, continue-on-error, with each log line prefixed `[member]`. The name is looked up in the Project's central `experiment_scripts:`, then the member's own `scripts:`, then the built-ins. A member with no script of that name, or a name in `only` that is not a member, is logged, counted as a failure and skipped; a member whose run fails is counted and the next member runs. A script that names an action the member's type does not provide is a hard error that **stops the Project Script**. Ends with `n/m member(s) completed`.

`render_publication_figures` — **Render publication figures**. `format` (choice `svg`, `pdf`, `png`; default `svg`). For each member with curated figure specs, renders them for each of its current Focuses into the member's `figures/<focus>/`. Members with nothing curated are skipped; a Project with nothing curated renders nothing and says so.

`project_report` — **Project report**.

Parameter	Type	Default	Meaning
formats	list	pdf, md	Which files to write (pdf, md)
with_narrative	on/off	off	Generate the AI narrative first (saving it) and include it
provider	choice: blank, anthropic, openai	blank (first configured provider)	The narrative's provider; used only when with_narrative is on

Writes `<project>_report.pdf` and/or `<project>_report.md` in the Project folder from each Focus's saved results. Fails if nothing was written. See [The Project Report](#).

`generate_ai_narrative` — **Generate AI narrative.**

Parameter	Type	Default	Choices
provider	choice	blank (first configured provider)	blank, anthropic, openai

Asks the AI provider for a paragraph per current Focus and an across-Foci paragraph, saves them to `<project>_narrative.json` in the Project folder, and logs how many sections were written and where. Never fails the script: a provider problem is logged and the script continues. It does not put the text in a report; to include the narrative in the Project Report, use `project_report` with `with_narrative: true`. See [The AI narrative](#).

See also

- [Experiment and Project Scripts](#)
- [The Script Editor](#)
- [Running Experiment Scripts](#)
- [Project Scripts](#)
- [Focus Shape — what a Focus is offered](#)

Reports

The experiment report

Every analysis run writes a report for the Focus it analysed: a PDF to read and print, and a Markdown copy of the same content. The report is about exactly one Focus and says so on its cover, so two reports from one data file can never be mistaken for each other. Anything that did not run — because the data could not support it, or because you left it out — is stated in the report rather than silently missing.

In the app

The report is written automatically by:

- **Run analysis** on the Analyze panel (under the Active Focus);
- an Experiment Script's `run_analysis` or `report` step;
- a Project Script or Batch Run that runs those steps in each member.

To open reports, use **View reports** on the Project panel's Actions card, which opens the Project Report and every member report, or open the files directly.

Where it is written

For a Focus named, say, `diet` in the experiment `Exp12`:

File	Content
Exp12/analysis/diet/Exp12_report_diet.pdf	the PDF report
Exp12/analysis/diet/report_diet.md	the same report as Markdown

The PDF and Markdown are built from the same content, so they always agree. If the PDF cannot be produced, the Markdown is still written. Re-running the analysis overwrites both.

The cover

The title is `<experiment> · <focus> – Survival Analysis`, with a short description of the experiment type beneath it. The cover then lists:

Item	Meaning
Focus	the Focus name
Slice	one line describing it: varying factors with levels and reference, any filters (<code>only sex = F</code>), any factors pooled over
Experiment type	e.g. Standard Lifespan
Data file	the input file name
Generated	date and time the report was built
Varying factors	the factors that define the treatments
Treatments, Individuals, Deaths	counts in this Focus
Censored	number and percentage censored
Exclusion group	the active Exclusion Group and how many chambers it actually removed — or "no chambers removed (... listed, none present in this data)", "nothing listed", "none"
Censoring policy	"unaccounted individuals censored" or "unaccounted individuals ignored"

Cover status lines

Up to three short coloured lines summarise the run:

- **Omnibus log-rank** — "treatments differ ($p = \dots$)" when $p < 0.05$, otherwise "no overall difference detected ($p = \dots$)". Shown only when the omnibus test ran.
- **Not applicable** — "N action(s) not applicable to this Focus — listed in the Focus section" (a warning).
- **Left out** — "N analysis or figure(s) left out of this run by choice — listed in the Focus section".

The sections

For the Standard Lifespan type the sections appear in this order. A section with nothing to show is left out entirely.

Focus. A sentence naming the Focus and the slice, then a *Focus definition* table (Factor, Levels in display order, Role — varying, filter or pooled over — and Reference level). A line gives the Focus Shape (e.g. 2×2 , or 2×2 (3 of 4 cells)), the treatments, and any cells the levels imply but the data never held. If some individuals have no recorded level for a factor the Focus names, it says how many were outside the analysis. Then two tables when they apply:

- **Not applicable to this Focus** — each real action the slice could not support, with the reason.
- **Left out of this run** — each analysis or figure you unticked (the config's `omit:`), with the reason. A choice, not a property of the data.

Experiment summary. The per-treatment summary table (individuals, deaths, censored, % censored and related columns) and the observation window and number of chambers. See [Summary statistics](#).

Survivorship figures. Every figure of the Plot Set that was drawn, the **Headline Figure** first (labelled "— headline figure"). Where a figure has been curated in the Plot Editor, the curated version replaces the default one (labelled "— curated figure"); a curated KM figure stands in for both default KM figures. Any Defined Plots follow. See [Defined Plots](#).

Factorial analysis. Only for a Focus that varies two or more factors. For each model of the Factorial Battery (Cox factorial, then RMST factorial): the model formula, n, events, concordance (Cox), τ , R^2 and the robust F test (RMST) and AIC where available, a sentence giving the RMST model's τ , the interaction likelihood-ratio test against the main-effects model, the coefficient table (relative to the stated Reference Levels), for Cox the Schoenfeld proportional-hazards check, and a highlighted table of any warnings raised while fitting the model. A model that could not be fitted says why. If the battery did not run, a single line says either "Factorial Battery: not run — reason" or "Interaction analyses: left out of this run". See [Interaction analyses](#).

Lifespan statistics. Median survival (with its 95% confidence interval) and mean survival (restricted to the common τ , stated in the caption) tables, lifespan statistics by treatment and (when two or more factors vary) by factor level, and survival quantiles. See [Summary statistics](#) and [Life tables](#).

Survival comparisons. The omnibus log-rank test (χ^2 , df, p and stars), then the pairwise log-rank, pairwise Gehan-Wilcoxon and pairwise hazard-ratio tables, pairs in the Focus's treatment order. Pairwise test rows are highlighted when they are significant after the Bonferroni adjustment (`p_bonferroni` < 0.05, the same rows as `significant_0.05`). The hazard-ratio caption says which way each ratio points: group1 / group2, group1 being the treatment earlier in the Focus's order.

Data quality. Which chambers were excluded, by id (and via which group), or "No chambers were excluded from this analysis" (saying so when the group lists chambers this data file does not have). Then, when they apply: a highlighted table of warnings the loader reported about the data file; a highlighted table of chambers with fewer individuals than `global.min_n_per_chamber` (flagged for review, not excluded); and the **Parametric model fits (AFT)** table — per treatment and model, AIC, Δ AIC, the model's median and a note, the best model highlighted and any treatment or model not fitted listed with the reason. See [Parametric AFT models](#).

Reading the numbers

- p-values are shown to four decimals, or < 0.0001, in every table. Stars: *** p < 0.001, ** p < 0.01, * p < 0.05, ns otherwise. Pairwise p-values are as the tests report them, beside their Bonferroni-adjusted values; see [Reading p-values](#) for multiple comparisons.

- — means a value is missing or could not be computed.
- Long tables show their first 200 rows, with a note saying so.

Pitfalls

- The report describes the run that wrote it. If you change the Focus or Exclusion Group afterwards, the Hub marks the results **Out of Date**; re-run to refresh the report.
- The experiment report does not include the AI narrative; that appears only in the [Project Report](#).

See also

- [The Run Summary](#)
- [The Project Report](#)
- [Not Applicable and Left Out](#)
- [Where results are written](#)
- [The Analyze panel and Run analysis](#)

The Run Summary

The Run Summary is a small JSON file written at the end of every analysis run, one per Focus. It records what the run analysed and how: the counts, the exact Focus definition, the Exclusion Group, the figures written, what was Not Applicable or Left Out, and the headline test results. The Hub and the Project Report read it instead of re-analysing, which is what makes "not analysed" and "Out of Date" visible states, and what makes building a Project Report fast.

You rarely need to open it, but it is plain text and is the authoritative record of a run.

Where it is written

`<experiment>/analysis/<focus>/run_summary_<focus>.json` — for example `Expl2/analysis/diet/run_summary_diet.json`. It is written by every **Run analysis**, every `run_analysis` script step and every Batch Run, after the report. Re-running the Focus overwrites it. Because there is one per Focus, analysing a second Focus never overwrites the first.

What it contains

Key	Meaning
<code>analyzed_at</code>	date and time of the run (to the second)
<code>input_file</code>	name of the data file
<code>data_sha256</code>	the SHA-256 hash of the data file's contents, so an edited data file can be told from the one analysed
<code>experiment_type</code>	the Experiment Type key, e.g. <code>standard_lifespan</code>
<code>focus</code>	the Focus record (see below)
<code>factors</code>	the Focus's varying factors
<code>discovered_factors</code>	every factor found in the data file
<code>n_total</code>	individuals in this Focus
<code>n_deaths</code>	deaths

n_censored	censored individuals
n_treatments	number of treatments
n_chambers	number of chambers
time_min,time_max	the observation window
assume_censored	the censoring policy: <code>true</code> when individuals unaccounted for at the last census are censored
exclusion_group	name of the config's active Exclusion Group, or <code>null</code>
excluded_chambers	the chambers actually removed, as a sorted list of ids (text): those the exclusions listed and the data file holds. Empty for data with no chamber identities
n_excluded	how many chambers were actually removed (the length of <code>excluded_chambers</code>) — 0 for data with no chamber identities, whatever the group lists
n_excluded_listed	chambers the exclusions listed, including any the data file never had
omit	the <code>omit</code> : selection the run applied: <code>{analyses: [...], plots: [...]}</code> , sorted ids
load_warnings	what the loader reported about the data file without refusing it (for example a chamber recording more deaths and censorings than its <code>SampleSize</code>)
min_n_per_chamber	the <code>global.min_n_per_chamber</code> threshold in force (0 = off)
small_chambers	list of <code>{chamber, treatment, n}</code> for each chamber of this Focus with fewer individuals than the threshold — flagged, not excluded
figures	figure id → file name in <code>plots/</code> , for each figure drawn
defined_plots	Defined Plot name → file name, for each drawn
not_applicable	list of <code>{action, reason}</code> — real actions this Focus could not support
left_out	list of <code>{id, item, reason}</code> — analyses and figures left out by the config's <code>omit</code> :
omnibus_lr	the omnibus log-rank result (<code>chi2, p_value, df, groups</code>), or <code>{}</code> when it did not run
parametric_models	the parametric AFT comparison, one record per treatment × model with <code>treatment, model, aic, delta_aic, log_likelihood, median_survival, best</code> and <code>note</code> (the reason, for a treatment or model not fitted); <code>[]</code> when the analysis was left out. See Parametric AFT models
factorial_models	one entry per Factorial Battery model (see below); empty when the battery did not run

The focus record

Key	Meaning
name, slug	the Focus name and the form used in file names
definition	the analytic definition : <code>factors</code> (each with its ordered levels) and <code>reference</code> (each factor's Reference Level)
description	the one-line description of the slice shown in reports
filters	factors held at a single level
pooled_over	factors in the data that the Focus does not name
shape	the Focus Shape, e.g. 2×2, 2×2 (3 of 4 cells) or single cell
treatments	the populated treatments
absent_cells	cells the levels imply but the data never held
headline	the Headline Figure's id
unassigned	individuals with no level for a factor the Focus names, so outside the analysis

Each `factorial_models` entry

`title`, `model_type`, `formula`, `error` (set when the model could not be fitted), `n_subjects`, `n_events`, `concordance`, `AIC`, `reference` (the Reference Levels), `warnings` (anything the fit or the PH test warned about; an empty list when nothing did), and `lr_interaction` — for the Cox model, the likelihood-ratio test of the interaction model against the main-effects model: `statistic` (the χ^2 statistic, also kept as `lr_stat` for older readers), `df`, `p_value`, the two log-likelihoods (`ll_main`, `ll_interaction`), `concordance_main` and the interaction terms tested (`interaction_cols`). For the RMST model, `tau` (the restriction time used), `rmst_overall` (the pooled RMST θ), `r_squared`, `f_statistic` and `f_p_value`; these are null for the Cox model. The coefficient tables are saved beside it as CSVs in `statistics/`.

Summaries written by earlier versions lack some of these keys (`excluded_chambers`, `parametric_models`, `statistic`, `tau`, ...). Readers fall back: a missing χ^2 statistic is read from `lr_stat`, and a report built from a summary with no `excluded_chambers` gives the count of excluded chambers only.

How it is used

- **Out of Date.** The Hub compares the recorded `focus.definition` and `exclusion_group` with what the config declares now, and, where this file recorded them, `data_sha256`, `excluded_chambers`, `assume_censored` and `omit` with the data file, exclusions and settings as they are now. If any of them changed, the Focus is **Out of Date**: the Hub says so, and the Project Report shows no results for it until it is re-run. Display names and colours are not part of the comparison. A summary with no recorded definition (from before Focuses existed) is always Out of Date. See [Analysed](#), [Out of Date](#), [Blocked](#) and [Orphaned results](#).
- **Not analysed.** A Focus with no Run Summary is "not analysed"; the report never analyses it on your behalf.
- **The Project Report** rebuilds each Focus's section from this file plus the saved CSVs and figures, and fills the Focus Inventory from it.
- **The AI narrative** reads the same saved numbers.

Not Applicable entries from scripts

When an Experiment Script runs `run_analysis` for a Focus and a later step in the same Focus is Not Applicable (say, a `cox_interaction` step on a Focus with an empty cell), that step is added to this file's `not_applicable` list. Duplicates are not added. A script that did not analyse the Focus itself never edits its Run Summary.

Pitfalls

- Do not edit the file by hand: a changed definition will be compared against the config, and hand-edited counts will appear in the Project Report as if computed.
- Renaming a Focus by hand in the YAML leaves its folder, and this file, behind as Orphaned Results. Rename Focuses in the Focus window instead.

See also

- [The experiment report](#)
- [The Project Report](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Not Applicable and Left Out](#)
- [Where results are written](#)

The Project Report

The Project Report binds the saved results of every Focus in every Member Experiment into one document. It **combines nothing**: each Focus keeps its own independent section, built from that Focus's own saved analysis, because a Project never pools data across members. A cover and a Focus Inventory give the reader a map of the whole Project, and a Divergence Note warns when members were processed under different data-source settings.

The report never analyses anything itself. A Focus that has not been analysed, is Blocked, or whose results are Out of Date gets a section saying so, not results.

In the app

- Project panel → **Actions** card → **Project report**.
- Experiment tile → **AI** → **Project report with narrative** (the same report with the [AI narrative](#) included).
- The `project_report` step of a Project Script — the last step of the default `batch` script, so every Batch Run ends with it. See [Project Scripts](#).

View reports on the Actions card opens the Project Report together with every member report.

Before building it, analyse the members (Run analysis per Focus, or run the `batch` script).

Building the report is fast because it only reads files.

Where it is written

In the Project folder, named after it: `<project>/<project>_report.pdf` and `<project>/<project>_report.md`. Each build overwrites the previous one. If the PDF cannot be produced the Markdown is still written and the log says why.

What it contains

Cover

Title `<project>` – Project Report, with the Project's question (from `project.yaml`) as the subtitle, then: Experiment type, Members, Focuses (the total number across members),

Generated (date and time) and Path.

Status line: "N of M Focus(es) analysed and current." — green when all are, amber otherwise. With no resolvable Focus it reads "No Focus could be resolved for any member."; with no members, "This Project has no Member Experiments yet."

Focus Inventory

One row per Focus of every member, in member order then the order the Focuses are declared:

Column	Meaning
Member	the Member Experiment
Focus	the Focus name
Slice	the one-line description: varying factors with levels and Reference Levels, filters, factors pooled over
N, Deaths, Censored	from the Run Summary (Censored as count and %); — when not analysed
Treatments	number of treatments
State	analysed, not analysed, out of date or blocked
Not applicable	the actions recorded as Not Applicable for that Focus

Rows not in the `analysed` state are highlighted. The caption reminds the reader that nothing is pooled across Focuses or members.

Divergence Note

Shown only when members differ in **data-source** settings — the censoring policy, the active Exclusion Group, the time unit. A paragraph explains that this is legal but changes how results are computed, followed by a table of each setting and how members differ. Differences in design (factors and levels) are not listed here; the Focus Inventory already shows every design side by side.

Across Focuses (optional)

When the report is built with the AI narrative and more than one Focus was summarised, an **Across Focuses** paragraph follows, captioned *"Qualitative summary only — no statistic in this report combines Focuses or members."*

One section per Focus

Each section starts on a new page titled `<member> · <focus>`, with the Focus's description beneath. What follows depends on its state:

- **Blocked** — "Focus ... is blocked", with a table of the reasons. The member's other Focuses are unaffected.
- **Not analysed** — "Focus ... has not been analysed. Run its analysis and rebuild this report; nothing was computed on its behalf here." If the member has configuration problems they are listed under *Why it cannot be analysed as configured*.
- **Out of date** — the results are not shown; the reasons (what changed since the analysis) are given, with a request to re-run.
- **Analysed and current** — the AI paragraph for this Focus (if any), then the same sections as the [experiment report](#): Focus, Experiment summary, Survivorship figures, Factorial analysis, Lifespan statistics, Survival comparisons, Data quality. They are rebuilt from the saved Run Summary, CSV tables and figure files, and curated Publication

Figures replace the default figures exactly as in the member report. If one section cannot be rebuilt, a one-line note replaces it and the rest of the report is still written.

Differences from the member reports

The per-Focus sections are rebuilt from what is saved on disk, so they show what the run recorded rather than recomputing anything:

- The parametric AFT table is rebuilt from the Run Summary's `parametric_models` records, so it matches the member report row for row.
- Data quality lists the excluded chambers by id from the Run Summary's `excluded_chambers`. A Run Summary written by an earlier version recorded only how many chambers were excluded; for those, the section gives the count alone and asks for a re-run to list them. No chamber id is ever made up.
- Chambers below `global.min_n_per_chamber` and any data-file warnings are the ones the run recorded, not re-checked against the current data.

Pitfalls

- An old report is not updated automatically. Rebuild it after re-running analyses.
- A Focus analysed before its definition or Exclusion Group was changed is shown as Out of Date, with no numbers. Re-run it.
- Orphaned Results (folders under `analysis/` that no declared Focus names) are never bound into the report.

See also

- [The experiment report](#)
- [The Run Summary](#)
- [The AI narrative](#)
- [Project actions — reports, figures, Plot Editor](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [project.yaml](#) and [Project Defaults](#)

The AI narrative

The AI narrative is an optional, AI-written prose summary for the Project Report: one paragraph for each analysed Focus, plus one closing paragraph on where the separate Focuses agree and disagree. It **summarises numbers the app has already computed and saved**; the AI never sees your raw data and never performs an analysis of its own. It is off unless you ask for it, and if the AI service fails you lose the narrative, never the report.

Setting up a provider

Two providers are supported. The app uses whichever has an API key set:

Provider	Environment variable	Model used	Python package
Anthropic	<code>ANTHROPIC_API_KEY</code>	<code>claude-opus-5</code>	<code>anthropic</code>
OpenAI	<code>OPENAI_API_KEY</code>	<code>gpt-5.2</code>	<code>openai</code>

Set the key either:

- as an environment variable before starting the app, or
- in a file named `.env` in the folder you start the app from (or any folder above it),

containing a line such as `ANTHROPIC_API_KEY=sk-...`

A variable already set in the environment takes precedence over the `.env` file. The model is fixed per provider; it cannot be chosen in the app. Keep your key out of shared folders and version control.

In the app

Open the Experiment tile and choose the **AI** sub-tile. The sub-tile reads "no API key — add one to `.env`" and is dimmed when no provider is configured, and is also dimmed until a Project is loaded — the narrative is written per Project.

The **AI narrative** card shows:

- a status line — "Ready: anthropic, openai" or "No provider configured — set ANTHROPIC_API_KEY or OPENAI_API_KEY in your environment or a `.env` file.";
- **Provider** — the configured providers to choose from;
- **Write narrative** — generates the narrative with the chosen provider, saves it (see *Where it is saved* below) and prints each paragraph in the log, headed by its section name. It is not added to any report;
- **Project report with narrative** — generates the narrative with the chosen provider, saves it, and builds the [Project Report](#) with it.

In scripts, use `project_report` with `with_narrative: true` (and an optional `provider`) to build the report with the narrative, or `generate_ai_narrative` (with an optional `provider`), which generates and saves the text and logs how many sections were written. See [Script action reference](#).

What it writes

- **One paragraph per Focus** (at most about 120 words), for every Focus of every member whose saved results are current. Focuses that have not been analysed, or whose results are Out of Date, are not summarised. In the Project Report it appears at the top of that Focus's section.
- **One "Across Focuses" paragraph** (at most about 100 words), written only when two or more Focuses were summarised. It is placed after the Focus Inventory under the heading *Across Focuses* and captioned *"Qualitative summary only — no statistic in this report combines Focuses or members."*

Every build of a report with the narrative asks the provider again, and the text may differ between builds. The experiment (member) reports never include it.

Where it is saved

Each time a narrative is written it is saved to `<project>_narrative.json` in the Project folder, beside the Project Report, replacing the previous one. The file records when it was written, the provider and model, and each paragraph together with a fingerprint of the Run Summary it summarised.

The narrative is a derivative of the analysis it describes, so **re-running the analysis deletes it**: once a Focus has been re-analysed, its paragraph no longer matches that Focus's Run Summary and is removed from the file the next time the narrative is read, together with the across-Focuses paragraph (which was written from it). When nothing is left, the file is deleted. A paragraph whose Focus has become Out of Date without being re-run stays in the file but is not used, just as the report does not present that Focus's results.

What is sent to the provider

For each Focus, the app sends a compact text **digest** of that Focus's saved results, with instructions. The digest contains only:

- the member and Focus names, and the Experiment Type;
- the Focus's one-line description and its treatment names;
- the varying factors;
- individuals, deaths, censored (count and %) and number of treatments;
- the name of the Exclusion Group, if one is active;
- each Not Applicable action with its reason, and each analysis or figure Left Out by choice (so the AI does not read a missing test as a null result);
- the median survival table;
- the mean survival (restricted mean) table;
- the omnibus log-rank result (χ^2 , df, p);
- the pairwise log-rank table (first 20 rows);
- for each Factorial Battery model: its formula, Reference Levels, interaction likelihood-ratio test and the first 12 rows of its coefficient table.

For the across-Foci paragraph it sends the Project's question (from `project.yaml`) and the per-Focus paragraphs the AI has just written — not the numbers again.

Never sent: the data file or any individual-level records, census counts, chamber identities, file names or paths, life tables, figures, the hazard-ratio, Gehan-Wilcoxon, parametric or proportional-hazards tables, and any Focus whose results are not current. Your API key goes only to its own provider.

The instructions the AI is given

Every request carries the same rules: summarise numbers that have already been computed; never compute, infer or estimate anything; never speculate about mechanism; write plain prose without headings or lists; and if a number is absent, say the analysis does not report it rather than guess. The across-Foci request adds: describe agreement and disagreement qualitatively, and do not combine, average or meta-analyse the numbers or state a combined effect or p-value. The Standard Lifespan type also asks for median and mean lifespan per treatment, which log-rank comparisons were significant, and — when a factorial model is present — each main effect and whether the interaction was significant, relative to the Reference Levels.

When it fails

Everything soft-fails. With no key configured the log says so and the report is built without a narrative. If the provider errors, declines, or its package is not installed, that Focus's paragraph (or the across paragraph) is skipped with a log line, and the rest continues.

Reading it responsibly

- Treat the narrative as a draft summary. Check every number in it against the tables in the same section.
- The across-Foci paragraph is qualitative only. No statistic in the app combines Foci or members.
- A saved narrative describes the runs it was written from. After a re-run, write it again rather than reusing old text — the app deletes the paragraphs a re-run invalidated.

See also

- [The Project Report](#)
- [The Run Summary](#)
- [Script action reference](#)
- [Analysed, Out of Date, Blocked and Orphaned results](#)

Reference

Glossary

The words pySurvAnalysis uses, in plain language, in alphabetical order. Each entry links to the page that explains it in full.

A

Active Focus – The Focus the Hub is currently showing. QC, Analyze, Plots and AI act on it. You choose it in the Focus card of the Experiment panel. Switching it changes nothing on disk, because every Focus's results are kept under their own name. → [What a Focus is](#)

AI Narrative – An optional AI-written summary attached to the Project Report: one paragraph per Focus from that Focus's own numbers, plus a qualitative "across Focuses" paragraph. It only summarises the pipeline's results, never performs its own analysis, and is deleted when the analysis is re-run. → [The AI narrative](#)

Analysis Hub – The main window. A ribbon of three tiles (Batch · Project · Experiment) with a status readout, panels that open under the tiles, and an output area below. The Experiment tile holds the Focus selector and five sub-tiles: QC, Analyze, Plots, Scripts and AI. → [The Analysis Hub](#)

Analysis Set – The optional analyses a run can perform: log-rank pairwise, log-rank omnibus, Gehan-Wilcoxon pairwise, pairwise hazard ratios, parametric AFT models, and the interaction analyses. It excludes any that the Focus is not offered. The survivorship core (life tables, summaries, median and mean survival) is not part of it and always runs. → [The Analyze panel and Run analysis](#)

Assumed Censoring – The DLife convention that individuals still unaccounted for at the end of the census are treated as right-censored (alive when last seen) rather than dead. On by default; can be switched off per experiment. → [Censoring policy](#)

At-Risk Band – In a Publication Figure, the row of number-at-risk counts drawn beneath the curves inside the same plot. Its size and times are set in the Plot Editor. → [Plot Editor — Panels & legend](#)

B

Batch – A folder containing one or more Projects at any depth, used to run many Projects unattended. It holds no analysis of its own and never combines results. → [Batch runs](#)

Batch Preflight – The window every Batch Run opens first. It lists the Projects that will run, their usable and blocked members, and every Blocked Focus, each with the action that fixes it, and then offers Run or Cancel. → [The Batch preflight](#)

Batch Project – One Project a Batch Run can target, identified by its path relative to the Batch folder (for example `Sept2026/ProjA`). → [Batch runs](#)

Batch Run – One execution of a Project Script in every checked Project of a Batch. Each Project's failures are logged and the run continues with the next Project. → [Batch runs](#)

batch script – The Project Script every new `project.yaml` is given. A Batch Run executes it

in each Project unless another script is chosen. → [Batch runs](#)

Blocked Focus – A Focus that cannot run as declared. Either it is *stale* (it names a factor or level the data file does not contain) or it is *empty* (a treatment the data holds has no individuals left after exclusions). The member's other Focuses still run. → [Analysed, Out of Date, Blocked and Orphaned results](#)

Blocked Member – A Member Experiment a run cannot use yet: data with no `survival_config.yaml`, a config with no data file, or several candidate data files with none named. The Project still runs its other members. → [The Batch preflight](#)

C

Chamber – One vial or container of individuals: the unit of the DLife census and the unit an Exclusion Group removes. All individuals in a chamber share a treatment. → [The Chamber QC viewer](#)

Custom Experiment – Retired. It used to mean "no Experiment Type chosen". A config with no `experiment_type` is now simply a Standard Lifespan. → [Experiment Types](#)

D

Defined Plot – A named group of treatments written into the workbook's `DefinedPlots` sheet. It is drawn as a KM figure for every Focus it fits, or recorded as Not Applicable with the missing treatments named. The Focus window can turn one into a Focus. → [Defined Plots](#)

Divergence Note – The Project Report's statement of where members differ in data-source settings (censoring policy, Exclusion Group, time unit). → [The Project Report](#)

E

Exclusion Group – A named set of chambers removed from analysis, stored in `qc/remove_chambers.csv`. One group is active per experiment directory and is shared by every Focus. Its name is stamped on every report and Run Summary. → [Exclusion Groups](#)

Experiment Directory – One data file's folder: `survival_config.yaml`, the data file (at the root or in `data/`), results in `analysis/<focus>/`, QC state in `qc/`, and publication figures in `figures/`. → [Where results are written](#)

Experiment Script – A saved, re-runnable list of experiment-level steps, stored in `survival_config.yaml` or centrally in the Project's `experiment_scripts:`. → [Running Experiment Scripts](#)

Experiment Type – A description of the data source: input shape, time unit, censoring default, quality criteria and report sections. It says nothing about the design. All members of a Project share one. → [Experiment Types](#)

F

Factorial Battery – The interaction analyses (the Cox factorial model with its likelihood-ratio and proportional-hazards tests, and the RMST factorial model) together with the faceted KM and the lifespan interaction plot. Offered when a Focus varies two or more factors. The models need every pair of factors fully crossed. → [Interaction analyses \(Factorial Battery\)](#)

Filter (factor role) – A factor a Focus names with exactly one level. Only individuals at that level are kept, and the factor does not appear in treatment labels. → [What a Focus is](#)

Focus – A named slice of the factors and levels found in the data file. It is the unit of analysis: every output carries its name and every report describes it. Declared in the `focuses:` block of `survival_config.yaml`. → [What a Focus is](#)

Focus Inventory – The Project Report's table of every Focus in every member, with its slice, N,

deaths, censoring, treatments, state and Not Applicable items. → [The Project Report](#)

Focus Shape – Which factors a Focus varies and how many levels each has (for example 2×2), which treatments hold individuals, and where a crossing has a hole. It decides which analyses and figures the Focus is offered. → [Focus Shape — what a Focus is offered](#)

Focus window – The dialog where Focuses are created, edited, renamed, imported and copied. → [The Focus window](#)

H

Headline Figure – The figure that states a Focus's main result and leads its report section: the faceted KM when the Focus crosses factors, otherwise the KM with at-risk table. → [The Plots panel and Generate plots](#)

I

Initialize – Give a folder that already exists the file that makes it a Project (`project.yaml`) or a Member Experiment (`survival_config.yaml`), without moving anything. → [The Experiments card — members](#)

L

Left Out – An offered analysis or figure you have unticked on the Analyze or Plots panel. It is stored in the config's `omit:` block, left out of every run (Hub, script and Batch alike), and recorded as a choice in the log, the Run Summary and the report. → [Not Applicable and Left Out](#)

M

Member Experiment – An experiment folder inside a Project: one data file, analysed on its own and never pooled with its sibling members. → [The Experiments card — members](#)

Minimal Member Config – What *Create experiment...* writes: only what the member must state itself, leaving everything the Project Defaults supply out, so later edits to the Project still reach it. → [The Experiments card — members](#)

N

Not Applicable – A relevant analysis or figure that the data cannot compute, for example the interaction model when one combination of levels has no individuals. Greyed in the Hub with the reason, and recorded in the log, the Run Summary and the report. → [Not Applicable and Left Out](#)

Not offered – An analysis or figure that is not relevant to a Focus's definition, for example the interaction model for a one-factor Focus. It has no checkbox and is not mentioned in the report. → [Focus Shape — what a Focus is offered](#)

O

Orphaned Results – A folder under `analysis/` that no declared Focus names, left behind when a Focus is deleted or renamed by hand in the YAML. Listed in the Focus card with *Adopt...* and *Delete...*, and never included in a report. → [Analysed, Out of Date, Blocked and Orphaned results](#)

Out of Date – A Focus whose saved results were produced from something that has since changed: its definition (factors, levels, level order, Reference Levels), the Exclusion Group or the chambers it removes, the data file's contents, the censoring policy, or what the run left out. The report shows none of those results until the Focus is re-run. → [Analysed, Out of Date, Blocked and Orphaned results](#)

P

Plot Set – The ordered list of figures a run produces for a Focus: the general survivorship figures, minus any the Focus is not offered, plus the faceted KM and interaction plot when the Focus varies two or more factors. → [The Plots panel and Generate plots](#)

Plot Spec – One Publication Figure's content: axis labels and limits, reference line, an optional narrowing to a subset of the Focus's treatments, and the name of its Plot Style. Stored in `plot_specs.yaml`. → [Publication Figures, Specs and Styles](#)

Plot Style – One Publication Figure's look (size, fonts, line widths, markers, panels, legend, colours), owned by that figure. Edited in the Plot Editor. → [The Plot Editor](#)

Pooled over (factor role) – A discovered factor a Focus does not name. Individuals at all its levels are combined. → [What a Focus is](#)

Project – A folder with a `project.yaml` whose subfolders holding a `survival_config.yaml` are its members. Members address one question in different ways and are never pooled. → [Creating and opening a Project](#)

Project Defaults – The `defaults:` section of `project.yaml`: settings members inherit unless they override them (Experiment Type, time unit, censoring, input settings, Exclusion Group name, quality criteria). Focuses are never inherited. → [project.yaml](#) and [Project Defaults](#)

Project Report – `<project>/<project>_report.pdf`: a cover with the question, the Focus Inventory, any Divergence Note, then one independent section per Focus built from its saved results. → [The Project Report](#)

Project Script – A saved list of project-level steps in `project.yaml`. Its `run_in_experiments` step runs an Experiment Script in every member. → [Project Scripts](#)

Publication Figure – A journal-ready vector figure (SVG or PDF) authored in the Plot Editor and rendered from the Project panel. → [Publication Figures, Specs and Styles](#)

R

Reference Level – The level a varying factor's model coefficients are measured against (the Cox and RMST baseline). It defaults to the first level listed and can be set separately from display order. → [Reference Levels](#)

Requirement – What an analysis or figure needs from a Focus: *comparison*, *factorial plot* or *factorial model*. It has two halves, relevance (decided by the definition) and computability (decided by the data). → [Focus Shape — what a Focus is offered](#)

`run_in_focuses` – A script step that runs the rest of an Experiment Script once per Focus (every Focus, or just those listed in `only:`). → [Script action reference](#)

Run Summary – `analysis/<focus>/run_summary_<focus>.json`: the small record of one run under one Focus (counts, the Focus definition, the Exclusion Group, figures written, Not Applicable and Left Out items, key test results). The Hub and the Project Report read it instead of re-analysing. → [The Run Summary](#)

S

Standard analysis **script** – The Experiment Script every new `survival_config.yaml` is given. The Project's `batch` script runs it in each member. → [Running Experiment Scripts](#)

Standard Lifespan – The one Experiment Type: a DLife census workbook of chambers scored to death, with assumed censoring on. Every analysis is reachable from it, gated by Focus Shape. → [Experiment Types](#)

T

Treatment – One cell of a Focus: a combination of levels of its varying factors, written as a label such as Female/20x. Every analysis and figure groups by it. → [What a Focus is](#)

Type Action – A script action an Experiment Type adds beyond the core actions. Script steps may use any core or type action; one the Focus does not admit is Not Applicable rather than an error. → [Script action reference](#)

U

Unfiltered – The Focus every experiment has when its config declares none: every discovered factor at every level. It is written into the config the first time the experiment is analysed or its Focuses are opened in the Focus window. → [What a Focus is](#)

Upgrade – Rewriting a pre-overhaul folder layout into the current one. No longer a Hub button; use *Initialize existing directory...* to bring an old folder into a Project. → [The Experiments card — members](#)

V

Varying (factor role) – A factor a Focus names with two or more levels. Its levels make up the treatment labels. → [What a Focus is](#)

See also

- [Batches, Projects, Experiments and Focuses](#)
- [What a Focus is](#)
- [Troubleshooting](#)
- [What pySurvAnalysis does](#)

Command-line use

Everything the Hub runs can also be run from a terminal, which is useful on a server, in a scheduled job, or to re-run a whole Batch overnight. The command is `pysurvanalysis` (run it as `uv run pysurvanalysis ...` from the repository folder, or `uv run python main.py ...`). With no subcommand it opens the Hub.

The command-line runs read the same configuration files as the Hub — `survival_config.yaml`, `project.yaml`, `batch.yaml` — so a run from the terminal produces exactly what the Hub would: the same Focuses, the same active [Exclusion Group](#), the same analyses and figures left out by the [omit: section](#), written to the same places.

Subcommands

Subcommand	What it does
<code>hub [path]</code>	Open the Analysis Hub, optionally with a Project, Batch folder or Experiment Directory selected.
<code>qc [path]</code>	Open the Chamber QC viewer.
<code>plots path</code>	Open the Plot Editor on an Experiment Directory.
<code>run INPUT [options]</code>	Analyse one experiment (see below).
<code>project path [--script NAME]</code>	Run a Project Script in a Project. Default script: <code>Report pipeline</code> .
<code>batch path [--script NAME]</code>	

	Run a Project Script in every Project found under a Batch folder, continue-on-error. Without <code>--script</code> , the Batch's designation in <code>batch.yaml</code> is used; with none, each Project runs its own <code>batch</code> script. Exit code 1 if any Project failed.
<code>upgrade path [--type KEY] [--dry-run]</code>	Upgrade a directory made by a pre-Project version of the app. <code>--dry-run</code> lists what would change and writes nothing; it never deletes or moves files.

run

`run` takes either an **Experiment Directory** (a folder with a `survival_config.yaml`) or a **bare data file** (`.xlsx`, `.csv`, `.tsv`, or a folder holding one `.xlsx`).

Experiment Directory. Every declared Focus is analysed, each into `analysis/<focus>/` with its report and Run Summary — exactly Run analysis in the Hub, once per Focus. The config is validated first; any problem stops the run with exit code 2 and the list of problems. A [Blocked Focus](#) is reported and skipped; the others still run.

Bare data file. The file is analysed under the **Unfiltered** Focus (every factor varying) into `<stem>_results/Unfiltered/` next to it, or into `--output-dir`. There is no config, so the CSV options below describe the file.

Option	Applies to	Meaning
<code>--focus NAME</code>	Experiment Directory	Analyse only this Focus. Repeat to name several. Naming a Focus that is not declared is an error (exit 1).
<code>--exclusion-group NAME</code>	both	Also exclude this group's chambers, in addition to the config's active group.
<code>--figures</code>	Experiment Directory	Also render the Publication Figures (see Publication Figures, Specs and Styles).
<code>--output-dir DIR, -o DIR</code>	bare file	Where to write the results.
<code>--no-assume-censored</code>	bare <code>.xlsx</code>	Do not treat individuals unaccounted for at the last census as censored (see Censoring policy). For an Experiment Directory the config's <code>global: assume_censored</code> decides.
<code>--format auto/long/wide</code>	bare CSV	The CSV layout (see Input data formats).
<code>--time-col COL, --event-col COL</code>	bare CSV	Column names for age and the 0/1 death indicator. Defaults: Age, Event.
<code>--factor-cols COL ...</code>	bare long CSV	The factor columns.
<code>--factor-names F1 F2</code>	bare wide CSV	The two factor names.
<code>--col-mapping</code>		

FILE.yaml	bare wide CSV	The column-to-group mapping.
-----------	---------------	------------------------------

When it finishes, `run` prints where each Focus's results are, a short summary (treatments, chambers, N, deaths, censored, time range) and every Not Applicable item.

Examples

```
## Analyse every Focus of one experiment
uv run pysurvanalysis run Projects/Diet2026/rep_a

## Only the "Diet x Genotype" Focus, and render its publication figures
uv run pysurvanalysis run Projects/Diet2026/rep_a --focus "Diet x Genotype" --fig

## A loose workbook, outside any Project
uv run pysurvanalysis run census.xlsx -o census_results

## Re-run every Project in a folder with each Project's own batch script
uv run pysurvanalysis batch Projects/
```

Exit codes

0 success; 1 a Focus or Project failed, or a named Focus does not exist; 2 the configuration has problems, or a bare file was given `--focus`, or the Project Script named does not exist.

See also

- [Installing and launching](#)
- [Experiment and Project Scripts](#)
- [Batch runs](#)
- [Where results are written](#)
- [survival_config.yaml reference](#)

Troubleshooting

This page collects the messages pySurvAnalysis shows, grouped by where you meet them, each with its cause and fix. In the Hub, a warning appears in a message box and is copied to the output log with a leading `!`. A task that fails in the background (Run analysis, a script, a Batch Run) is logged with a leading `✖`. Text in angle brackets, like `<name>`, stands for your own folder, file, Focus or factor names.

Finding the data file

`<name>`: no .xlsx/.csv/.tsv found in `<folder>` or its data/ subdirectory. The experiment folder has no data file. Put the workbook or CSV in the folder's data/ subfolder or at its root. (Excel lock files starting `~$` and `remove_chambers.csv` are ignored.)

`<name>`: 2 data files in data (a.xlsx, b.xlsx). Name one with `data_file`: in `survival_config.yaml`. More than one candidate file was found, and the app never guesses. Add `data_file: data/a.xlsx` to the config, or move the extra file out. data/ is searched before the folder root. The Batch preflight shows the same problem as "N data files in data/ (...) — which is the experiment?".

``data_file: <path>`` does not exist (looked at `<full path>`). The file named in `data_file`: is missing. Correct the path (relative paths are relative to the experiment folder).

No input data file found in `<name>` (looked in data/ and the directory root). Shown by *Validate YAMLS* and in the Project Report when a member has a config but

no data. Add the data file.

Members-table row reading missing in the Config column That folder holds no `survival_config.yaml`, so it is not a member yet. Double-click it, or use *Initialize existing directory...*, to scaffold one from the Project Defaults. See [The Experiments card — members](#).

Reading the data

No treatment factor columns found after `StartTime` in Design sheet The DLife workbook's Design sheet has no factor columns to the right of `StartTime`. Factors are discovered from those columns, so add them.

RawData sheet missing columns: [...], or when adding a workbook: `<file>`: RawData sheet is missing AgeH, ..., `<file>`: no 'Design' sheet., `<file>`: Design sheet has no 'Chamber' column. The workbook is not in DLife layout. The Design sheet needs Chamber, SampleSize, StartTime and factor columns after StartTime. RawData needs AgeH, Chamber, IntDeaths and Censored. See [Input data formats](#).

Long-format CSV missing columns: [...] The time or event column named in `input:` (default Age and Event) is not in the CSV. Rename the columns or set `time_col / event_col` in [the input: section](#).

Event column must contain only 0 and 1 values. Use 1 for a death and 0 for a censored individual.

No factor columns found in CSV. Specify `factor_cols` explicitly. / Factor columns not found in CSV: [...] The CSV has no columns besides time and event, or `factor_cols` names columns the file does not have.

Wide format requires exactly 2 factor names. / A wide-format CSV needs `input.col_mapping`, or `input.factor_levels` ... / Could not infer complete wide mapping from column names. ... Provide explicit `col_mapping`. A wide-format CSV needs `factor_names` (two of them) and either `factor_levels` with column names the app can parse, or an explicit `col_mapping`. See [the input: section](#).

Chamber `<id>`: RawData records `<n>` deaths and censorings but its SampleSize is `<s>` ... (log, report's Data quality section) More individuals were scored in that chamber than the Design sheet says were set up. The run uses the recorded count. Correct SampleSize or the RawData rows.

N row(s) dropped: their time or event is blank or not a number. (log, report's Data quality section) Some CSV rows could not be read as an individual. Fix or delete them.

Unsupported file type: '.xls'. Expected .xlsx, .csv, or .tsv. Save the file in one of the supported formats.

Configuration (Validate YAMLS)

`<file>`, line N: not valid YAML — `<problem>`. (`<path>`) The file is not YAML at all: often a tab used for indentation ("YAML indents with spaces, never tabs"), an unclosed quote or bracket, or a key missing its colon. Fix that line. A member with a broken `survival_config.yaml` is listed with this problem (and as a bad config Blocked Member) while the rest of the Project carries on; nothing runs or saves for that member until it is fixed. A broken `project.yaml` stops the Project opening at all: the Hub shows this message when you select it and keeps the folder selected, so **Validate YAMLS** still reports the line to fix. See [Validating configuration](#).

``input.format`` must be `auto`, `excel`, `long` or `wide` (got `'...'`). Fix the value in `input``.

``exclusions:`` must be a mapping, e.g. ``{group: default}``. Write `exclusions: {group: <name>}`.

Unknown `experiment_type` `'...'`. Known types: `standard_lifespan`, or omit the key. Remove the key or correct it. A config with no `experiment_type` is a `Standard Lifespan`. See [Experiment Types](#).

``global.assume_censored`` must be `true` or `false`.

``global.min_n_per_chamber`` must be a non-negative integer (0 turns the check off). Fix the value in [the global: section](#).

``omit.analyses`` names `'...'`, which no run produces. Known: ... A misspelt id in `omit``: would leave nothing out without telling you. Use one of the listed ids, or tick and untick the box in the Hub, which writes the id for you. See [The omit: section](#).

Focus `'<F>'` names no factors – give factors: at least one factor with its levels. Focus `'<F>'`: factor `'<x>'` lists no levels. / ... lists a level twice. Each Focus needs at least one factor, and each factor a non-empty list of distinct levels.

Focus `'<F>'`: reference level `'<lv>'` is not one of `<x>'s` levels (...). / Focus `'<F>'`: reference names factor `'<x>'`, which the Focus does not name. The reference: must name a level the Focus keeps, for a factor it names. See [Reference Levels](#).

Focus `'<F>'` would write to `analysis/plots/`, which is reserved for pre-Focus results; rename it. A Focus cannot be named `plots`, `statistics` or `data_output`.

Focus `'<A>'` and Focus `'<B>'` would write to the same output directory (`<slug>`); rename one. Two names that differ only in spaces or punctuation produce the same folder name. Rename one.

`project.yaml: defaults: focuses:` is ignored – a member never inherits `Focuses`. `Focuses` belong to each member. Remove the block and use *Copy Focuses from...* in the [Focus window](#).

`project.yaml: defaults: factors: predates` `Focuses`. ... An old shared `factors:` block. Members that inherit it become an `Interaction Focus` on their next analysis. Remove it once each member has been analysed.

`<member>` is a ... but the Project's type is ... Every member must share the Project's Experiment Type.

Focuses

Focus `'<F>'` is blocked: **followed by reasons** The Focus cannot run as declared, and nothing was written. The reasons say which of these applies:

- *names factor/level `'...'`, which the data file does not contain ... — rename it to `'...'`?* The Focus is **stale**: the Design sheet changed. Use *Fix selected...* in the Batch preflight, or edit the Focus.
- *treatment `'...'` is in the data file but has no individuals once exclusions are applied.* The active Exclusion Group (or the workbook's chamber flags) removed every chamber of that cell. Change the Exclusion Group or drop that level from the Focus.
- *Focus `'...'` selects no individuals.* Edit the Focus.

See [Analysed, Out of Date, Blocked and Orphaned results](#).

<experiment>: no Focus could be resolved – is the data file readable? The config declares no Focus, and the data file could not be read to build Unfiltered. Fix the data file first.

<experiment> has no Focus named '<F>'. Focuses: ... A script, a run_in_focuses list or a command named a Focus this member does not declare. In run_in_focuses, an unknown name is logged as Focus '<F>': not declared here – skipped.

The Focus window could not read this experiment's data: ... The Focus window needs the data file to discover factors. Fix the error shown, usually one from *Finding the data file* or *Reading the data* above.

Cannot save (**Focus window**) The set of Focuses fails validation (see *Configuration* above), or Two Focuses share a name., or Focus '<F>': factor '<x>' is ticked but none of its levels is ... (tick a level, or untick the factor to pool over it). Correct it. The window stays open.

Save blocked Focuses? (Focus window) A Focus still names a factor or level the data file lacks (shown in red in the window). Cancel and untick it, or Save to keep it as written; it stays Blocked until the names match the data.

analysis/<new>/ already exists; delete or adopt it before renaming '<old>' to '<new>'. A results folder of the new name is already on disk (often an orphan). Delete or adopt it from the Focus card, then rename again.

An experiment needs at least one Focus. You tried to delete the last Focus.

This workbook has no DefinedPlots sheet. *Import from DefinedPlots...* has nothing to import. See [Defined Plots](#).

This Project has no other member. *Copy Focuses from...* needs a second member.

Not copied – they would be blocked here or clash by name: ... Each listed Focus either names a factor or level this member's data file lacks, has a treatment this member's active exclusions would empty, or has a name a Focus here already uses. Copy the others, then create the rest by hand.

There are no orphaned results to adopt – pre-Focus results record no Focus, so they are never adopted. Only folders left by a deleted or hand-renamed Focus can be adopted. Old pre-Focus results can only be deleted.

Every Focus already has results – create or rename one first. /Focus '<F>' already has results. Orphaned results can only be adopted by a Focus with no results folder of its own.

N individual(s) have no recorded level of <factors> and belong to no treatment – outside this Focus. (log) Some individuals have a blank level for a factor the Focus names. They are not analysed. Fill in the Design sheet or CSV if they should be.

"Why is this missing?"

The Interaction analyses box is not on the Analyze panel. The Active Focus varies fewer than two factors, so the interaction analyses are not offered. Vary a second factor, or make a Focus that does. See [Focus Shape](#).

A box is greyed out. Hover over it. *Not applicable to Focus ...* means the data cannot compute it (for example a crossing gap, or only one populated treatment). *Draws from Pairwise hazard ratios* means the forest needs that analysis ticked.

Not applicable – <item>: <reason> in the log, or a *Not applicable to this Focus* table in the report: the item was relevant but the data could not support it. See [Not Applicable and Left](#)

Out.

Left out – `<item>`: unticked in the Hub (...), or a *Left out of this run* table: the box is unticked, so it is in `omit`:. Tick it and re-run. An earlier copy of a left-out figure or table is deleted on purpose.

A Defined Plot is never drawn. It is drawn only for a Focus that pools over nothing and contains every listed treatment the data has. For other Focuses it is silently not relevant. If it is relevant but a treatment is missing, the run records it as Not Applicable with the missing labels. A plot whose labels are not full treatments of the file (the wrong number of / parts), or name nothing the file contains, is recorded as Not Applicable under every Focus. See [Defined Plots](#).

A Focus's section in the Project Report shows no results. The Focus is not analysed, Out of Date or blocked, and the section says which. Re-run the Focus (or fix it) and rebuild the report.

A treatment reads not fitted: ... in the parametric AFT table. It has fewer than 5 individuals or fewer than 2 deaths (only N individual(s), only N death(s)), or that model family failed to converge (fit failed: ...). The other treatments and families are unaffected. See [Parametric AFT models](#).

Data quality says "This run recorded only the count" of excluded chambers. The Focus was analysed by an earlier version, whose Run Summary saved how many chambers were excluded but not which. Re-run the Focus to have them listed by id.

A median's upper confidence limit (`median_ci_hi`) is blank. The upper edge of the KM confidence band never fell to 50%, so the limit is "not reached". The median itself can still exist. See [Summary statistics](#).

A Focus reads out of date and I changed nothing in its definition. Out of Date also notices an edited data file, a change to the chambers the active group lists (within that Focus), a change of `assume_censored`, and boxes ticked or unticked on the Analyze or Plots panels. The reason is in the tooltip. Re-run the Focus. See [Analysed, Out of Date, Blocked and Orphaned results](#).

Pre-Focus results in `analysis/` – never adopted and never replaced by a re-run; `Delete...` removes them. Results from before Focuses existed are still in bare `analysis/`. A re-run writes new results under `analysis/<focus>/` and leaves the old ones in place. Delete them with **Delete...** → *pre-Focus results* on the Focus card.

Scripts and Batch Runs

Step N uses action '`<x>`', which a Standard Lifespan does not provide. Available: ... A typo or unknown action. This is a hard error: the script refuses to start, and in a Batch Run that Project is counted as failed while the Batch continues. A real action that merely does not fit the Focus is *Not Applicable* instead. See [Script action reference](#).

Step N has no action. Give the step an action, or delete it.

`run_in_focuses` appears more than once; ... Keep only one `run_in_focuses` step.

`run_in_focuses` can only run inside an Experiment Script – ... The step was used where it cannot split the script (for example as a one-off action).

`<member>`: script '`<name>`' cannot run as a Standard Lifespan: ... The Experiment Script that `run_in_experiments` named has invalid steps, so the Project's run stops there.

`batch.yaml`, line N: not valid YAML – ... The batch is listed but cannot run until `batch.yaml` is fixed. The Batch folder's `batch.yaml` does not parse. Its Projects are still found and listed, but **Run batch** refuses, and choosing a script in the picker is not saved, until the file is fixed: its designated script and central `project_scripts`: are

unknown, and writing to it would replace your file. Fix the line, then click **Rescan**, which re-reads batch.yaml.

Not saved – <file> could not be read, so it was not overwritten (Script Editor). The survival_config.yaml or project.yaml you are editing no longer parses — perhaps it was edited by hand while the editor was open. Fix the file; the editor never writes over one it cannot read.

[<member>] no Experiment Script named '<name>' – skipped. /<member>: not a member of this Project – skipped. The script name or an only: entry in run_in_experiments does not resolve. That member is counted as failed.

No Experiment Script named '<name>'. /No Project Script named '<name>'. The chosen script no longer exists in the config.

Project validation failed: ... The validate_project step found problems. They are the same ones *Validate YAMLS* shows.

<action>: no data loaded – add a 'Load data' step first. Add a *Load data* step before steps that need data.

Select a directory that holds Projects first. /No Projects checked – check at least one row. A Batch Run needs a folder containing Projects and at least one checked Project in the preflight. See [Batch runs](#).

Projects and members

<folder> is not a Project – no project.yaml. Create one with the Hub's Create/Load card. Open a folder that has a project.yaml, or initialize one.

'<name>' already exists. Use 'Initialize existing directory...' ... *Create experiment...* makes new folders only. Use Initialize for a folder you already have.

'<name>' is already in this Project. Use 'Initialize existing directory...' ... *Add directory...* copies folders in from **outside** the Project.

Every directory in '<project>' already has a survival_config.yaml. ... Nothing to initialize. Use *Create experiment...* for a new member.

<folder> has no data/ subdirectory. /No .xlsx file in <folder>/data. *Add directory...* expects an experiment folder with a workbook in data/.

Create or select a Project first – a member's defaults are inherited from its project.yaml. Member actions need a Project selected.

No reports yet – run Project report first. Build the Project Report before viewing it.

A task is already running. Wait for the current task to finish (watch the log).

Load an experiment first. Double-click a member in the Project panel's members table, or select a standalone experiment folder.

See also

- [Analysed, Out of Date, Blocked and Orphaned results](#)
- [Not Applicable and Left Out](#)
- [Validating configuration](#)
- [Input data formats](#)
- [The Batch preflight](#)
- [Glossary](#)